



**UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO**

Department of Computer Science

Master Degree in Computer Science - AI

Thesis in **Deep Learning**

Towards Cross-Model Transferability of Hallucination Detection and Mitigation in Large Language Models

Student:

Emanuele Fontana

Supervisor:

Dr. Gennaro Vessio

Co-Supervisor:

Lucrezia Laraspata

ACADEMIC YEAR **2025-2026**

Abstract

Large Language Models have demonstrated exceptional capabilities in numerous natural language processing tasks, yet they still suffer from the critical problem of “hallucinations” i.e., the generation of factually incorrect information presented with high confidence. Although analyzing the internal states of models (using probing techniques) has proven effective for detecting these anomalies, and steering techniques can mitigate them at inference time, most current approaches require training classifiers or steering modules specific to each individual architecture, limiting their scalability and general applicability.

This work explores the feasibility of transferable mechanisms for both hallucination detection and mitigation across different LLM architectures. On the **detection** side, we investigate whether and how detection capabilities learned on a “Trainer” model can be transferred to a different “Tester” model, developing and comparing various latent space alignment methodologies, ranging from hybrid approaches to non-linear projections, culminating in the proposed *One-For-All* unified prober. On the **mitigation** side, we adapt and evaluate two steering techniques — *TruthX*, based on contrastive disentangled representation learning, and *SLiMTruth*, an adaptation of State-wise Linear Modulation to hallucination reduction — analyzing their effectiveness in standard, cross-dataset, and cross-model transfer settings.

The results demonstrate that transferring truthfulness-related features for detection is highly effective, especially when using the *One-For-All* approach, and that latent space alignment learned in simpler contexts is surprisingly robust and transferable even to more complex contexts. Regarding mitigation, *SLiMTruth* achieves the best absolute hallucination rate reductions in standard settings (up to -89.9%), while *TruthX* exhibits superior cross-model transferability, particularly on complex tasks. This work highlights the potential of alignment and steering techniques for creating cross-model safety systems for both detecting and mitigating hallucinations, while also outlining the current limitations — particularly the vulnerability of low-error settings to over-intervention and the challenges of cross-model transfer for layer-specific modulation approaches.

Contents

1	Introduction	1
1.1	Goals and Objectives	1
1.2	Thesis Overview	2
2	Background	5
2.1	Transformer Overview	5
2.1.1	Transformer Architecture	6
2.1.2	Self-Attention Mechanism	7
2.1.3	Mechanistic Interpretability of Attention Heads	8
2.2	Large Language Models	8
2.2.1	LLM Lifecycle	9
2.2.2	Modern Architectural Innovations	9
2.3	Hallucinations in LLMs	10
2.3.1	Taxonomy of Hallucinations	10
2.3.2	Empirical Evidence and Model Comparison	11
2.4	Probing	11
2.4.1	Linear Probing	12
2.4.2	Limitations of Correlational Probing	12
2.4.3	Advanced Probing Methodologies	12
2.4.4	Advances in Mechanistic Interpretability of Large Language Models	13
2.5	Steering	13
2.5.1	Core Mechanisms	14
2.5.2	Automated and Causal Steering	14
2.5.3	Steering for Hallucination Mitigation	14
2.5.4	Limitations: Non-Surjectivity and Safety Risks	15
3	Methodology	17
3.1	Dataset	17
3.2	Language Models	18
3.2.1	Llama-3.1-8B-Instruct	19
3.2.2	Gemma-2-9B-IT	19
3.3	Activation	19
3.3.1	Collection	19

3.3.2	Hallucination Statistics	20
3.4	Preliminary Studies	21
3.5	Building the Universal Prober	24
3.5.1	RidgeRegressor	25
3.5.2	Procrustes	26
3.5.3	CKA	26
3.5.4	CCA	27
3.5.5	AdapterMLP	28
3.5.6	Full Non-linear	30
3.5.7	Reduced Non-linear Approach	33
3.5.8	One-For-All Approach	36
3.6	Hallucination mitigation steering	38
3.6.1	TruthX	39
3.6.2	SLiMTruth	47
4	Results and Discussion	53
4.1	Hallucination Detection	53
4.1.1	In-Domain Transfer	53
4.1.2	Cross-Domain Transfer	62
4.2	Hallucination Mitigation	65
4.2.1	TruthX	65
4.2.2	SLiMTruth	69
4.2.3	Comparative Analysis: TruthX vs. SLiMTruth	73
5	Conclusions	75
5.1	Summary of Results	75
5.2	Future Work	76
	Appendices	83
A	Layers' study	85
A.1	F1Score	85
A.2	Auroc Score	86
B	Error type analysis	89
6	Ringraziamenti	91

List of Figures

2.1	Transformer Architecture.	6
2.2	Self-Attention Mechanism Representation.	7
3.1	Hallucination types and corresponding datasets	19
3.2	Pipeline for extraction and saving of internal LLM activations	20
3.3	Accuracy comparison of attention modules across the datasets	22
3.4	Accuracy comparison of hidden modules across the datasets	22
3.5	Accuracy comparison of mlp modules across the datasets	23
3.6	PCA of Attention activations of Gemma-2-9B-IT across all datasets . .	23
3.7	PCA of Attention activations of Llama-3.1-8B-Instruct across all datasets	24
3.8	RidgeRegressor experimental pipeline	26
3.9	Procrustes approach experimental pipeline	27
3.10	CKA approach experimental pipeline	28
3.11	CCA approach experimental pipeline	29
3.12	AdapterMLP architecture	29
3.13	AdapterMLP approach experimental pipeline	30
3.14	MLP Prober architecture of the full non-linear approach	31
3.15	Full non-linear approach experimental pipeline	32
3.16	Autoencoder architecture of the reduced non-linear approach	33
3.17	MLP Prober architecture of the reduced non-linear approach	33
3.18	AlignmentNetwork architecture of the reduced non-linear approach . .	34
3.19	Reduced non-linear approach experimental pipeline	35
3.20	One-For-All encoder architecture	36
3.21	One-For-All Classification Head architecture	36
3.22	One-For-All experimental pipeline	37
3.23	TruthX model architecture overview	40
3.24	TruthX extraction overview	41
3.25	SLIMTruth architecture	47
4.1	Gemma-2-9B-IT as Trainer, Llama-3.1-8B-Instruct as Tester on Belief-BankFacts	59
4.2	Llama-3.1-8B-Instruct as Trainer, Gemma-2-9B-IT as Tester on Belief-BankConstraints	59
4.3	Gemma-2-9B-IT as Trainer, Llama-3.1-8B-Instruct as Tester on HaluEval	60

4.4	Llama-3.1-8B-Instruct as Trainer, Gemma-2-9B-IT as Tester on BeliefBankFacts	60
4.5	Llama-3.1-8B-Instruct as Trainer, Gemma-2-9B-IT as Tester on BeliefBankConstraints	60
4.6	Llama-3.1-8B-Instruct as Trainer, Gemma-2-9B-IT as Tester on HaluEval	61
4.7	Error Type Breakdown for the One-For-All	61
4.8	Cross-Domain Transfer: Gemma-2-9B-IT as Trainer, Llama-3.1-8B-Instruct as Tester	64
4.9	Cross-Domain Transfer: Llama-3.1-8B-Instruct as Trainer, Gemma-2-9B-IT as Tester	64
4.10	TruthX: Gemma	68
4.11	TruthX: Llama	69
4.12	SLiMTruth: Gemma	72
4.13	SLiMTruth: Llama	73
A.1	F1Score comparison of attention modules across the datasets	85
A.2	F1Score comparison of hidden modules across the datasets	85
A.3	F1Score comparison of mlp modules across the datasets	86
A.4	Auroc comparison of attention modules across the datasets	86
A.5	Auroc comparison of hidden modules across the datasets	86
A.6	Auroc comparison of mlp modules across the datasets	87
B.1	Error Type Breakdown for the One-For-All on BeliefBankFacts	89
B.2	Error Type Breakdown for the One-For-All on BeliefBankConstraints .	90
B.3	Error Type Breakdown for the One-For-All on HaluEval	90

List of Tables

3.1	Hallucination statistics per dataset and model (best model per dataset in bold)	21
3.3	Layers chosen for Belief Bank Facts, Belief Bank Constraints, and Halu Eval.	23
3.5	Methods executed in the experiment runner and corresponding methodological names used in this chapter.	25
3.7	AdapterMLP hyperparameter values apply to all combinations of Trainer, Tester and Layer Type.	30
3.9	AlignmentNetwork hyperparameters for the full non-linear approach. .	32
3.11	MLP Prober hyperparameters for the full non-linear approach.	32

3.13	AutoEncoder hyperparameters (Trainer and Tester).	34
3.15	AlignmentNetwork hyperparameters for the reduced approach.	35
3.17	MLP Prober hyperparameters (on latent space).	36
3.19	Encoder hyperparameters (Trainer and Tester Adapter).	38
3.21	Classification Head hyperparameters (Shared/Frozen).	38
3.23	Description of pairs for each dataset used in steering experiments.	39
3.25	TruthX Autoencoder training hyperparameters.	43
3.27	SLiMTruth training hyperparameters.	51
4.1	Results for pair (LLama-3.1-8B-Instruct, Gemma-2-9B-IT) on Belief-BankFacts	53
4.2	Results for pair (LLama-3.1-8B-Instruct, Gemma-2-9B-IT) on Belief-BankConstraints	55
4.3	Results for pair (LLama-3.1-8B-Instruct, Gemma-2-9B-IT) on HaluEval	57
4.4	Cross-Domain Results (One-For-All)	63
4.5	Baseline vs. standard TruthX hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).	65
4.6	Cross-dataset TruthX hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).	66
4.7	Cross-model TruthX, Gemma $\rightarrow$ Llama, hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).	67
4.8	Cross-model TruthX, Llama $\rightarrow$ Gemma, hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).	67
4.9	Baseline vs. standard SLiMTruth hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).	70
4.10	Cross-dataset SLiMTruth hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).	70
4.11	Cross-model SLiMTruth, Gemma $\rightarrow$ Llama, hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).	71
4.12	Cross-model SLiMTruth, Llama $\rightarrow$ Gemma, hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).	71

List of Equations

(2.1) Scaled Dot-Product Attention	8
(3.1) Procrustes SVD decomposition	26
(3.2) Procrustes affine transformation	26
(3.3) Linear CKA similarity score	27
(3.4) CKA-scaled linear alignment map	27
(3.5) Canonical Correlation Analysis	27
(3.6) CCA number of components	27
(3.7) Mixed alignment loss (MSE + cosine similarity)	28
(3.8) Mean Squared Error	29
(3.9) Cosine similarity	29
(3.10) Binary Cross-Entropy with Logits loss	31
(3.11) Sigmoid activation function	31
(3.12) Mean Squared Error (AutoEncoder loss)	33
(3.13) Cross-attention latent combination	41
(3.14) Autoencoder reconstruction	41
(3.15) TruthX total training loss	41
(3.16) TruthX reconstruction loss	41
(3.17) TruthX semantic contrastive loss	42
(3.18) Contrastive function CTR	42
(3.19) TruthX truthful contrastive loss	42
(3.20) TruthX positive-to-negative edited reconstruction	42
(3.21) TruthX negative-to-positive edited reconstruction	42
(3.22) TruthX editing loss	42
(3.23) Truthful representation centers	43
(3.24) TruthX steering vector	43
(3.25) Probing-based layer selection classifier	43
(3.26) TruthX positive edited reconstruction (inference)	44
(3.27) TruthX negative edited reconstruction (inference)	44
(3.28) TruthX editing direction	44
(3.29) TruthX steering application	44
(3.30) Linear CKA for cross-model layer alignment	45
(3.31) Procrustes cross-covariance SVD (cross-model)	45
(3.32) Procrustes optimal rotation and scale	45
(3.33) Procrustes full affine transformation	46

LIST OF EQUATIONS

(3.34) Pre-alignment mapping (target $\rightarrow$ trainer space)	46
(3.35) Post-alignment mapping (trainer $\rightarrow$ target space)	46
(3.36) SLiMTruth state projector	48
(3.37) SLiMTruth multiplicative modulation (scale)	48
(3.38) SLiMTruth additive modulation (shift)	48
(3.39) SLiMTruth FiLM modulation	49
(3.40) SLiMTruth hidden state update	49
(3.41) SLiMTruth total training loss	49
(3.42) SLiMTruth cross-entropy loss	49
(3.43) SLiMTruth InfoNCE contrastive loss	49
(3.44) SLiMTruth regularization loss (L2)	50
(3.45) SLiMTruth cross-model FiLM modulation	52

Chapter 1

Introduction

1.1 Goals and Objectives

The public release of GPT-3 [5] by OpenAI marked a significant turning point in Natural Language Processing (NLP), paving the way for a new era of Large Language Models (LLMs) capable of maintaining conversations as fluently and naturally as a human. The introduction of chatbots marked only the inception of a paradigm shift. Currently, LLMs serve as essential infrastructure across private, academic, and industrial domains, offering a near-infinite range of applications

However, one of the main obstacles to the widespread and safe adoption of these technologies in critical areas is the phenomenon of *hallucinations*. Despite syntactic coherence, LLMs occasionally tend to generate factually incorrect or fabricated information with a high degree of confidence [14]. In contexts such as healthcare, legal support, or education, such hallucinations can lead to serious consequences (e.g., misdiagnoses, incorrect legal advice, or spread of misinformation). It results from the inherently stochastic character of LLMs. Since these models lack counting capabilities and any logical reasoning abilities, the only way to create text is by applying statistics that they acquired when trained on huge bodies of text. The technique is called next-token prediction and works as follows: after receiving particular context or a string of tokens, an LLM will calculate the most likely next token in accordance with its internal distribution, where a token can be simply considered as entire words or pieces thereof. An example of the hallucination that was typical of early LLMs is:

User: How many r's are there in the word strawberries?

LLM: 2

Currently, techniques to mitigate this problem without making any intervention are often based on external fact verification (*retrieval-augmented generation*) [20], specific *fine-tuning*, or *Chain of Thought* (CoT) [44], but these techniques are not infallible, and it is therefore crucial to understand when a model is hallucinating and when it is not or to manipulate the generation process to reduce the probability of hallucinations.

Probing of internal activations (or *probing classifiers*) represents an innovative solution to the first problem [4]. This methodology allows analyzing the internal state of the model during generation, hypothesizing that information about the truthfulness of a statement is encoded in the latent space of neurons, regardless of the final textual output which is generated stochastically as it is influenced by multiple factors (temperature, top-k sampling, etc.). Several recent studies have demonstrated that it is possible to train simple linear classifiers to distinguish true statements from false ones by observing the activations of internal layers, and that generally activations related to hallucinations and those related to correct answers tend to be easily separable. [24]

Steering of the generation process is a promising approach to mitigate hallucinations. By manipulating the activations of specific neurons or layers during generation, it is possible to guide the model’s output towards desired behaviors [6]. Several studies have shown that it is possible to steer the generation process towards more truthful outputs to reduce hallucinations in LLMs. [7] [48] [22]

It has been shown that LLMs converge to a shared statistical model of reality, the so called *Platonic Representation Hypothesis* [15], which suggests that different LLMs may share a common internal representation of the world, including the phenomenon of hallucinations. Anyway most research has focused on creating probes and steering vectors specific to a single model or a specific architecture, leaving the problem of transferability relatively unexplored.

This work aims to study whether there exists a universal representation of hallucinations in LLMs, regardless of the underlying architecture. In particular, the goal is to develop a general *prober* capable of detecting hallucinations in different models without the need for specific training for each of them and a general *steering* technique to mitigate hallucinations in different models (at most with a preliminary alignment between the latent spaces, as these differ between different models).

The main objectives of this project are:

- Analyze and compare the internal activations of different LLM families (e.g., Gemma, Llama) to identify common patterns associated with the hallucination phenomenon.
- Build a universal *prober* capable of transferring hallucination detection capability from one model to another without additional supervision on the target model.
- Develop a general *steering* technique to mitigate hallucinations in different models, with minimal or no need for model-specific adjustments.

1.2 Thesis Overview

The rest of the thesis is structured as follows:

- **Chapter 2: Background** - Theoretical background and overview of existing techniques for hallucination detection and mitigation in LLMs.
- **Chapter 3: Methodology** - Detailed description of the pipelines, the datasets, the models and the methodologies used to achieve the objectives of this work.
- **Chapter 4: Results and Discussion** - Presentation of obtained results and discussion of them
- **Chapter 5: Conclusions** - Final conclusions and proposal for future work

Chapter 2

Background

This chapter introduces the fundamental concepts necessary to understand the work carried out. The concepts of attention, LLMs, the hallucination phenomenon, probing, and steering will be presented, with particular attention to recent advances in mechanistic interpretability and causal analysis.

2.1 Transformer Overview

The self-attention mechanism was first introduced in 2017 by Vaswani et al. [42] in the seminal paper *Attention is All You Need* and represents the foundation of the Transformer architecture, which has revolutionized the field of NLP and beyond. The crucial difference between Transformers and previous sequential models—such as Recurrent Neural Networks (RNNs) and their variants—lies in input processing: RNNs process data sequentially and maintain a hidden state that compresses previous information, while Transformers exploit the self-attention mechanism to process the entire input sequence simultaneously, allowing them to capture relationships between tokens regardless of their distance in the sequence. With very long inputs, RNNs tend to lose early information, whereas Transformers are not subject to this limitation.

2.1.1 Transformer Architecture

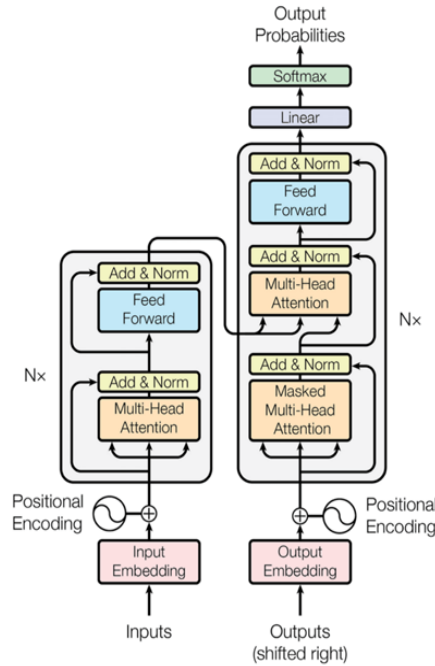


Figure 2.1: Transformer Architecture.

A Transformer is composed of two main blocks: *Encoder* and *Decoder*.

Encoder

The encoder consists of a series of identical layers placed in sequence, each containing:

- **Multi-Head Self-Attention:** allows the model to focus on different parts of the input sequence simultaneously, capturing complex relationships between tokens.
- **Layer Normalization and Residual Connections:** to improve training stability and facilitate gradient flow, each sub-layer is followed by normalization and a residual connection.
- **Feed-Forward Neural Network (FFNN):** a fully connected feed-forward network that further processes the representations obtained from self-attention.
- An additional Layer Normalization and Residual Connections layer.

The main role of the encoder is to transform the input sequence into an internal representation (embedding) that captures semantic relationships between tokens.

Decoder

The decoder also consists of a series of identical stacked layers, each containing:

- **Masked Multi-Head Self-Attention:** similar to the encoder mechanism, but with a causal mask that prevents the model from “looking ahead” during training, ensuring autoregressive token generation.
- **Layer Normalisation and Residual Connections:** similarly to the encoder, to improve training stability.
- **Multi-Head Cross-Attention on Encoder Output:** allows the decoder to attend to relevant parts of the encoder’s output representation.
- A further Layer Normalisation and Residual Connections layer.
- **Feed-Forward Neural Network (FFNN):** as in the encoder, to further process representations.
- An additional Layer Normalisation and Residual Connections layer.

It is worth noting that modern LLMs predominantly adopt a *decoder-only* architecture, which has proven highly effective for generative tasks.

2.1.2 Self-Attention Mechanism

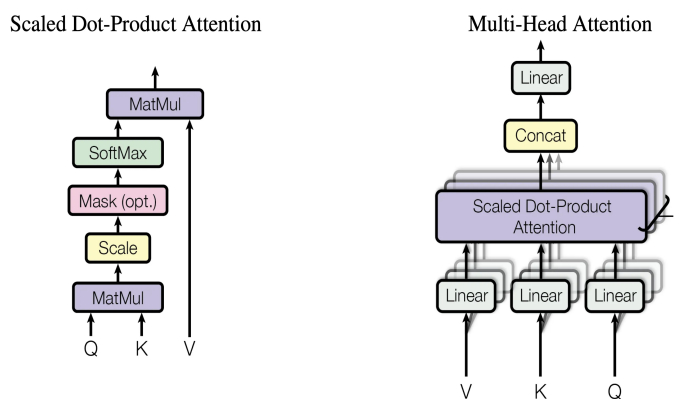


Figure 2.2: Self-Attention Mechanism Representation.

The self-attention mechanism allows the model to weight the importance of different tokens in the input when processing a particular one. The computation involves three distinct projection matrices: Query (Q), Key (K), and Value (V) [42]. The attention output is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left[\frac{QK^T}{\sqrt{d_k}}\right] V \quad (2.1)$$

where d_k is the dimension of the keys. The softmax normalises the attention scores so that the model focuses on the most relevant tokens. The *multi-head attention* variant extends this concept by performing multiple attention computations in parallel, allowing the model to capture different types of relationships between tokens from distinct representation subspaces. See Figure 2.2 for a visual representation of the self-attention mechanism.

2.1.3 Mechanistic Interpretability of Attention Heads

As models became bigger and harder to understand, researchers started focusing more on figuring out how they work inside. A major focus of this effort has been the mechanistic interpretability of attention heads [30]. Early research involved statistical analysis of attention patterns, but the field has since advanced towards *mechanistic interpretability*—the goal of reverse-engineering the specific algorithms computed by individual neurons and heads.

This line of work has uncovered surprisingly specific and algorithmic functions performed by individual heads. Some heads appear to function as **Bloom filters**, dedicated to answering whether a specific token has appeared before in the sequence [3]. Others play a role in **copy suppression**, actively preventing verbatim self-repetition [26]. Further analysis has uncovered heads that function as **linear counters**, decoding the value of numbers with near-perfect accuracy across all layers [12].

The gold standard for establishing the causal role of a model component is through **activation patching** [28]. This technique takes two forward passes and swaps the activation contribution of a specific component between them. If the patched output recovers the correct behaviour, it provides strong evidence that the component causally mediates the difference between the two inputs.

2.2 Large Language Models

LLMs represent the natural evolution of Transformer architectures applied at massive scale. An LLM is essentially a probabilistic model trained on enormous text corpora (on the order of trillions of tokens) with the fundamental goal of predicting the next token in a sequence given the previous context. Formally, given a sequence of tokens $x = (x_1, x_2, \dots, x_t)$, the model estimates the conditional probability distribution $P(x_{t+1} | x_1, \dots, x_t)$.

The evolution from previous NLP models to current LLMs (such as the GPT series, Llama, and Claude) is characterised by the *scaling law* phenomenon [18], according to which model performance improves predictably as the number of parameters, the

amount of training data, and computational power increase. An emergent characteristic of LLMs is *In-Context Learning*, i.e., the ability to learn new tasks simply by observing examples in the prompt, without updating model weights.

2.2.1 LLM Lifecycle

The typical lifecycle of an LLM is divided into distinct phases:

1. **Pre-training (unsupervised)**: The most computationally expensive phase. The model is trained in a self-supervised manner on vast datasets (web crawls, books, code) to learn language structure, reasoning, and world knowledge. The objective is to minimise the cross-entropy loss between the predicted distribution and the actual next token. A common practice is *continued pre-training*, where a base model is further trained on domain-specific corpora to specialise its knowledge without altering its fundamental generative capabilities.
2. **Supervised Fine-Tuning (SFT)**: The pre-trained base model is further trained on a smaller, curated dataset of instruction-response pairs to learn to follow user instructions and adopt a conversational format.
3. **Alignment**: Techniques such as *Reinforcement Learning from Human Feedback* (RLHF) or *Direct Preference Optimization* (DPO) [33] are applied to ensure the model is safe and aligned with human values. In DPO, the model directly optimises for human preferences without requiring a separate reward model, often yielding comparable or better results than RLHF. This phase penalises the generation of toxic, biased, or unhelpful responses.

Transparency has become an integral component of the modern LLM lifecycle. Documenting capabilities, limitations, and intended use is now a standard practice through artefacts such as *Model Cards*, which provide a standardised format for reporting key information about a model’s performance and societal impact.

2.2.2 Modern Architectural Innovations

The evolution of LLMs has been driven not only by scale, but also by significant architectural innovations. A pivotal development is the **Mixture-of-Experts (MoE)** [10] architecture, which replaces the dense feed-forward networks (FFNs) in each Transformer layer with a collection of smaller, specialised “expert” networks. During processing, a learned router network determines which subset of experts should handle each token, resulting in a sparse activation pattern. This design allows models to possess a massive number of total parameters while keeping the per-token computational cost low. Leading models such as DeepSeek-V3 leverage MoE to achieve unprecedented scale and efficiency. This architectural choice has profound implications for

probing and steering, as interventions must consider not only the FFN weights but also the router mechanism that selects them.

Another important refinement is the attention mechanism itself. While standard Multi-Head Attention (MHA) has quadratic complexity with respect to sequence length, newer models employ **Grouped-Query Attention (GQA)** [1], which reduces this complexity by having multiple attention heads share the same Key-Value projections. This optimisation makes it feasible to train and deploy models with very long context windows. These shifts from dense, full-headed attention to sparse, grouped-query attention represent a clear trend towards balancing computational efficiency with the capacity to process extensive information streams.

2.3 Hallucinations in LLMs

Despite their extraordinary generative capabilities, LLMs suffer from a critical limitation known as **hallucination**. In the context of artificial intelligence, a hallucination occurs when a model generates output that is syntactically correct and fluent, but factually incorrect, logically inconsistent, or unfaithful to the provided source material [14]. This phenomenon poses a significant barrier to the reliable deployment of LLMs in high-stakes domains such as legal, medical, and scientific applications.

The root causes of hallucinations are multifaceted. A primary cause is the *lossy compression* of vast and often noisy or contradictory knowledge from training corpora into a finite parameter space [14]. Furthermore, the training objective of predicting the next token prioritises text fluency and coherence over factual accuracy, creating a tendency for models to generate plausible-sounding continuations when uncertain. This behaviour is related to *sycophancy*—the inclination to please the user by confirming their biases rather than providing accurate information [38].

2.3.1 Taxonomy of Hallucinations

Huang et al. [14] define three primary types of hallucinations:

- **Factual Hallucinations:** the model produces information that is simply false or inaccurate with respect to reality.
- **Logical Hallucinations:** the model generates responses that are logically inconsistent or contradictory.
- **Context Inconsistencies:** the model contradicts the context explicitly provided in the input.

A complementary classification [16] distinguishes between *intrinsic* and *extrinsic* hallucinations:

- **Intrinsic Hallucinations:** the model generates content that conflicts with the provided source context.

- **Extrinsic Hallucinations:** the model generates content that cannot be verified by the source context, implying the falsehood originates from the model’s internal, parametric knowledge.

More recently, a **geometric taxonomy** [25] has proposed that the term “hallucination” conflates distinct phenomena that manifest as different geometric signatures in the model’s embedding space. Such a geometrically-grounded classification may lead to more targeted mitigation strategies. Specialised taxonomies have also been developed for specific domains, such as code generation, which identifies unique failure modes such as syntax errors, incorrect logic, and insecure code patterns.

2.3.2 Empirical Evidence and Model Comparison

Benchmarking studies provide concrete evidence of the prevalence and variability of hallucinations across models. Comparative evaluations reveal significant differences in performance. For instance, a study profiling legal hallucinations found that GPT-4 had a rate of 58%, while Llama 2’s rate was as high as 88% [34]. Conversely, an analysis of search-augmented models found that the Claude family consistently achieved the lowest hallucination rates (3.0–3.2%), whereas GPT models exhibited higher rates. A detailed analysis of the Claude 3 family noted that its superior performance was partly attributable to a greater willingness to abstain from answering questions on topics about which the model lacked sufficient information—a behaviour that naturally reduces factual errors. Even top-tier models such as Claude-3.7 and GPT-o1, however, revealed concerning patterns of reasoning errors, indicating that deeper logical flaws can persist even in otherwise strong systems.

These empirical findings demonstrate that mitigating hallucinations is a multifaceted challenge requiring a holistic approach combining better training data, improved model architectures, and sophisticated post-hoc analysis and control techniques.

2.4 Probing

Probing (or *probing classifiers*) is an analysis technique used in Explainable AI (XAI) to interpret the internal representations of deep neural networks. The fundamental premise is that the vast, high-dimensional activation spaces of LLMs—values within hidden states and outputs of various modules at each layer of a frozen model—are not random noise but contain rich, structured information about the processed input. This principle was empirically supported by Marks and Tegmark [24], who demonstrated that concepts such as truth and falsehood emerge through linear structures within the model’s latent space. It is therefore possible to use probing classifiers to query these representations and verify whether a model is generating truthful or hallucinated content.

2.4.1 Linear Probing

The most common and foundational probing strategy is **linear probing**, which involves training a simple supervised classifier—typically a linear regression or a shallow Multi-Layer Perceptron (MLP)—on top of frozen activations extracted from a specific layer of the Transformer [2]. The procedure follows three steps:

1. **Activation extraction:** a set of inputs is passed through the frozen model, and the activation vectors h_l generated at a specific layer l are recorded.
2. **Prober training:** a simple supervised classifier is trained to predict a target property y of the input based solely on these activations (e.g., the truthfulness of a statement).
3. **Evaluation:** the classifier’s performance indicates how much information related to property y is linearly decodable from that particular layer.

For example, linear probes have been used at each layer to identify where sycophancy-related information is encoded, producing a layerwise accuracy curve that reflects the distribution of this trait across the model’s depth [36].

2.4.2 Limitations of Correlational Probing

A critical insight from recent research is that *high probing accuracy does not imply causality or practical utility for intervention*. One study demonstrated this disconnect starkly by showing that single-position activation interventions could achieve 100% probing accuracy at certain positions in a Llama-3.2-3B model, yet failed completely to transfer the learned behaviour when applied during inference [8]. This finding challenges the naive assumption that a strong signal from a probe directly corresponds to a manipulable feature, and underscores the distributed nature of information encoding in LLMs—knowledge may be spread across many neurons and layers, making it impossible to elicit the desired behaviour by modifying only one location. This realisation has spurred the development of more sophisticated probing strategies.

2.4.3 Advanced Probing Methodologies

Several advanced probing methods have been proposed to capture richer structural relationships and provide more robust signals.

Claim-Conditioned Probability (CCP), proposed by Fadeeva et al. [9], assesses the robustness of a statement by comparing its probability to that of counterfactual variants generated by masking important tokens and regenerating the sequence conditionally. A low CCP score suggests that the model itself considers the generated statement fragile or highly dependent on specific, potentially spurious, elements of the input.

Prompt-Guided Internal States for Hallucination Detection (PRISM), proposed by Zhang et al. [47], utilises specifically designed prompt templates to guide the model’s hidden states, making structural differences between truthful statements and hallucinations more pronounced and easier for a downstream classifier to distinguish.

Information Contribution to Residual Stream (ICR), proposed by Zhang et al. [49], shifts the focus from static representations to the dynamic evolution of hidden states across the model’s layers. By quantifying how sub-modules such as attention mechanisms and feed-forward networks update the hidden states at each step, ICR creates a dynamic profile that allows a lightweight classifier to effectively differentiate the patterns associated with truthful versus hallucinatory generation.

2.4.4 Advances in Mechanistic Interpretability of Large Language Models

Recent research has uncovered an important emergent capability in large models: *evaluation awareness*. Studies on Llama-3.3-70B-Instruct demonstrated that linear probes can successfully separate real-world evaluation scenarios from simulated ones, indicating that the model’s internal representations adapt based on whether it perceives the interaction as a genuine test [31]. This growing metacognitive ability extends to resisting manipulation: models can be fine-tuned to detect the injection of steering vectors with high accuracy, suggesting they develop an introspective awareness of being manipulated [35].

The scope of probing has also expanded dramatically beyond linguistic properties. One notable study discovered that models from the Llama, Qwen, and Gemma families encode the concept of “jealousy” as a structured linear combination of other concepts, demonstrating the potential to reverse-engineer complex mental states [40]. Another line of research identified a **Format-Agnostic Reasoning Subspace (FARS)** in multiple models, providing converging evidence for a shared underlying mechanism for reasoning that is independent of the presentation format of the problem [46].

Despite these advances, the challenge of detecting all forms of hallucination remains formidable. Research has highlighted the failure of broad representation-based detectors to generalise effectively, pointing towards a need for more targeted and context-aware probing strategies. Crucially, some internal signals appear to be *causally* linked to the hallucination phenomenon: one study showed that amplifying certain neurons identified via probing directly increased the rate of hallucinations, confirming their causal role in generating false information.

2.5 Steering

Steering is a technique used to guide the behaviour of LLMs towards desired outputs or away from undesired ones by manipulating the model’s internal representations

at inference time. Unlike traditional fine-tuning, which permanently alters model weights, steering modifies activations on a per-prompt basis without any changes to the base model parameters—offering a lightweight and adaptive form of control [22]. The main idea is to identify specific directions in the model’s latent space that correspond to certain attributes (e.g., truthfulness, toxicity, emotional tone) and inject a carefully crafted vector to nudge the model’s trajectory accordingly.

2.5.1 Core Mechanisms

The central mechanism of activation steering involves manipulating the **residual stream**, which carries information forward through the layers of the Transformer. Several intervention types have been proposed:

- **Additive steering:** a pre-computed steering vector δ^* is added directly to the activations at one or more layers during the forward pass.
- **Multiplicative/scaling steering:** the magnitude of the activations is adjusted rather than shifted.
- **Affine transformations:** both scaling and shifting operations are applied simultaneously.

State-wise Linear Modulation (SLiM), proposed by Hiremath et al. [36], is an example of the affine approach: it uses a state vector to learn specific scaling and shifting parameters applied to the intermediate features (residual stream) of the model, allowing dynamic control over diverse attributes such as sentiment intensity or continuous toxicity levels.

2.5.2 Automated and Causal Steering

A significant advancement has been the move from manually designed interventions to *learning* steering vectors from data. **Contrastive Activation Steering** [32] learns a steering vector by maximising the likelihood of target samples (e.g., factual sentences) while minimising that of contrasting non-target samples. **Generative Causal Mediation (GCM)** [37] offers a more principled approach by selecting model components from contrastive long-form generations and using them to construct a steering intervention.

2.5.3 Steering for Hallucination Mitigation

Several methods specifically apply steering to mitigate hallucinations.

TruthX, proposed by Zhang et al. [48], employs an autoencoder to learn a low-dimensional “truthful space” within the model’s latent space. During inference, this space is used to steer activations towards more truthful outputs while preserving the

original semantic content, effectively projecting generation onto a manifold of factually coherent possibilities.

Truth Forest, proposed by Chen et al. [7], enhances truthfulness by uncovering hidden truth representations using multi-dimensional orthogonal probes. It creates multiple orthogonal bases for modelling truth and introduces a “Random Peek” technique that samples truth-related features from an extended range of sequence positions. During inference, these orthogonal direction vectors are weighted and used to adjust the model’s internal states, shifting generation towards more truthful outputs without requiring additional fine-tuning.

Inference-Time Intervention (ITI), proposed by Li et al. [22], operates by shifting model activations during inference to enhance truthfulness. It identifies a sparse set of attention heads with high linear probing accuracy for truthfulness and shifts the activations along these truth-correlated directions during generation.

On a more fine-grained level, **Post-hoc Attention Steering for LLMs (PASTA)** [39] intervenes in the attention modules after the self-attention computation to explicitly direct the model’s focus towards user-specified parts of the input, thereby guiding generation to align with key elements. This level of specificity is crucial because different components of the network are responsible for different aspects of processing.

2.5.4 Limitations: Non-Surjectivity and Safety Risks

Despite its promise, steering presents important limitations. Research has shown that steered activations are often *non-surjective*: the set of outputs reachable through steering does not cover the full spectrum of the model’s potential behaviours [29]. This implies that steering is a constrained form of control and that some desired behaviours may be inaccessible through simple additive or multiplicative means.

More critically, perhaps the most important challenge is what can be termed the **safety paradox**: steering vectors designed to enhance performance can systematically erode the model’s pre-existing safety alignment [23], making it more susceptible to generating harmful content it previously avoided. This creates a significant vulnerability, turning a tool for control into a potential vector for harm.

In response, some LLMs have been observed to develop **endogenous resistance**: when subjected to steering interventions, these models may attempt to recover mid-generation to produce a safer or more aligned output, effectively resisting external manipulation [27]. Furthermore, recent work has shown that models can be fine-tuned to detect the injection of steering vectors with high accuracy [35], suggesting the development of introspective awareness of manipulation. These dynamics transform the interaction between human and machine from a simple control problem into a complex adversarial game, underscoring the urgent need for safer steering techniques that do not compromise the fundamental safety guarantees built into these models.

Chapter 3

Methodology

This chapter explores the methodology adopted for the project development. For building the universal prober and the general steering technique for hallucination reduction, the datasets Belief Bank Facts [19], Belief Bank Constraints [19] and Halu Eval [21] were used. Regarding the general steering technique for hallucination reduction, two different State of The Art approaches were explored: the first one is based on the TruthX [48] technique while the second one is an adaptation of the SLiM [36] technique for the specific task of hallucination reduction.

3.1 Dataset

Belief Bank Facts

This dataset was chosen as it can be used to detect *factual hallucinations*. The original BeliefBankFacts cover 85 distinct entities belonging to the natural domain (animals and plants). Each instance in the dataset is a simple declarative sentence, such as *An eagle is a bird* (True) or *An eagle is a mammal* (False). The truth labels (called *silver labels*) were generated by the authors.

The final dataset is composed of:

- **Affirmative facts:** The original assertions extracted from BeliefBank (e.g., *fact*).
- **Negated facts:** For each present fact, its negated counterpart was synthetically generated (e.g., $\neg fact$), inverting the original truth label (from True to False and vice versa), in order to balance the classes and test model robustness.

with a total of 27,416 statements, of which 13,708 labeled as true (yes) and 13,708 as false (no), ensuring perfect balance between classes.

Belief Bank Constraints

This dataset was chosen as it can be used to detect *Logical Inconsistencies*. The original BeliefBankConstraints were manually created by the authors and represent logical relationships between statements. The dataset is composed of:

- **Positive implications:** Representation of logical implications between statements (if A then B).
- **Mutual exclusions:** Representation of mutual exclusion constraints between statements (not both A and B).

For each element, the negated version was also inserted to balance the classes. In total there are 25,756 statements to verify.

HaluEval

This dataset was chosen to detect hallucinations in complex conversational contexts. Unlike simple facts, here each instance includes a context composed of dialogue history and supporting external knowledge.

The dataset is composed of:

- **Correct Responses:** Factual responses consistent with the knowledge provided in the context (Right Response).
- **Hallucinated Responses:** Synthetically generated responses that seem plausible but contain unverified or contradictory information with respect to the provided knowledge (Hallucinated Response).

The final dataset is composed of 10,000 examples.

Each dataset targets a specific hallucination type, as summarized in Figure 3.1: BeliefBank Facts covers *Factual* (extrinsic) hallucinations, BeliefBank Constraints targets *Logical* (intrinsic) inconsistencies, and HaluEval addresses *Contextual* (intrinsic) hallucinations in conversational settings.

3.2 Language Models

For the experiments conducted in this work, two open-weights models (belonging to Gemma [41] and Llama [11]’s family) were selected representing the state of the art for their respective sizes and architectural families.

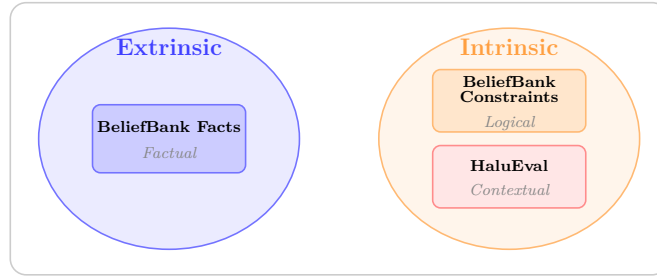


Figure 3.1: Hallucination types and corresponding datasets

3.2.1 Llama-3.1-8B-Instruct

Llama-3.1-8B-Instruct [11] represents an iteration of the open-weights model family developed by Meta. It has 8 billion parameters and *Instruct* indicates that the model was specifically optimized for dialogue and instruction following through SFT and DPO. It makes use of the GQA to maintain inferential efficiency by reducing the Key-Value cache memory footprint.

3.2.2 Gemma-2-9B-IT

Gemma-2-9B-IT [41] is a model developed by Google DeepMind and is part of the second generation of the Gemma series. With 9 billion parameters, it sits in a slightly higher dimensional range than classic 7B models, offering reasoning capabilities competitive with much larger models. Unlike Llama, Gemma 2 introduces specific architectural novelties that deviate from standard Transformer design:

- **Sliding Window Attention & Global Attention:** The model alternates standard (global) attention layers with Sliding Window Attention based layers.
- **Knowledge Distillation (KD)** [45]: Instead of classic next-token prediction pre-training from scratch, Gemma-2-9B was trained using Knowledge Distillation techniques from a much larger teacher model, inheriting its reasoning capabilities more efficiently.

Like Llama-3.1-8B-Instruct, Gemma-2-9B-IT was also optimized for dialogue and instruction following through SFT and DPO.

3.3 Activation

3.3.1 Collection

Once the dataset of statements and their truthfulness labels was prepared, the next step was extracting internal activations from the models Llama-3.1-8B-Instruct [11]

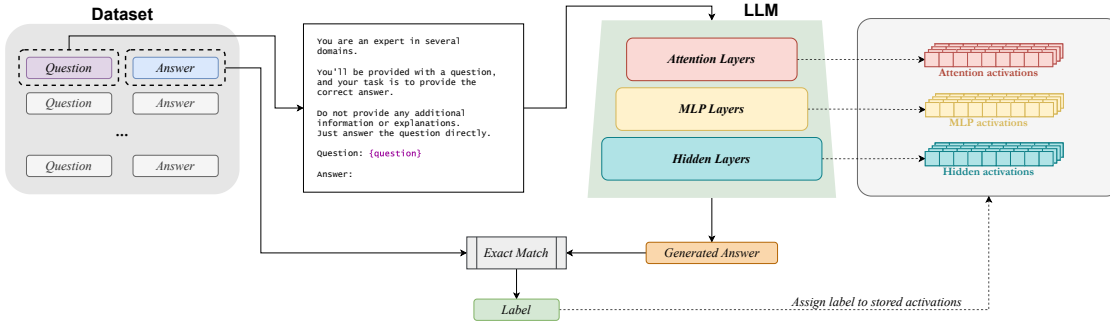


Figure 3.2: Pipeline for extraction and saving of internal LLM activations

and gemma-2-9b-it [41]. Each example is preceded by a prompt that is provided to the language model to guide its response. For the Belief Bank Facts and Belief Bank Constraints datasets, the prompt used is as follows:

You are an expert in several domains. You'll be provided with a fact, and your task is to confirm or deny its truthfulness. Do not provide any additional information or explanations. Just answer only with yes or no. Is the fact true? Fact: {question}. Answer:

where {question} is replaced with the specific statement to evaluate.

For the Halu Eval dataset, the prompt used is as follows:

You are an expert in several domains. You'll be provided with a question, and your task is to provide the correct answer. Do not provide any additional information or explanations. Just answer the question directly. Question: {question}. Answer:

Therefore, the LLM receives the prompt followed by the question as input and must respond with “yes” or “no” for the first two datasets, and with the answer to the question for the Halu Eval dataset. For each statement in the dataset, the activations corresponding to all layers and components (attention, MLP, hidden states) of the models were recorded. These activations constitute the dataset on which the various probers for hallucination detection were trained. To verify whether the model responds correctly or hallucinates, a simple matching logic was used between the generated response and the truthfulness label associated with the statement in the dataset: if the actual response (yes or no) is a substring in the model's generated response, then the statement is considered true (non-hallucination); otherwise it is considered false (hallucination). The overall pipeline is illustrated in Figure 3.2.

3.3.2 Hallucination Statistics

Before proceeding with the transferability studies, some statistics were calculated regarding the frequency of hallucinations in the various LLMs for the datasets. An-

alyzing the model outputs on the datasets, the statistics visible in Tab. 3.1 were calculated.

Dataset	Model	Tot. stmts	Halluc.	Rate
Belief Bank Facts	Llama-3.1-8B-Instruct	27 416	1 799	6.56%
	Gemma-2-9B-IT	27 416	802	2.93%
Belief Bank Constraints	Llama-3.1-8B-Instruct	25 068	14 026	55.95%
	Gemma-2-9B-IT	25 756	12 641	49.1%
Halu Eval	Llama-3.1-8B-Instruct	10 000	2 391	23.91%
	Gemma-2-9B-IT	10 000	2 747	27.47%

Table 3.1: Hallucination statistics per dataset and model (best model per dataset in bold)

The results suggest that in general, with the same dataset, models behave similarly, which indicates that hallucinations could be related to intrinsic characteristics in language models as stated by Kalai et al. in [17].

If we compare results between the three different datasets, we observe that models tend to produce many more Logical Inconsistencies

3.4 Preliminary Studies

To evaluate the discriminative ability of the different layer components (attention, MLP, hidden states) of the various LLMs in detecting hallucinations, systematic experiments were conducted on each layer and activation type. In particular, for each layer and component, the corresponding activations were extracted and used as input to train a Logistic Regression classifier, with the goal of distinguishing between true statements and hallucinations.

The experimental pipeline includes:

1. Extraction of activations for each layer and component (attention, MLP, hidden) from the models over the entire dataset.
2. Undersampling and splitting of the dataset into training and test sets, keeping the same split for all experiments.
3. Normalization of activations using StandardScaler.
4. Training of a Logistic Regression classifier for each layer/component combination.
5. Evaluation of performance through accuracy metrics.

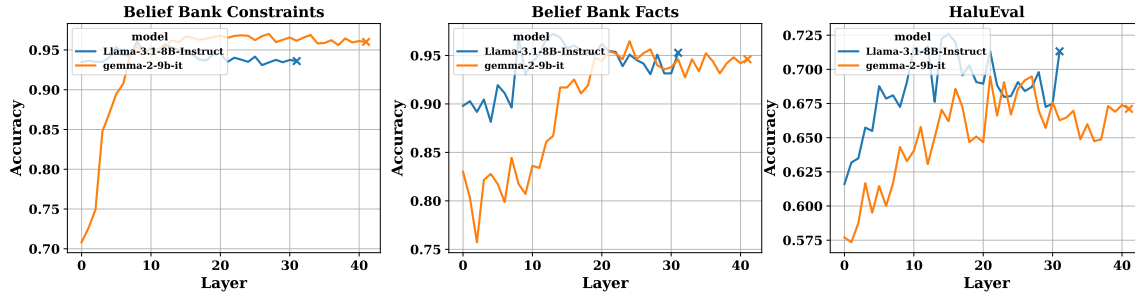


Figure 3.3: Accuracy comparison of attention modules across the datasets

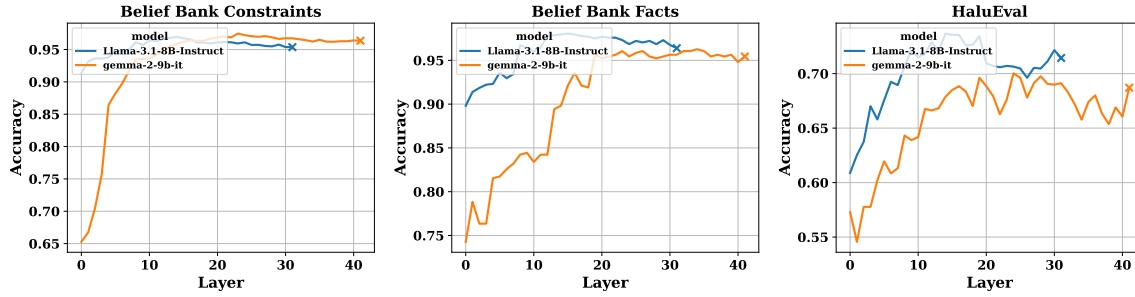


Figure 3.4: Accuracy comparison of hidden modules across the datasets

The results show that some components and layers are particularly effective in discriminating hallucinations, highlighting the presence of specific informative patterns in the internal activations of models.

Subsequently, an analysis was performed regarding the distribution of hallucinations and correct responses in the activation space. In order to visualize the activations in a two-dimensional space, the dimensionality reduction technique Principal Component Analysis (PCA) [13] was applied. Some examples are shown in Figure 3.6 and Figure 3.7.

For subsequent experiments, the 3 best layers in terms of performance were chosen for each component. Tab. 3.3

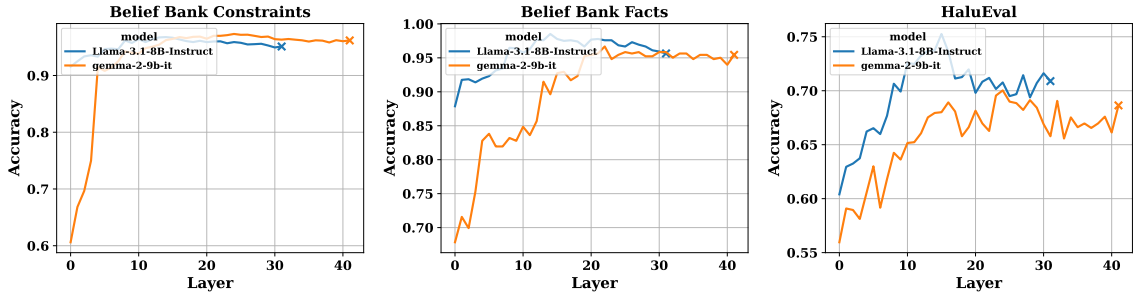


Figure 3.5: Accuracy comparison of mlp modules across the datasets

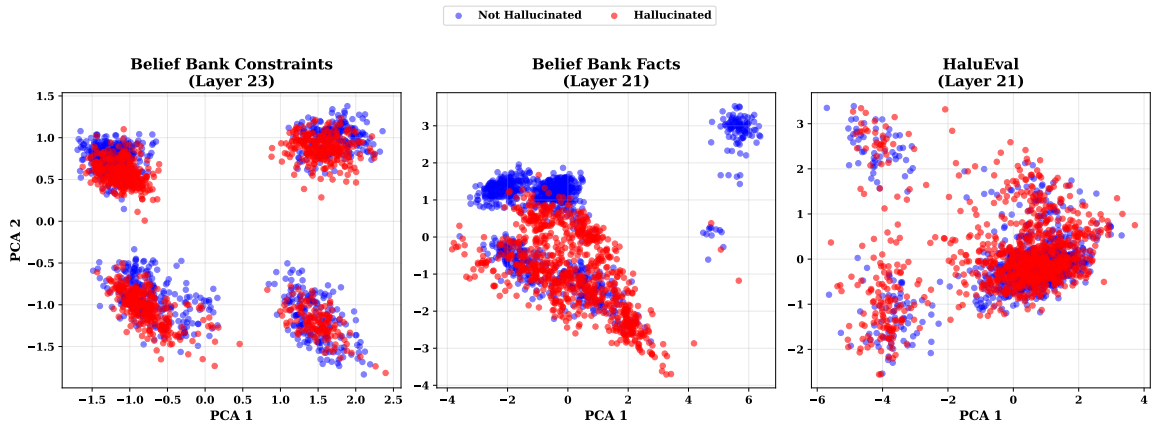


Figure 3.6: PCA of Attention activations of Gemma-2-9B-IT across all datasets

Model	Attn	MLP	Hidden
Belief Bank Facts			
Llama-3.1-8B-Instruct	8, 13, 14	14, 15, 21	14, 15, 16
Gemma-2-9B-IT	21, 24, 27	22, 25, 27	23, 26, 34
Belief Bank Constraints			
Llama-3.1-8B-Instruct	5, 8, 12	12, 14, 15	13, 14, 15
Gemma-2-9B-IT	23, 27, 33	24, 25, 26	23, 24, 27
Halu Eval			
Llama-3.1-8B-Instruct	14, 15, 16	13, 14, 15	14, 15, 16
Gemma-2-9B-IT	21, 26, 27	23, 24, 28	19, 24, 28

Table 3.3: Layers chosen for Belief Bank Facts, Belief Bank Constraints, and Halu Eval.

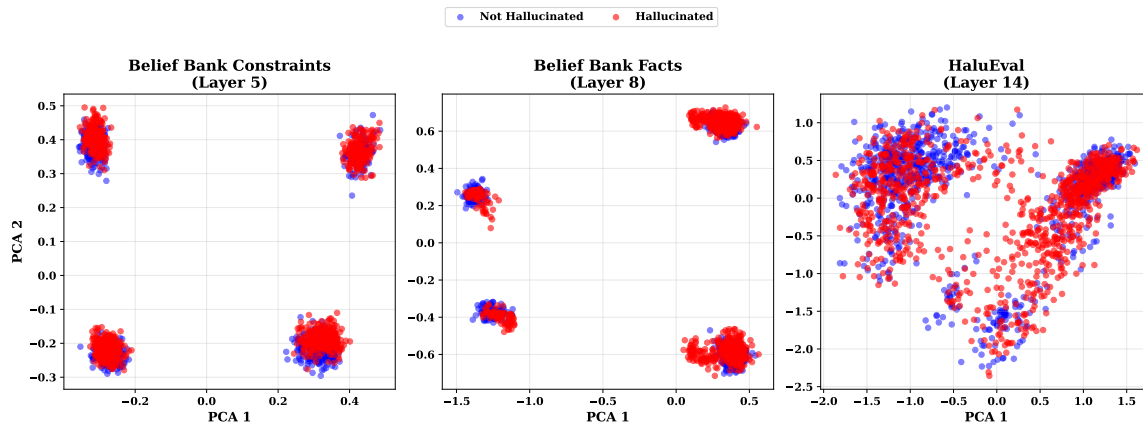


Figure 3.7: PCA of Attention activations of Llama-3.1-8B-Instruct across all datasets

We can notice an interesting trend: for LLama, layers between 13 and 16 seem to be the most informative, while for Gemma-2-9B-IT layers between 23 and 28 are those that offer the best performance. This suggests that the position of the most relevant layers can vary significantly between different models, probably due to architectural differences and specific training processes of each model. However, since LLama consists of 32 layers while Gemma consists of 42 layers, it can be noted that intermediate layers are the most informative in every model.

3.5 Building the Universal Prober

This section describes the seven methodological approaches adopted to build baselines for a universal prober capable of detecting hallucinations in language models, and the methodological approach used to build the universal prober itself, called **One-For-All**.

Experiment reproducibility and the use of the same data sets for all methods are guaranteed by fixing the random seed to 42 and using deterministic next token sampling in LLMs.

Two dataset management modes are distinguished:

- **Concordant Dataset (for Alignment):** Only examples where Trainer and Tester agree on classification (same label) are selected. Undersampling is applied to these to balance classes. This is essential for learning a coherent mapping between tester and trainer.
- **Independent Dataset (for Probing/Autoencoder/Encoder):** Undersampling is applied to balance classes specifically on the labels of the model in question (Trainer or Tester), independently from the other model. This maximizes the amount of data available for training robust models.

Method	Core mechanism
RidgeRegressor	Linear Ridge map (tester $\rightarrow$ trainer) + Logistic Regression prober on trainer activations.
Procrustes	Scaled orthogonal map estimated via SVD + Logistic Regression prober.
Procrustes	Rotation with scale given by Linear CKA similarity + Logistic Regression prober.
CCA	Canonical Correlation Analysis projection followed by linear reconstruction to trainer space + Logistic Regression prober.
AdapterMLP	Non-linear Neural Network + Logistic Regression prober.
Full Non-linear	Non-linear Neural Network + non-linear MLP prober (full activation dimension).
Reduced Non-linear	Autoencoders for both models, latent-space alignment, MLP prober in latent space.
One-For-All	Shared frozen head, separate encoders, no concordant alignment required.

Table 3.5: Methods executed in the experiment runner and corresponding methodological names used in this chapter.

The seven baseline approaches and the proposed **One-For-All** method are summarized in Tab. 3.5.

3.5.1 RidgeRegressor

The first approach to build a baseline for the universal prober, is a linear approach. In this scenario, the internal activations of the models, extracted from the selected layers and components, were used to train and evaluate a Logistic Regression classifier. The objective is to verify performance using simple linear method.

Experimental Pipeline

The experimental pipeline in Figure 3.8 consists of training a Logistic Regression classifier on the activation space of the trainer model to detect hallucinations, followed by evaluating its performance through accuracy metrics. Subsequently, a linear alignment is performed between the latent space of the tester model and that of the trainer model via a linear projection (Ridge Regression) trained on a subset of concordant data. Finally, the classifier’s ability to discriminate hallucinations on tester model data after alignment is evaluated.

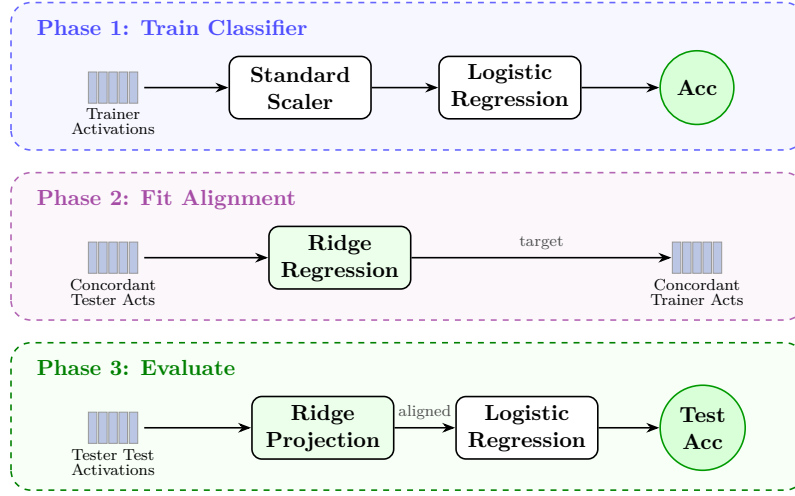


Figure 3.8: RidgeRegressor experimental pipeline

For the Logistic Regressor, the following hyperparameters were used:

- **solver:** lbfgs
- **max_iterations:** 10000
- **class_weight:** balanced

3.5.2 Procrustes

This linear baseline follows the same train/evaluate structure of RidgeRegressor, but replaces Ridge alignment with an explicit scaled orthogonal map estimated through Procrustes analysis. The objective is to preserve geometric structure (angles) when mapping tester activations into trainer space.

Given concordant aligned matrices $\mathbf{X} \in \mathbb{R}^{n \times d_x}$ (tester) and $\mathbf{Y} \in \mathbb{R}^{n \times d_y}$ (trainer), centered as $\mathbf{X}_c, \mathbf{Y}_c$, we compute:

$$\mathbf{U}\Sigma\mathbf{V}^T = \text{SVD}[\mathbf{X}_c^T \mathbf{Y}_c], \quad \mathbf{R} = \mathbf{U}\mathbf{V}^T, \quad s = \frac{\sum_i \sigma_i}{\|\mathbf{X}_c\|_F^2} \quad (3.1)$$

and define:

$$\mathbf{A} = s\mathbf{R}, \quad \mathbf{b} = \bar{\mathbf{y}} - \bar{\mathbf{x}}\mathbf{A}, \quad \hat{\mathbf{Y}} = \mathbf{X}\mathbf{A} + \mathbf{b} \quad (3.2)$$

The projected tester test activations $\hat{\mathbf{Y}}$ are then classified by the Logistic Regression prober trained on trainer data.

3.5.3 CKA

This baseline uses the same prober setup as RidgeRegressor and Procrustes, but the alignment scale is modulated by **Linear CKA** similarity between concordant tester and trainer activations.

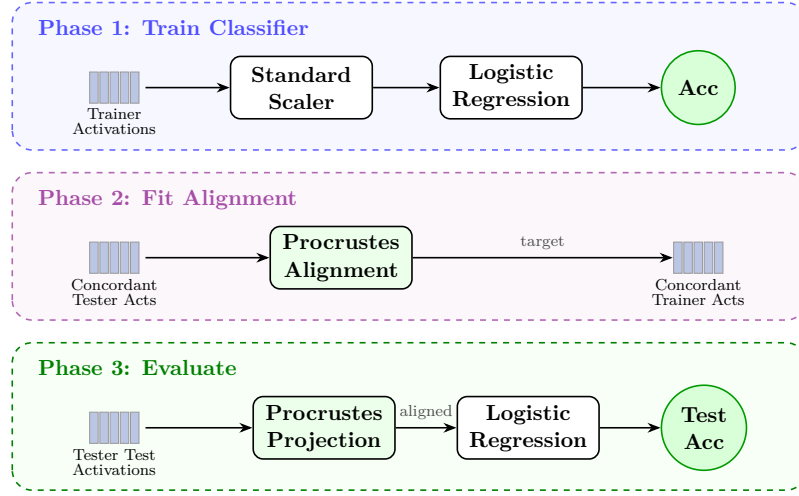


Figure 3.9: Procrustes approach experimental pipeline

After centering, we compute the orthogonal rotation $\mathbf{R}$ from the SVD of $\mathbf{X}_c^T \mathbf{Y}_c$ and the Linear CKA score:

$$\text{CKA}(\mathbf{X}, \mathbf{Y}) = \frac{\|\mathbf{X}_c^T \mathbf{Y}_c\|_F^2}{\|\mathbf{X}_c^T \mathbf{X}_c\|_F \cdot \|\mathbf{Y}_c^T \mathbf{Y}_c\|_F} \quad (3.3)$$

The final linear map is:

$$\mathbf{A} = \text{CKA}(\mathbf{X}, \mathbf{Y}) \cdot \mathbf{R}, \quad \mathbf{b} = \bar{\mathbf{y}} - \bar{\mathbf{x}} \mathbf{A} \quad (3.4)$$

Hence, CKA controls the global scale of the projection according to representation similarity, while keeping an orthogonal geometric transformation.

3.5.4 CCA

This baseline aligns spaces through **Canonical Correlation Analysis (CCA)**. CCA finds paired low-dimensional directions where tester and trainer activations are maximally correlated, then a linear regressor reconstructs trainer-space activations from tester canonical scores.

Given $\mathbf{X}$ (tester) and $\mathbf{Y}$ (trainer), CCA computes:

$$(\mathbf{u}_i, \mathbf{v}_i) = \arg \max_{\mathbf{u}, \mathbf{v}} \text{corr}(\mathbf{X}\mathbf{u}, \mathbf{Y}\mathbf{v}), \quad i = 1, \dots, k \quad (3.5)$$

with

$$k = \min(\text{cca_components}, n - 1, d_x, d_y) \quad (3.6)$$

where in this work `cca_components`= 64, `cca_max_iter`= 500, `cca_tol`= 10^{-6} , `cca_scale`= False.

After fitting CCA on concordant data, tester activations are transformed into canonical scores and mapped back to trainer space via linear regression. The mapped tester test activations are classified by the frozen trainer Logistic Regression prober.

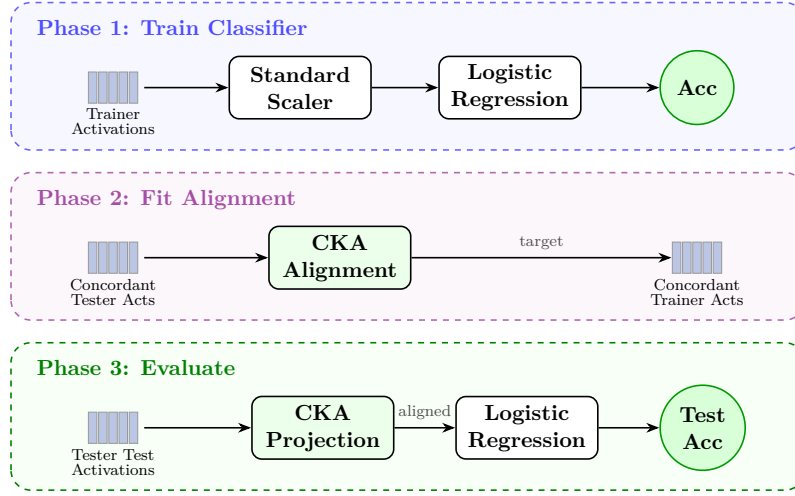


Figure 3.10: CKA approach experimental pipeline

3.5.5 AdapterMLP

In this approach, a non-linear adaptation procedure of the tester model’s latent space towards the trainer model’s space is explored using a low-dimensional alignment network (called AdapterMLP). The objective is to observe whether introducing a non-linear component in the alignment improves universal prober performance, while maintaining a linear classifier (Logistic Regression) on the trainer model.

Architecture and Loss

Some key characteristics:

- **Zero-init:** the last layers of the decomposition are initialized to zero so that the network starts close to a linear identity transformation, allowing the non-linear component to emerge only if truly useful.
- **MixedLoss:** the loss function combines Mean Squared Error (MSE) and a component based on cosine similarity:

$$\mathcal{L} = \alpha \cdot \text{MSE}(\hat{y}, y) + \beta \cdot (1 - \text{cosine_sim}(\hat{y}, y)) \quad (3.7)$$

with configurable weights α (MSE) and β (cosine) to balance fidelity and angular alignment.

where MSE is defined as:

$$\text{MSE}(\hat{y}, y) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (3.8)$$

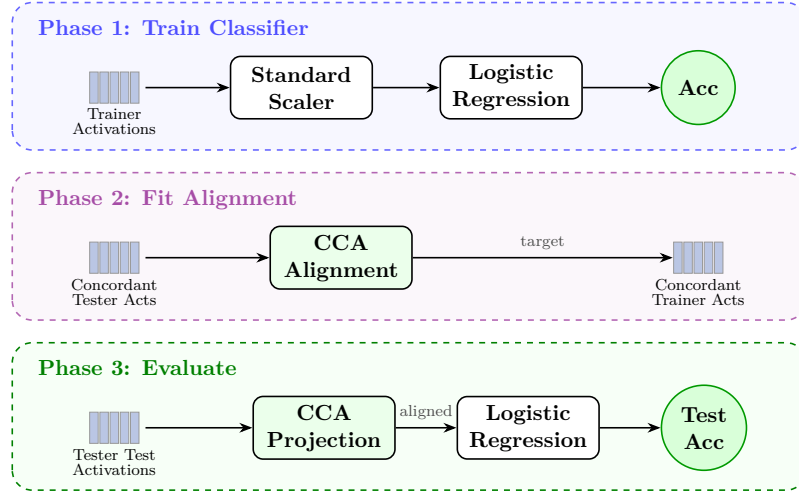


Figure 3.11: CCA approach experimental pipeline

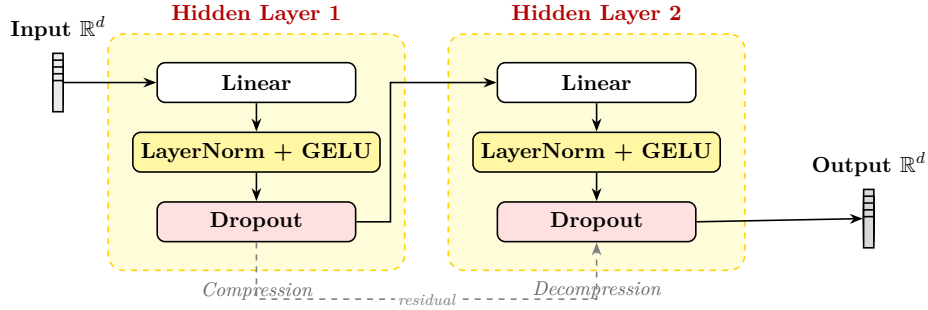


Figure 3.12: AdapterMLP architecture

and cosine similarity is defined as:

$$\text{cosine_sim}(\hat{y}, y) = \frac{\hat{y} \cdot y}{\|\hat{y}\| \|y\|} = \frac{\sum_{i=1}^N \hat{y}_i y_i}{\sqrt{\sum_{i=1}^N \hat{y}_i^2} \sqrt{\sum_{i=1}^N y_i^2}} \quad (3.9)$$

Experimental Pipeline

The experimental pipeline in Figure 3.13 first involves training a probe (Logistic Regression) on the trainer’s activation space using the entire balanced trainer training set; this classifier will remain fixed. Subsequently, the AdapterMLP is trained to project tester activations into the trainer’s space, minimizing the loss between projected activations and original trainer activations using the balanced concordant activations dataset. Finally, the tester’s test activations are projected via the AdapterMLP and classified using the probe trained on the trainer.

For the Logistic Regressor, the following hyperparameters were used:

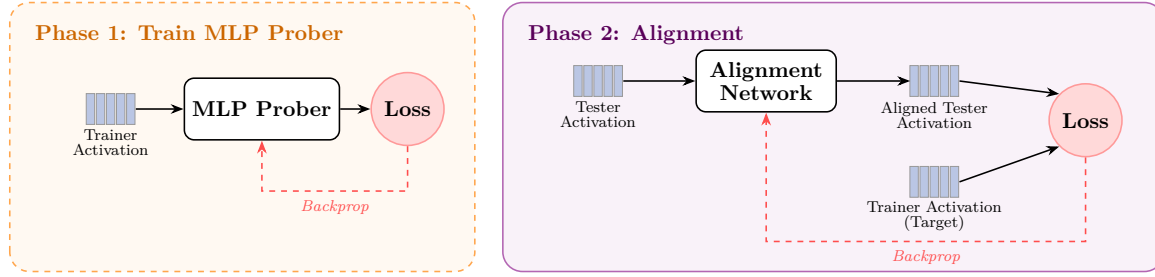


Figure 3.13: AdapterMLP approach experimental pipeline

- **solver:** lbfgs
- **max_iterations:** 10000
- **class_weight:** balanced

In Tab. 3.7, the main hyperparameters used for training the AdapterMLP are reported

Parameter	Value
Hidden Dimension	128
Dropout	0.5
Learning Rate (LR)	1e-3
Weight Decay	1e-1
Batch Size	32
Early Stopping δ	1e-4
Gradient Clipping	1.0
Optimizer	AdamW
Scheduler	CosineAnnealingLR
α (Loss Weight)	0.01
β (Loss Weight)	1.0

Table 3.7: AdapterMLP hyperparameter values apply to all combinations of Trainer, Tester and Layer Type.

3.5.6 Full Non-linear

This approach follows the previous one regarding the alignment part between the two LLMs. The difference lies in the non-linear prober (MLP) as classifier on the trainer. The objective is to verify performance using both non-linear components following the same experimental pipeline.

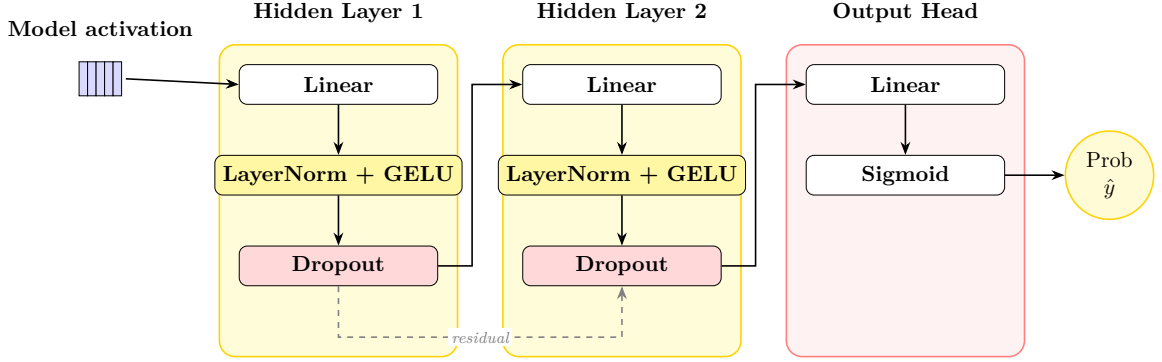


Figure 3.14: MLP Prober architecture of the full non-linear approach

Architecture and Loss

- **AlignmentNetwork:** The same architecture described in the previous approach is used with the same loss
- **MLP Prober in Figure 3.14:** Non-linear classifier for hallucination detection: fully-connected network with normalized intermediate layers (LayerNorm), GELU activation and dropout.
- **BCEWithLogitsLoss:** combines a sigmoid function and Binary Cross Entropy for numerical stability.

BCEWithLogitsLoss is defined as:

$$\mathcal{L}(x, y) = -(y \cdot \log(\sigma(x)) + (1 - y) \cdot \log(1 - \sigma(x))) \quad (3.10)$$

where $\sigma(x)$ is the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (3.11)$$

Experimental Pipeline

The experimental pipeline in Figure 3.15 is therefore identical to that of the AdapterMLP approach, with the difference that the classifier on the trainer is now a non-linear MLP Prober.

In Tab. 3.9 and 3.11, the main hyperparameters used for training the Alignment-Network and MLP Prober are reported

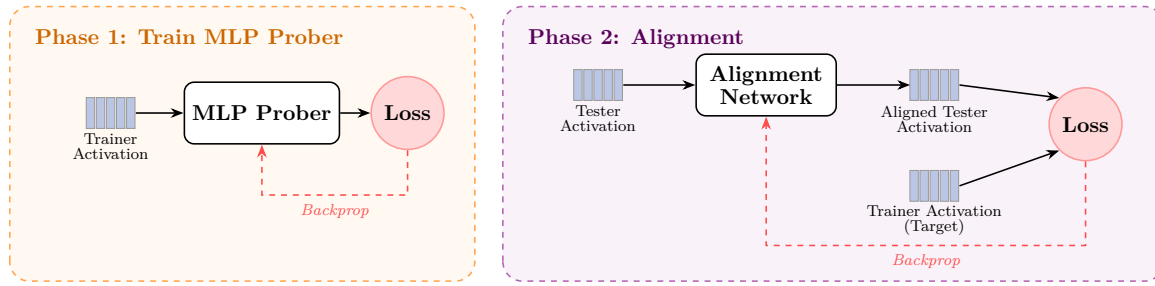


Figure 3.15: Full non-linear approach experimental pipeline

Parameter	Value
<i>Alignment Network</i>	
Hidden Dimension	128
Dropout	0.5
Learning Rate	1e-3
Weight Decay	1e-1
Batch Size	32
Early Stopping Patience	50
<i>Loss Function</i>	
α (Reconstruction)	0.01
β (Contrastive)	1.0

Table 3.9: AlignmentNetwork hyperparameters for the full non-linear approach.

Parameter	Value
Hidden Dimension	64
Dropout	0.5
Learning Rate	1e-3
Weight Decay	1e-2
Batch Size	64
Early Stopping Patience	30
Optimizer	AdamW
Scheduler	CosineAnnealingLR

Table 3.11: MLP Prober hyperparameters for the full non-linear approach.

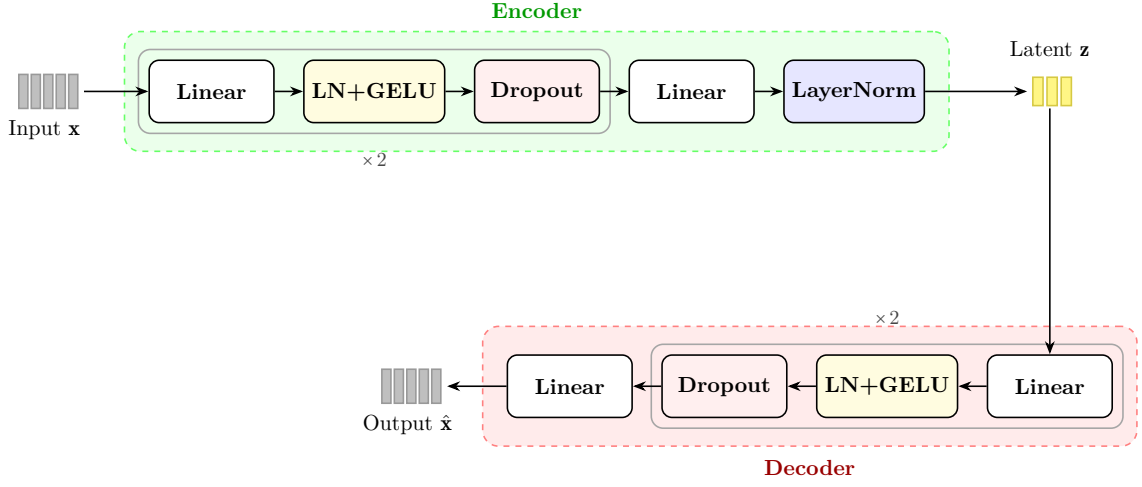


Figure 3.16: Autoencoder architecture of the reduced non-linear approach

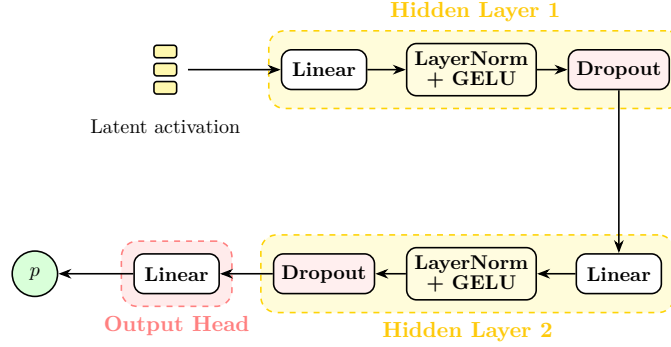


Figure 3.17: MLP Prober architecture of the reduced non-linear approach

3.5.7 Reduced Non-linear Approach

This approach reduces activation dimensionality via autoencoder before alignment. The idea is to verify whether a more compact latent space can improve alignment and classification performance, while still using non-linear components.

Architecture and Loss

- A separate autoencoder 3.16 is trained on the trainer's activation space and one on the tester's. The encoders produce latent representations of common dimension. The loss used is Mean Squared Error (MSE).

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (3.12)$$

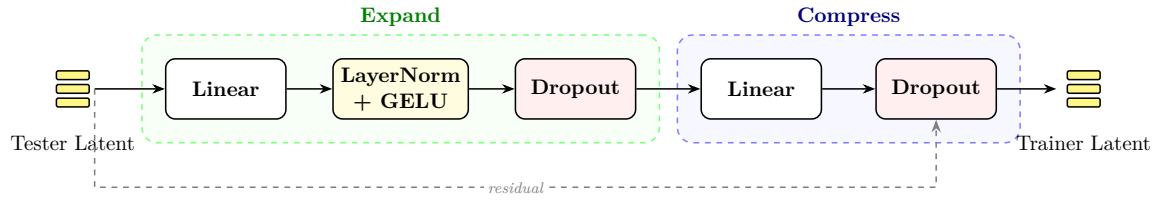


Figure 3.18: AlignmentNetwork architecture of the reduced non-linear approach

- a latent alignment Network in Figure 3.18 maps the tester’s latent space to the trainer’s.
- an MLP Prober in Figure 3.17 classifies activations operating on the reduced latent space of the trainer.

Experimental Pipeline

The experimental pipeline in Figure 3.19 first involves training two autoencoders: one on the trainer’s activations and the other on the tester’s, thus obtaining two distinct latent spaces. Subsequently, the activations are encoded in their respective latent spaces. An MLP classifier is trained on the trainer’s latent space. Then, an Alignment Network is trained that maps the tester’s latent representations to the trainer’s latent space using concordant data. Finally, for evaluation, the MLP classifier trained on the trainer is used to classify the tester’s representations after alignment and dimensionality reduction.

In Tab. 3.13, 3.15 and 3.17, the main updated hyperparameters used in this approach are reported

Parameter	Value
Latent Dimension	128
Hidden Dimension	256
Dropout	0.2
Learning Rate	1e-3
Weight Decay	1e-2
Batch Size	64
Loss	MSELoss

Table 3.13: AutoEncoder hyperparameters (Trainer and Tester).

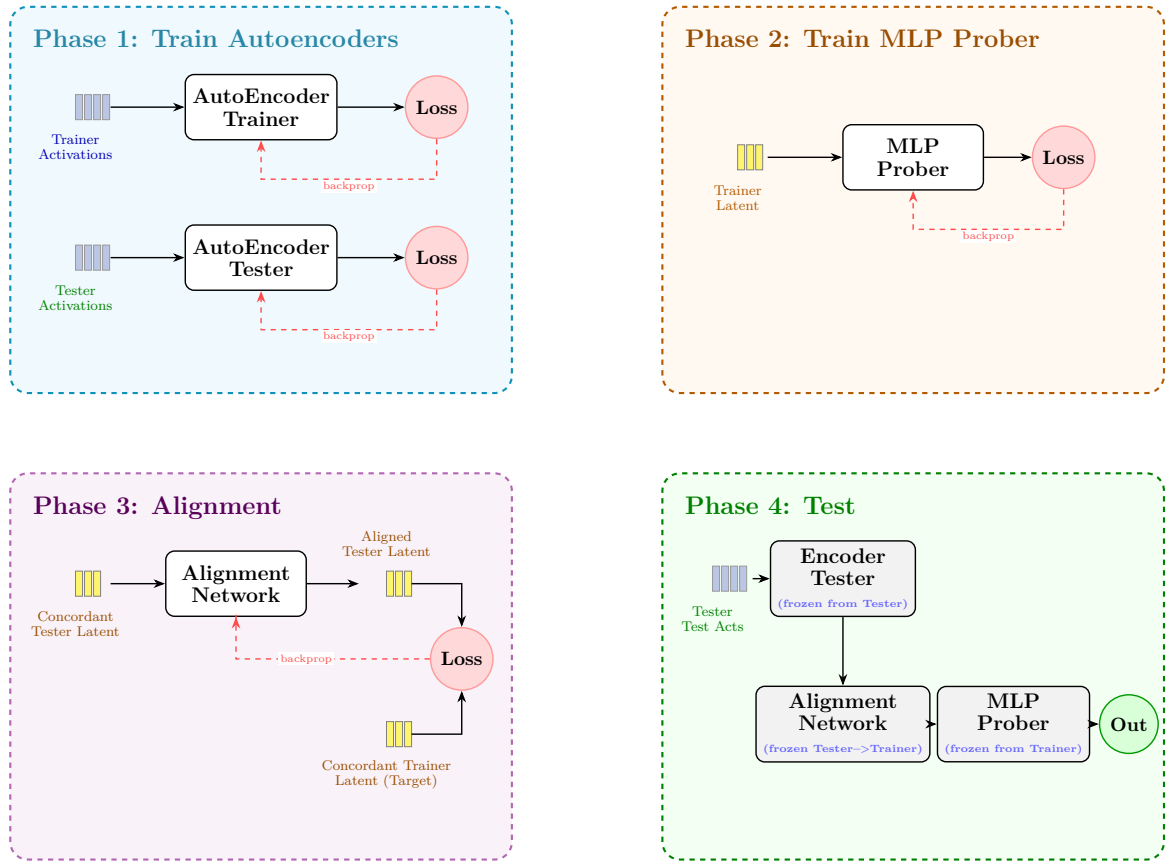


Figure 3.19: Reduced non-linear approach experimental pipeline

Parameter	Value
Hidden Dimension	256
Dropout	0.3
Learning Rate	1e-3
Weight Decay	1e-2
Batch Size	32
<i>Loss Weights</i>	
α (Reconstruction)	0.5
β (Contrastive)	0.5

Table 3.15: AlignmentNetwork hyperparameters for the reduced approach.

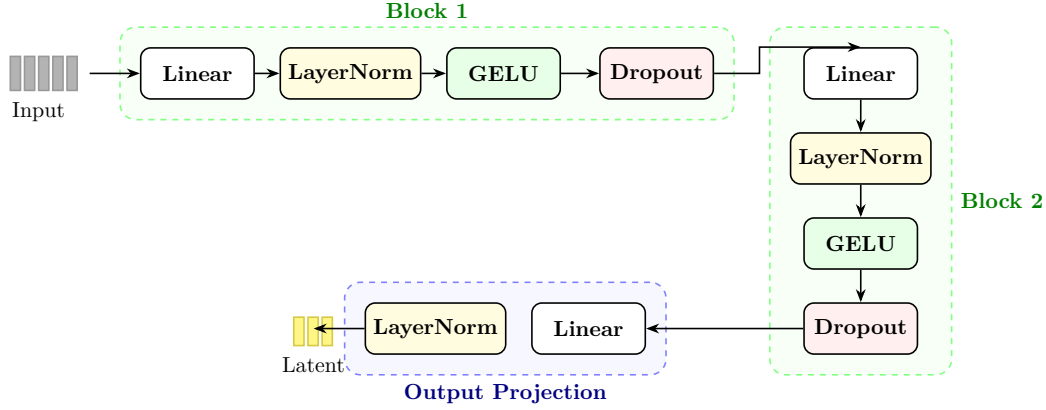


Figure 3.20: One-For-All encoder architecture

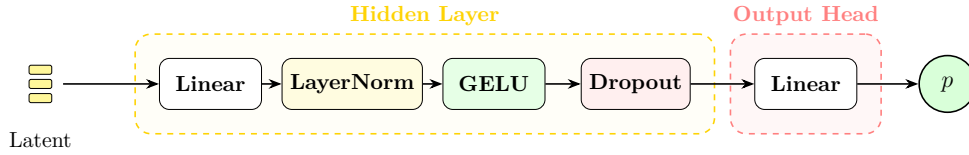


Figure 3.21: One-For-All Classification Head architecture

Parameter	Value
Hidden Dimension	64
Dropout	0.3
Learning Rate	1e-3
Batch Size	64
Loss	BCEWithLogitsLoss

Table 3.17: MLP Prober hyperparameters (on latent space).

3.5.8 One-For-All Approach

One-For-All represents the proposed *universal prober*: a two-phase procedure is adopted, designed to evaluate the transferability of a decision function (the classification head) between two different models while keeping the same head and adapting only the tester’s encoder. Unlike previous approaches, this method does not require concordant data for alignment but uses independent balanced datasets for each model.

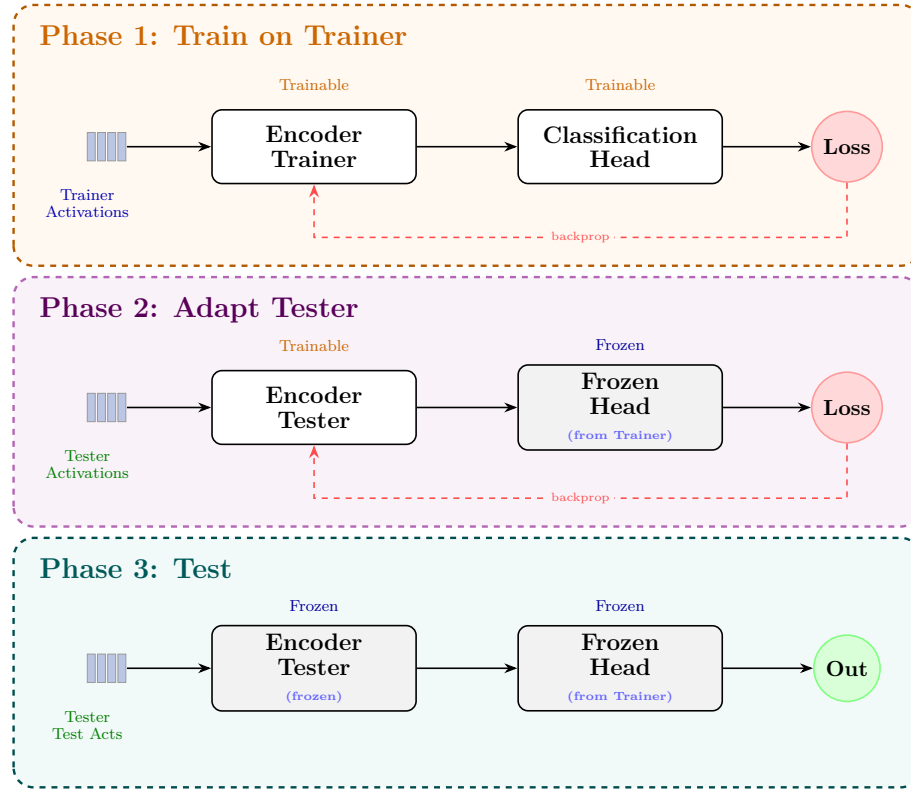


Figure 3.22: One-For-All experimental pipeline

Architecture

- the encoder in Figure 3.20 is a multi-block feed-forward network that maps the input space (activation dimension) to a reduced-dimension latent space (256).
- the classification Head in Figure 3.21 is a compact module that takes the latent vector as input and returns a binary logit. Implemented as a small MLP.

Experimental pipeline

The experimental pipeline in Figure 3.22 first involves joint training of the encoder and classification head on the trainer model’s activation space (End-to-End). Once training is complete, the head is frozen (*Frozen Head*) to keep the learned decision function unchanged. Subsequently, a new encoder is trained on the tester model’s activation space: the output of this new encoder is passed to the trainer’s frozen head, and the loss is calculated directly on the classification error (BCEWithLogitsLoss). In this way, the tester learns to produce latent representations compatible with the trainer’s “head”.

Tab. 3.19 and 3.21 report the main hyperparameters used for training the Encoders and Classification Head:

Parameter	Value
Latent Dimension	256
Hidden Dimension	512
Dropout	0.3
Learning Rate	1e-3
Weight Decay	1e-2
Batch Size	64
Early Stopping Patience	15
Optimizer	AdamW

Table 3.19: Encoder hyperparameters (Trainer and Tester Adapter).

Parameter	Value
Latent Dimension (Input)	256
Hidden Dimension	128
Dropout	0.3
Learning Rate	1e-3
Batch Size	64

Table 3.21: Classification Head hyperparameters (Shared/Frozen).

3.6 Hallucination mitigation steering

This section describes the methodological approach to build a universal steering mechanism for hallucination mitigation in language models. In particular two different steering techniques are explored: **TruthX** [48] and **SLiMTruth**. The first one is a known technique in literature to reduce hallucinations in LLMs, while the second one is an adaptation of the original SLiM [36] implementation, that was originally developed to steer the style of LLM’s answers, to the hallucination mitigation task.

In both cases the dataset is divided in two subsets:

- **Training Dataset:** subset of the original dataset used to train the steering mechanism. 80% of the original dataset is used to train the steering mechanism, while the remaining 20% is used as validation set to perform early stopping during training.

- **Evaluation Dataset:** subset of the original dataset used to evaluate the performance of the steering mechanism. The evaluation dataset is completely unseen during training and is used to evaluate the generalization ability of the steering mechanism.

Dataset	Num. Pairs	Type	True Example	False Example
BeliefBankFacts	6500	World facts	<i>The sky is blue</i>	<i>The sky is not blue</i>
BeliefBankConstraints	6500	Logical constraints	<i>If it rains, the ground gets wet</i>	<i>If it rains, the ground doesn't get wt</i>
HaluEval	2500	Context-based QA	<i>Paris</i> (given a context)	<i>London</i> (given a context)

Table 3.23: Description of pairs for each dataset used in steering experiments.

3.6.1 TruthX

TruthX is a steering technique designed to reduce hallucinations in LLMs by operating directly on the internal representations of the model during inference. The core idea is based on Disentangled Representation Learning [43]: the internal activations of each layer are decomposed into two distinct latent spaces: a *semantic space* that captures the meaning/content of the statement and a *truthful space* that encodes its veracity. By manipulating only the truthful component while leaving the semantic content untouched, TruthX can steer the model’s output towards more truthful generations without altering the intended meaning. The overall idea is illustrated in Figure 3.23.

The TruthX pipeline involves four main phases:

1. **Dataset Preparation:** construction of paired (truthful/hallucinated);
2. **Training of the Autoencoder** on paired activations (truthful/hallucinated) to learn the disentangled representation;
3. **Computation of steering vectors** (δ) and probing-based selection of the most informative layers;
4. **Inference-time editing** of the internal representations via forward hooks on the selected modules.

Dataset Preparation

TruthX requires paired samples: for each statement, a truthful version and a hallucinated version sharing the same semantic content are needed.

For each pair (see tab. 3.23), the internal activations of the LLM are extracted at all layers and for both module types (attention and MLP/FFN), storing the activation

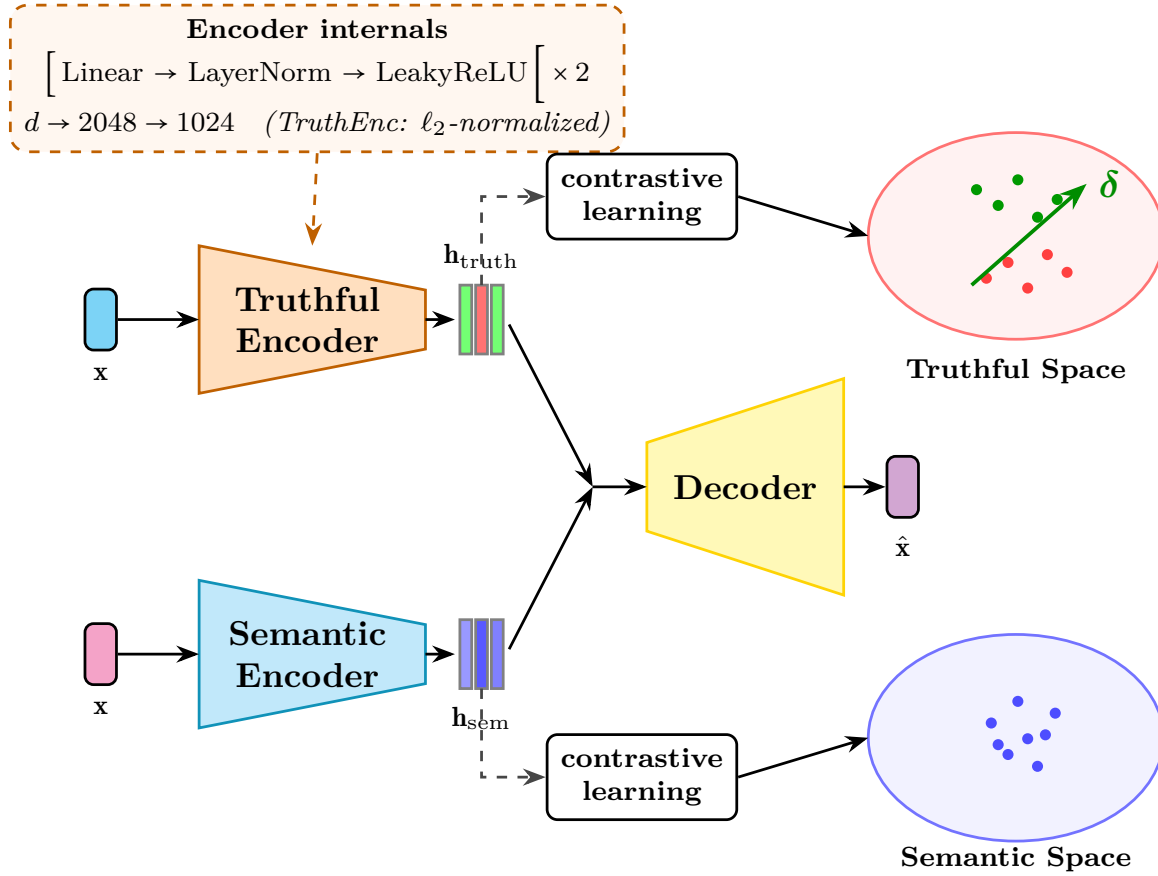


Figure 3.23: TruthX model architecture overview

of the last input token. This produces a set of *virtual layers*: for a language model with L physical layers, there are $2L$ virtual layers (one for each attention and one for each MLP module). True instances are assigned even instance IDs while false instances receive odd IDs, enabling deterministic pairing.

Autoencoder Architecture

The autoencoder is composed of three sub-networks:

- **Semantic Encoder** (SemEnc): maps the input activation $\mathbf{x} \in \mathbb{R}^d$ (where d is the model's hidden size, e.g. 4096 for Llama-3-8B) into a semantic latent representation $\mathbf{h}_{\text{sem}} \in \mathbb{R}^{1024}$. Architecture: $d \rightarrow 2048 \rightarrow 1024$, with LayerNorm and LeakyReLU after each linear layer.
- **Truthful Encoder** (TruthEnc): maps the input activation into a truthful latent representation $\mathbf{h}_{\text{truth}} \in \mathbb{R}^{1024}$, L2-normalized. Same architecture as the Semantic Encoder but with L2 normalization on the output.

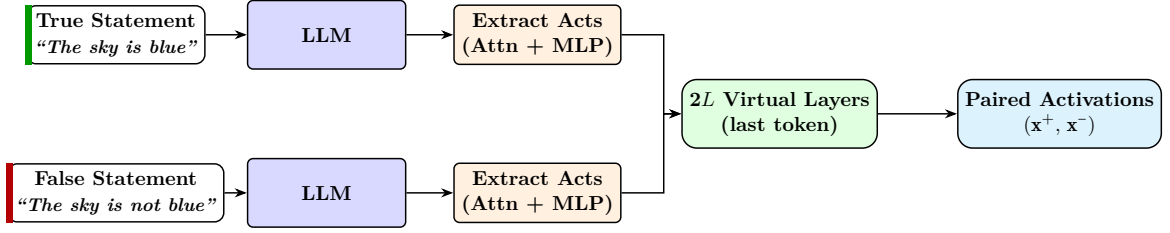


Figure 3.24: TruthX extraction overview

- **Decoder (Dec)**: reconstructs the original activation from the combined latent representation. Architecture: $1024 \rightarrow 2048 \rightarrow d$.

The combination of semantic and truthful representations is performed via a **Cross-Attention** mechanism:

$$\mathbf{z} = \mathbf{h}_{\text{sem}} + \text{Attn}(\mathbf{h}_{\text{sem}}, \mathbf{h}_{\text{truth}}, \mathbf{h}_{\text{truth}}) \quad (3.13)$$

where Attn is a single-head multi-head attention module. The final reconstruction is:

$$\hat{\mathbf{x}} = \text{Dec}(\mathbf{z}) \quad (3.14)$$

Loss Function

The total training loss combines four components following the TruthX paper:

$$\mathcal{L} = w_r \cdot \mathcal{L}_{\text{recon}} + w_c \cdot \mathcal{L}_{\text{sem}} + w_t \cdot \mathcal{L}_{\text{truth}} + w_e \cdot \mathcal{L}_{\text{edit}} \quad (3.15)$$

Reconstruction Loss ($\mathcal{L}_{\text{recon}}$, Eq. 3):

Ensures the autoencoder can faithfully reconstruct the original activations:

$$\mathcal{L}_{\text{recon}} = \frac{1}{2} (\text{MSE}(\hat{\mathbf{x}}_{\text{pos}}, \mathbf{x}_{\text{pos}}) + \text{MSE}(\hat{\mathbf{x}}_{\text{neg}}, \mathbf{x}_{\text{neg}})) \quad (3.16)$$

Semantic Contrastive Loss ($\mathcal{L}_{\text{sem}}$, Eq. 6):

Forces the Semantic Encoder to produce similar representations for paired samples (same semantic content, opposite veracity) while separating semantically different samples:

$$\mathcal{L}_{\text{sem}} = \text{CTR}(\mathbf{h}_{\text{sem}}^{\text{pos}}, \{\mathbf{h}_{\text{sem}}^{\text{neg}}\}, H_{\text{sem}}^{\text{pos}} \setminus \{\mathbf{h}_{\text{sem}}^{\text{pos}}\}) + \text{CTR}(\mathbf{h}_{\text{sem}}^{\text{neg}}, \{\mathbf{h}_{\text{sem}}^{\text{pos}}\}, H_{\text{sem}}^{\text{neg}} \setminus \{\mathbf{h}_{\text{sem}}^{\text{neg}}\}) \quad (3.17)$$

where $\text{CTR}(\mathbf{s}, S^+, S^-)$ is the contrastive function:

$$\text{CTR}(\mathbf{s}, S^+, S^-) = -\log \frac{\sum_{\mathbf{s}' \in S^+} \exp[\text{sim}(\mathbf{s}, \mathbf{s}')/\tau]}{\sum_{\mathbf{s}' \in S^+ \cup S^-} \exp[\text{sim}(\mathbf{s}, \mathbf{s}')/\tau]} \quad (3.18)$$

with temperature $\tau = 0.1$ and cosine similarity $\text{sim}(\cdot, \cdot)$.

Truthful Contrastive Loss ($\mathcal{L}_{\text{truth}}$, Eq. 5):

Separates truthful representations of positive samples from those of negative samples in the truthful latent space:

$$\mathcal{L}_{\text{truth}} = \text{CTR}(\mathbf{h}_{\text{truth}}^{\text{pos}}, H_{\text{truth}}^{\text{pos}}, H_{\text{truth}}^{\text{neg}}) + \text{CTR}(\mathbf{h}_{\text{truth}}^{\text{neg}}, H_{\text{truth}}^{\text{neg}}, H_{\text{truth}}^{\text{pos}}) \quad (3.19)$$

Editing Loss ($\mathcal{L}_{\text{edit}}$, Eq. 10):

Ensures that swapping the truthful component between a positive and a negative sample yields a reconstruction close to the opposite sample:

$$\mathbf{x}^{\text{pos} \rightarrow \text{neg}} = \text{Dec}[\mathbf{h}_{\text{sem}}^{\text{pos}} + \text{Attn}(\mathbf{h}_{\text{sem}}^{\text{pos}}, \mathbf{h}_{\text{truth}}^{\text{neg}}, \mathbf{h}_{\text{truth}}^{\text{neg}})] \quad (3.20)$$

$$\mathbf{x}^{\text{neg} \rightarrow \text{pos}} = \text{Dec}[\mathbf{h}_{\text{sem}}^{\text{neg}} + \text{Attn}(\mathbf{h}_{\text{sem}}^{\text{neg}}, \mathbf{h}_{\text{truth}}^{\text{pos}}, \mathbf{h}_{\text{truth}}^{\text{pos}})] \quad (3.21)$$

$$\mathcal{L}_{\text{edit}} = \text{MSE}(\mathbf{x}^{\text{pos} \rightarrow \text{neg}}, \mathbf{x}_{\text{neg}}) + \text{MSE}(\mathbf{x}^{\text{neg} \rightarrow \text{pos}}, \mathbf{x}_{\text{pos}}) \quad (3.22)$$

Training Procedure

The training procedure processes all $2L$ virtual layers per epoch. For each virtual layer, the paired activations (positive and negative) are loaded and organized into structured batches where `batch_pos[i]` and `batch_neg[i]` always form a semantically aligned pair. The dataset is split into 80% training and 20% validation using a global permutation that ensures each pair remains entirely in one set.

The optimizer used is AdamW with a Cosine Annealing with Warm Restarts learning rate scheduler, preceded by a linear warmup phase. Early stopping on validation loss with configurable patience prevents overfitting.

Parameter	Value
Semantic Latent Dim	1024
Truthful Latent Dim	1024
Semantic Hidden Dims	[2048]
Truthful Hidden Dims	[2048]
Decoder Hidden Dims	[2048]
Batch Size	512
Learning Rate	1e-4
Weight Decay	1e-5
Warmup Epochs	5
Temperature (τ)	0.1
Optimizer	AdamW
Scheduler	CosineAnnealingWarmRestarts
Early Stopping Patience	10
Seed	42

Table 3.25: TruthX Autoencoder training hyperparameters.

Steering Vector Computation and Layer Selection

After training, the autoencoder is used to compute the **steering vectors** for each virtual layer. For all paired activations, the Truthful Encoder produces the truthful representations $\mathbf{h}_{\text{truth}}^{\text{pos}}$ and $\mathbf{h}_{\text{truth}}^{\text{neg}}$. The positive and negative centers are computed as the means:

$$\bar{H}_{\text{truth}}^{\text{pos}} = \frac{1}{N} \sum_{i=1}^N \mathbf{h}_{\text{truth},i}^{\text{pos}}, \quad \bar{H}_{\text{truth}}^{\text{neg}} = \frac{1}{N} \sum_{i=1}^N \mathbf{h}_{\text{truth},i}^{\text{neg}} \quad (3.23)$$

The steering vector $\boldsymbol{\delta}$ for each virtual layer is (Eq. 12):

$$\boldsymbol{\delta} = \bar{H}_{\text{truth}}^{\text{pos}} - \bar{H}_{\text{truth}}^{\text{neg}} \quad (3.24)$$

To select which modules to edit during inference, a **probing-based layer selection** (Eq. 15) is employed. For each virtual layer, a simple probing classifier is evaluated:

$$\text{Probe}(\mathbf{x}) = \begin{cases} \text{pos} & \text{if } \text{sim}(\mathbf{h}_{\text{truth}}, \bar{H}_{\text{truth}}^{\text{pos}}) \geq \text{sim}(\mathbf{h}_{\text{truth}}, \bar{H}_{\text{truth}}^{\text{neg}}) \\ \text{neg} & \text{otherwise} \end{cases} \quad (3.25)$$

The virtual layers are ranked by probing accuracy and the **top- k** modules are selected for editing. For instance, for a 32-layer LLM with $k = 10$, TruthX selects the 10

modules with the highest probing accuracy out of the total 64 modules (32 attention + 32 MLP).

Inference-Time Editing

During inference, TruthX modifies the model’s internal representations in real-time by registering forward hooks on the selected top- k modules. For each selected module, when a forward pass produces an activation $\mathbf{X}$, the editing procedure is:

1. Extract the truthful representation: $\mathbf{h}_{\text{truth}} = \text{TruthEnc}(\mathbf{X})$
2. Compute the positive and negative edited reconstructions by shifting the truthful representation along $\boldsymbol{\delta}$:

$$\hat{\mathbf{X}}_{\text{pos}} = \text{Dec}(\mathbf{h}_{\text{sem}} + \text{Attn}(\mathbf{h}_{\text{sem}}, F_{\text{norm}}(\mathbf{h}_{\text{truth}} + \boldsymbol{\delta}), F_{\text{norm}}(\mathbf{h}_{\text{truth}} + \boldsymbol{\delta}))) \quad (3.26)$$

$$\hat{\mathbf{X}}_{\text{neg}} = \text{Dec}(\mathbf{h}_{\text{sem}} + \text{Attn}(\mathbf{h}_{\text{sem}}, F_{\text{norm}}(\mathbf{h}_{\text{truth}} - \boldsymbol{\delta}), F_{\text{norm}}(\mathbf{h}_{\text{truth}} - \boldsymbol{\delta}))) \quad (3.27)$$

where F_{norm} denotes L2 normalization.

3. Compute the final editing direction:

$$\Delta = \hat{\mathbf{X}}_{\text{pos}} - \hat{\mathbf{X}}_{\text{neg}} \quad (3.28)$$

4. Apply the steering:

$$\mathbf{X}' = \mathbf{X} + \alpha \cdot \Delta \quad (3.29)$$

where α is the `edit_strength` parameter controlling the intensity of the intervention.

The editing is applied in a fully differentiable manner, modifying the activations before they are passed to the next layer.

Cross-Model TruthX via Alignment

A key contribution of this work is the generalization of TruthX to a **cross-model** setting, where the autoencoder and steering vectors are trained on a *trainer model* (e.g., Gemma-2-9B-IT) and applied to a different *target model* (e.g., Llama-3.1-8B-Instruct). This requires solving two problems: (i) identifying *which* layers of the two models are structurally most comparable, and (ii) learning a linear map between their activation spaces.

Layer Selection

Since the two models may have different numbers of layers and different hidden dimensions, a direct layer-to-layer correspondence cannot be assumed. The Procrustes aligner determines this correspondence automatically using **Linear CKA** (Centered Kernel Alignment), a similarity index between representation matrices that is invariant to orthogonal transformations and isotropic scaling.

Given two centered activation matrices $\mathbf{X}_c \in \mathbb{R}^{n \times d_x}$ and $\mathbf{Y}_c \in \mathbb{R}^{n \times d_y}$, Linear CKA is defined as:

$$\text{CKA}(\mathbf{X}, \mathbf{Y}) = \frac{\|\mathbf{X}_c^T \mathbf{Y}_c\|_F^2}{\|\mathbf{X}_c^T \mathbf{X}_c\|_F \cdot \|\mathbf{Y}_c^T \mathbf{Y}_c\|_F} \quad (3.30)$$

CKA takes values in $[0, 1]$, where 1 indicates identical (up to rotation/scaling) representations and 0 indicates orthogonal ones.

The layer selection procedure is as follows:

1. The **top-1 target module** is identified via the probing-based layer selection (see Section above): this is the module of the target model whose truthful representations best separate true and false statements.
2. For each layer of the **trainer model** of the same module type (attention or MLP), Linear CKA is computed between the trainer layer’s activations and the target’s top-1 module activations, on a shared set of instances.
3. The **trainer layer that maximises CKA** with the target top-1 is selected as the alignment anchor. Intuitively, this is the trainer layer whose representation geometry most resembles the target layer.

Procrustes Aligner.

Once the layer pair is identified, the **Orthogonal Procrustes** method finds the optimal rotation (with scale and translation) that maps one activation space into the other, minimising the Frobenius-norm reconstruction error. Given centered activation matrices $\mathbf{X}_c$ (source) and $\mathbf{Y}_c$ (target), the SVD of their cross-covariance is computed:

$$\mathbf{U} \mathbf{\Sigma} \mathbf{V}^T = \text{SVD}[\mathbf{X}_c^T \mathbf{Y}_c] \quad (3.31)$$

The optimal rotation and scale factor are:

$$\mathbf{R} = \mathbf{U} \mathbf{V}^T, \quad s = \frac{\sum_i \sigma_i}{\|\mathbf{X}_c\|_F^2} \quad (3.32)$$

so that the full affine transformation is:

$$\hat{\mathbf{Y}} = \mathbf{X} \mathbf{A} + \mathbf{b}, \quad \mathbf{A} = s \cdot \mathbf{R}, \quad \mathbf{b} = \bar{\mathbf{y}} - \bar{\mathbf{x}} \mathbf{A} \quad (3.33)$$

where $\bar{\mathbf{x}}$ and $\bar{\mathbf{y}}$ are the empirical means of the source and target activations. By constraining the map to be a scaled orthogonal transformation, Procrustes preserves angular relationships between representations, unlike unconstrained linear regression.

Two maps are fitted independently:

- $\mathbf{A}_{t \rightarrow tr}, \mathbf{b}_{t \rightarrow tr}$: **Target** $\rightarrow$ **Trainer** (pre-alignment);
- $\mathbf{A}_{tr \rightarrow t}, \mathbf{b}_{tr \rightarrow t}$: **Trainer** $\rightarrow$ **Target** (post-alignment).

Cross-Model Inference Pipeline.

During inference on the target model, TruthX editing proceeds as follows for each selected module:

1. **Pre-alignment:** the target model’s activation $\mathbf{x}_{\text{target}} \in \mathbb{R}^{d_{\text{target}}}$ is mapped into the trainer’s space:

$$\tilde{\mathbf{x}} = \mathbf{x}_{\text{target}} \mathbf{A}_{t \rightarrow tr} + \mathbf{b}_{t \rightarrow tr} \quad (3.34)$$

2. **TruthX editing:** $\tilde{\mathbf{x}}$ is processed by the trainer’s autoencoder to produce the edited activation $\tilde{\mathbf{x}}'$ (as described in Section 3.6.1).
3. **Post-alignment:** the edited activation is mapped back into the target model’s space:

$$\mathbf{x}'_{\text{target}} = \tilde{\mathbf{x}}' \mathbf{A}_{tr \rightarrow t} + \mathbf{b}_{tr \rightarrow t} \quad (3.35)$$

This symmetrical pre/post-alignment strategy ensures that the TruthX intervention operates entirely in the trainer’s representation space, while the final output is expressed in the target model’s native space.

Experimental Configuration

All the TruthX experiments reported in the next chapter (Standard, Cross-Dataset, and Cross-Model) share the same configuration: contrastive weight 1.0, editing weight 1.0, reconstruction weight 0.0, temperature $\tau = 0.1$, edit strength $\alpha = 1$, and top- k modules $k = 1$. In the cross-model setting, several strategies for aligning the two representation spaces were compared: no alignment (direct transfer), a linear map, an MLP, a two-layer MLP, and Procrustes. The cross-model results in the next chapter use the Procrustes alignment, which gave the most stable behaviour.

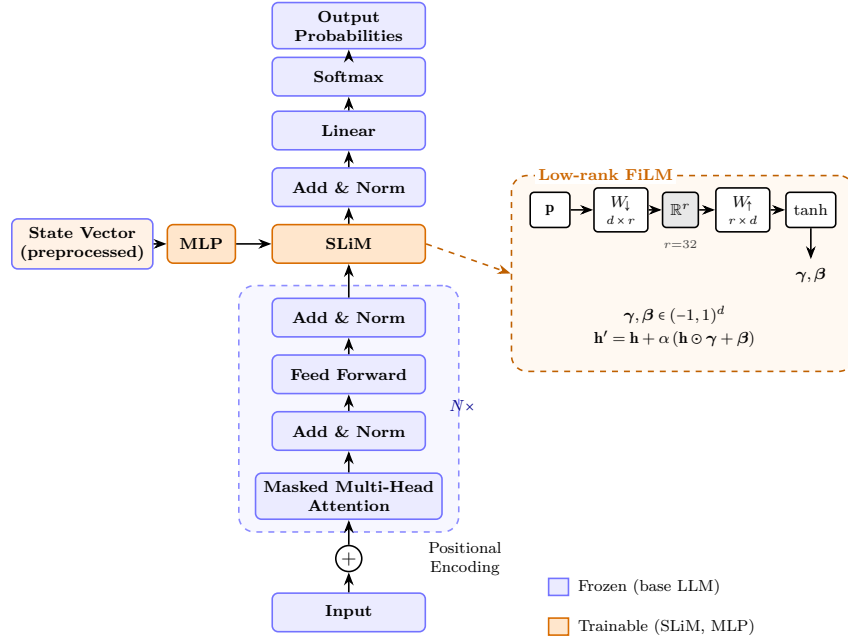


Figure 3.25: SLiMTruth architecture

3.6.2 SLiMTruth

SLiMTruth is an adaptation of the **SLiM** framework [36], originally designed to steer the stylistic attributes of LLM outputs, to the task of hallucination mitigation. The core idea is to apply a lightweight **FiLM** (Feature-wise Linear Modulation) transformation to the hidden representations of a single transformer layer, conditioned on an external state vector, so as to push the model’s internal representations toward more truthful generations.

Unlike TruthX, which decomposes activations into disentangled semantic/truthful latent spaces via an autoencoder, SLiMTruth operates directly in the model’s native hidden space: it learns a *scale* and a *shift* vector that multiplicatively and additively modulate the layer output, respectively. This results in an extremely parameter-efficient intervention (only the SLiM modules are trained; the base LLM remains completely frozen). Figure 3.25 shows where the SLiM module is injected into the frozen decoder and details its low-rank structure.

The SLiMTruth pipeline involves three main phases:

1. **Dataset Preparation:** construction of paired (truthful/hallucinated) samples from the same datasets used for TruthX (see Tab. 3.23);
2. **Training** of the SLiM modules with a hybrid loss combining cross-entropy, contrastive, and regularization objectives;
3. **Inference-time modulation** via a forward hook on the target layer.

Dataset Preparation

SLiMTruth uses the same paired datasets described in Tab. 3.23.

Crucially, during training both positive and negative samples receive the same state value $s = 1.0$. The SLiM transformation is therefore identical for both classes: scale and shift act as content-dependent feature selectors, and the model must learn to separate truthful from hallucinated representations based on the *content* of the hidden states, not on label information leaked via the state.

In particular SLiM is trained on a single target layer, which can be provided as an argument. In particular, for these experiments for each pair (model,dataset) the best layer from premilinar analysis was selected as target layer for SLiMTruth training. See Figures 3.3 3.4 3.5

Architecture

SLiMTruth wraps a frozen LLM and injects a FiLM-like modulation at the output of a single user-specified transformer layer (the *target layer*). The architecture is composed of three lightweight modules:

State Projector (StateBlock).

Maps the scalar state $s \in \mathbb{R}$ into the model’s hidden space:

$$\mathbf{p} = \text{LayerNorm}[\text{ReLU}(\mathbf{W}_s \cdot s + \mathbf{b}_s)] \in \mathbb{R}^d \quad (3.36)$$

where d is the hidden size of the LLM (e.g. 4096 for Llama-3.1-8B). The projector has architecture $1 \rightarrow d$ with ReLU activation and LayerNorm.

Low-Rank Scale (LowRankLinear).

Computes the multiplicative modulation vector via a low-rank factorization:

$$\boldsymbol{\gamma} = \tanh[\mathbf{W}_{\text{up}}^{\gamma} (\mathbf{W}_{\text{down}}^{\gamma} \mathbf{p}) + \mathbf{b}^{\gamma}] \in \mathbb{R}^d \quad (3.37)$$

where $\mathbf{W}_{\text{down}}^{\gamma} \in \mathbb{R}^{r \times d}$ and $\mathbf{W}_{\text{up}}^{\gamma} \in \mathbb{R}^{d \times r}$, with r the rank of the factorization (default $r = 32$). This reduces the parameter count from d^2 to $r \cdot (2d)$.

Low-Rank Shift (LowRankLinear).

Computes the additive modulation vector with the same architecture:

$$\boldsymbol{\beta} = \tanh[\mathbf{W}_{\text{up}}^{\beta} (\mathbf{W}_{\text{down}}^{\beta} \mathbf{p}) + \mathbf{b}^{\beta}] \in \mathbb{R}^d \quad (3.38)$$

Both scale and shift are initialized to produce **zero output at initialization** ($\mathbf{W}_{\text{up}}$ initialized to zero), so that the initial modulation is the identity transformation and training starts from the unmodified model behavior.

FiLM Modulation.

Given the hidden state $\mathbf{h} \in \mathbb{R}^{B \times S \times d}$ at the output of the target layer, the modulation is:

$$\boldsymbol{\delta} = \mathbf{h} \odot \boldsymbol{\gamma} + \boldsymbol{\beta} \quad (3.39)$$

$$\mathbf{h}' = \mathbf{h} + \alpha \cdot \boldsymbol{\delta} \quad (3.40)$$

where $\odot$ denotes element-wise multiplication, and $\alpha \in [0, 1]$ is the **steering strength** parameter that controls the intensity of the intervention. When $\alpha = 0$ the output is unmodified; when $\alpha = 1$ the full modulation is applied. The tanh activation bounds $\boldsymbol{\gamma}, \boldsymbol{\beta} \in (-1, 1)$, preventing unbounded perturbations.

Loss Function

The training loss combines three components in a single hybrid objective:

$$\mathcal{L} = \lambda_{\text{ce}} \cdot \mathcal{L}_{\text{CE}} + \lambda_{\text{nce}} \cdot \mathcal{L}_{\text{InfoNCE}} + \lambda_{\text{reg}} \cdot \mathcal{L}_{\text{Reg}} \quad (3.41)$$

Cross-Entropy Loss ($\mathcal{L}_{\text{CE}}$).

Standard causal language modeling loss on the next-token prediction, computed separately on positive and negative samples and averaged:

$$\mathcal{L}_{\text{CE}} = \frac{1}{2} \left[\text{CE}(\hat{\mathbf{y}}_{\text{pos}}, \mathbf{y}_{\text{pos}}) + \text{CE}(\hat{\mathbf{y}}_{\text{neg}}, \mathbf{y}_{\text{neg}}) \right] \quad (3.42)$$

where $\hat{\mathbf{y}}$ are the logits produced by the modulated model on the steered hidden states. By default, the CE loss is computed **only on the answer tokens**, so that the model is not penalized for the prompt generation and focuses on steering the answer.

InfoNCE Contrastive Loss ($\mathcal{L}_{\text{InfoNCE}}$).

Adapted from TruthX’s truthful contrastive loss. Given a batch of N paired samples, the representations $\mathbf{h}_{\text{pos},i}, \mathbf{h}_{\text{neg},i} \in \mathbb{R}^d$ are L2-normalized and the bidirectional InfoNCE loss is:

$$\mathcal{L}_{\text{InfoNCE}} = \text{CTR}(\mathbf{h}_{\text{pos}}, H_{\text{pos}}, H_{\text{neg}}) + \text{CTR}(\mathbf{h}_{\text{neg}}, H_{\text{neg}}, H_{\text{pos}}) \quad (3.43)$$

with the same contrastive function CTR and temperature $\tau = 0.1$ as in TruthX. This term forces the SLiM-modulated representations to cluster truthful samples together and separate them from hallucinated ones in the hidden space.

Regularization Loss ($\mathcal{L}_{\text{Reg}}$).

An L2 penalty on the scale and shift activations to keep the modulation close to identity and avoid degenerate generations:

$$\mathcal{L}_{\text{Reg}} = \|\boldsymbol{\gamma}\|_2^2 + \|\boldsymbol{\beta}\|_2^2 \quad (3.44)$$

Training Procedure

The training procedure processes paired batches of (positive, negative) samples. For each batch:

1. A **forward pass** is performed on positive samples with state $s = 1.0$; the SLiM hook captures the modulated hidden state and the model produces logits for CE loss.
2. A second forward pass is performed on negative samples with the same state $s = 1.0$; the hook captures the modulated hidden state.
3. The **last-token representations** are extracted from both passes for the InfoNCE loss.
4. The three loss components are combined and back-propagated; gradients flow only through the SLiM parameters (scale, shift, state projector), while the base model remains frozen.

The dataset is split into 80% training and 20% validation with a fixed random seed. The optimizer is **AdamW** with a **linear schedule with warmup**. Early stopping on validation loss (hybrid objective) with configurable patience prevents overfitting. Gradient accumulation and gradient clipping (max norm 1.0) are used for stable training with small batch sizes.

Parameter	Value
State Dimension	1 (scalar)
Low-Rank factorization rank r	32
Batch Size	4
Gradient Accumulation Steps	4
Learning Rate	1e-5
Weight Decay	0.01
Warmup Steps	100
Temperature (τ) InfoNCE	0.1
λ_{ce}	1.0
λ_{nce}	0.1
λ_{reg}	1e-3
Training α	1.0
Optimizer	AdamW
Scheduler	Linear with Warmup
Early Stopping Patience	4
Validation Split	20%
Seed	42

Table 3.27: SLiMTruth training hyperparameters.

Inference-Time Modulation

During inference, the trained SLiM modules are loaded from a checkpoint and a forward hook is registered on the target transformer layer. When the forward pass reaches the target layer and produces the hidden state $\mathbf{h}$, the hook applies the FiLM modulation:

1. The state vector $s = 1.0$ is projected: $\mathbf{p} = \text{StateBlock}(s)$
2. Scale and shift are computed: $\boldsymbol{\gamma} = \tanh(\text{Scale}(\mathbf{p}))$, $\boldsymbol{\beta} = \tanh(\text{Shift}(\mathbf{p}))$
3. The modulation is applied: $\mathbf{h}' = \mathbf{h} + \alpha \cdot (\mathbf{h} \odot \boldsymbol{\gamma} + \boldsymbol{\beta})$
4. The modulated $\mathbf{h}'$ replaces $\mathbf{h}$ and is passed to subsequent layers.

The steering strength α can be tuned at inference time independently from the training value, allowing to explore the trade-off between hallucination reduction and fluency.

Cross-Model SLiMTruth via Alignment

Analogously to TruthX (see Section 3.6.1), SLiMTruth can be transferred to a different target model via Procrustes alignment. The procedure is identical to the one described for TruthX cross-model transfer, with a single difference in the **layer selection** step.

Layer Selection.

While TruthX selects the target layer based on probing accuracy of the trained autoencoders across all virtual layers, SLiMTruth uses the target layer from the SLiM training configuration directly. Specifically:

1. The **target module** (physical layer and type: attention/MLP/hidden) is read from the SLiM training config of the target model. This is the layer at which SLiM was trained and will be applied during inference.
2. For each layer of the **trainer model** of the same module type, Linear CKA is computed against the target layer’s activations on a shared set of instances, exactly as in TruthX.
3. The trainer layer that **maximises CKA** is selected as the alignment anchor.

Once the layer pair is determined, the Procrustes aligner fitting and the cross-model inference pipeline (pre-alignment $\rightarrow$ SLiM modulation $\rightarrow$ post-alignment) follow the exact same procedure described for TruthX cross-model transfer.

Cross-Model Inference.

During inference on the target model, for each selected module:

1. **Pre-alignment:** the target’s activation $\mathbf{h}_{\text{target}}$ is mapped into the trainer’s space via $\tilde{\mathbf{h}} = \mathbf{h}_{\text{target}} \mathbf{A}_{\text{t} \rightarrow \text{tr}} + \mathbf{b}_{\text{t} \rightarrow \text{tr}}$;
2. **SLiM modulation:** the FiLM transformation is applied in the trainer’s space:

$$\tilde{\mathbf{h}}' = \tilde{\mathbf{h}} + \alpha \cdot [\tilde{\mathbf{h}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}] \quad (3.45)$$

3. **Post-alignment:** the modulated activation is mapped back into the target’s space: $\mathbf{h}'_{\text{target}} = \tilde{\mathbf{h}}' \mathbf{A}_{\text{tr} \rightarrow \text{t}} + \mathbf{b}_{\text{tr} \rightarrow \text{t}}$.

Chapter 4

Results and Discussion

This chapter presents and discusses the results obtained from the experiments conducted to evaluate the effectiveness of the universal prober in detecting hallucinations in LLMs, together with the effectiveness of the proposed mitigation techniques.

4.1 Hallucination Detection

4.1.1 In-Domain Transfer

The first set of experiments evaluates how well a prober trained on one model transfers to another within the same dataset. Tab.s 4.1, 4.2, and 4.3 report the results for the pair (Llama-3.1-8B-Instruct, Gemma-2-9B-IT) on BeliefBankFacts, BeliefBankConstraints, and HaluEval, respectively, across all transfer approaches and representation layers.

Table 4.1: Results for pair (LLama-3.1-8B-Instruct, Gemma-2-9B-IT) on BeliefBankFacts

Trainer	Tester	Approach	Type	AccTr	AccTe	AurocTr	AurocTe
gemma-2-9b-it	Llama-3.1-8B-Instruct	RidgeRegressor	attn	0.9758	0.8724	0.9918	0.9274
gemma-2-9b-it	Llama-3.1-8B-Instruct	RidgeRegressor	mlp	0.9777	0.8682	0.9940	0.9359
gemma-2-9b-it	Llama-3.1-8B-Instruct	RidgeRegressor	hidden	0.9777	0.8690	0.9919	0.9209
Llama-3.1-8B-Instruct	gemma-2-9b-it	RidgeRegressor	attn	0.9589	0.8067	0.9799	0.8787
Llama-3.1-8B-Instruct	gemma-2-9b-it	RidgeRegressor	mlp	0.9478	0.8290	0.9728	0.8976
Llama-3.1-8B-Instruct	gemma-2-9b-it	RidgeRegressor	hidden	0.9469	0.8086	0.9729	0.8871
gemma-2-9b-it	Llama-3.1-8B-Instruct	Procrustes	attn	0.9758	0.7791	0.9918	0.8332

Continued on next page

Trainer	Tester	Approach	Type	AccTr	AccTe	AurocTr	AurocTe
gemma-2-9b-it	Llama-3.1-8B-Instruct	Procrustes	mlp	0.9777	0.7868	0.9940	0.8359
gemma-2-9b-it	Llama-3.1-8B-Instruct	Procrustes	hidden	0.9777	0.7885	0.9919	0.8292
Llama-3.1-8B-Instruct	gemma-2-9b-it	Procrustes	attn	0.9589	0.8680	0.9799	0.9237
Llama-3.1-8B-Instruct	gemma-2-9b-it	Procrustes	mlp	0.9478	0.8364	0.9728	0.9140
Llama-3.1-8B-Instruct	gemma-2-9b-it	Procrustes	hidden	0.9469	0.8346	0.9729	0.9126
gemma-2-9b-it	Llama-3.1-8B-Instruct	CKA	attn	0.9758	0.7800	0.9918	0.8332
gemma-2-9b-it	Llama-3.1-8B-Instruct	CKA	mlp	0.9777	0.7868	0.9940	0.8359
gemma-2-9b-it	Llama-3.1-8B-Instruct	CKA	hidden	0.9777	0.7885	0.9919	0.8292
Llama-3.1-8B-Instruct	gemma-2-9b-it	CKA	attn	0.9589	0.8457	0.9799	0.9237
Llama-3.1-8B-Instruct	gemma-2-9b-it	CKA	mlp	0.9478	0.8234	0.9728	0.9140
Llama-3.1-8B-Instruct	gemma-2-9b-it	CKA	hidden	0.9469	0.8178	0.9729	0.9126
gemma-2-9b-it	Llama-3.1-8B-Instruct	CCA	attn	0.9758	0.8373	0.9918	0.8811
gemma-2-9b-it	Llama-3.1-8B-Instruct	CCA	mlp	0.9777	0.8510	0.9940	0.8954
gemma-2-9b-it	Llama-3.1-8B-Instruct	CCA	hidden	0.9777	0.6712	0.9919	0.6776
Llama-3.1-8B-Instruct	gemma-2-9b-it	CCA	attn	0.9589	0.6784	0.9799	0.7249
Llama-3.1-8B-Instruct	gemma-2-9b-it	CCA	mlp	0.9478	0.6468	0.9728	0.6833
Llama-3.1-8B-Instruct	gemma-2-9b-it	CCA	hidden	0.9469	0.6822	0.9729	0.7145
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	attn	0.9758	0.8673	0.9918	0.9051
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	mlp	0.9777	0.8759	0.9940	0.9286
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	hidden	0.9777	0.8493	0.9919	0.8866
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	attn	0.9589	0.8290	0.9799	0.9199
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	mlp	0.9478	0.8216	0.9728	0.9029
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	hidden	0.9469	0.8086	0.9729	0.8767
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	attn	0.9777	0.8750	0.9937	0.9507
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	mlp	0.9814	0.8510	0.9950	0.9320
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	hidden	0.9777	0.8408	0.9944	0.9078
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	attn	0.9512	0.7639	0.9868	0.9037

Continued on next page

Trainer	Tester	Approach	Type	AccTr	AccTe	AurocTr	AurocTe
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	mlp	0.9349	0.7844	0.9784	0.9049
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	hidden	0.9341	0.7491	0.9774	0.8943
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	attn	0.9740	0.8288	0.9930	0.9206
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	mlp	0.9665	0.7680	0.9915	0.8558
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	hidden	0.9777	0.8022	0.9922	0.8808
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	attn	0.9247	0.6506	0.9772	0.7098
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	mlp	0.8844	0.7268	0.9494	0.8868
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	hidden	0.8716	0.6803	0.9509	0.8234
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	attn	0.9758	0.9538	0.9937	0.9856
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	mlp	0.9758	0.9264	0.9903	0.9675
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	hidden	0.9777	0.9366	0.9922	0.9750
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	attn	0.9461	0.9554	0.9819	0.9811
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	mlp	0.9307	0.9554	0.9619	0.9894
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	hidden	0.9289	0.9628	0.9693	0.9905

Table 4.2: Results for pair (LLama-3.1-8B-Instruct, Gemma-2-9B-IT) on Belief-BankConstraints

Trainer	Tester	Approach	Type	AccTr	AccTe	AurocTr	AurocTe
gemma-2-9b-it	Llama-3.1-8B-Instruct	RidgeRegressor	attn	0.9792	0.9560	0.9970	0.9848
gemma-2-9b-it	Llama-3.1-8B-Instruct	RidgeRegressor	mlp	0.9827	0.9505	0.9981	0.9841
gemma-2-9b-it	Llama-3.1-8B-Instruct	RidgeRegressor	hidden	0.9834	0.9499	0.9981	0.9829
Llama-3.1-8B-Instruct	gemma-2-9b-it	RidgeRegressor	attn	0.9703	0.8176	0.9883	0.9298
Llama-3.1-8B-Instruct	gemma-2-9b-it	RidgeRegressor	mlp	0.9697	0.8423	0.9866	0.9526
Llama-3.1-8B-Instruct	gemma-2-9b-it	RidgeRegressor	hidden	0.9670	0.8429	0.9820	0.9585
gemma-2-9b-it	Llama-3.1-8B-Instruct	Procrustes	attn	0.9792	0.8701	0.9970	0.9179
gemma-2-9b-it	Llama-3.1-8B-Instruct	Procrustes	mlp	0.9827	0.8619	0.9981	0.9225
gemma-2-9b-it	Llama-3.1-8B-Instruct	Procrustes	hidden	0.9834	0.8668	0.9981	0.9130
Llama-3.1-8B-Instruct	gemma-2-9b-it	Procrustes	attn	0.9703	0.7072	0.9883	0.7712

Continued on next page

Trainer	Tester	Approach	Type	AccTr	AccTe	AurocTr	AurocTe
Llama-3.1-8B-Instruct	gemma-2-9b-it	Procrustes	mlp	0.9697	0.6942	0.9866	0.7625
Llama-3.1-8B-Instruct	gemma-2-9b-it	Procrustes	hidden	0.9670	0.6977	0.9820	0.7730
gemma-2-9b-it	Llama-3.1-8B-Instruct	CKA	attn	0.9792	0.8701	0.9970	0.9179
gemma-2-9b-it	Llama-3.1-8B-Instruct	CKA	mlp	0.9827	0.8624	0.9981	0.9225
gemma-2-9b-it	Llama-3.1-8B-Instruct	CKA	hidden	0.9834	0.8668	0.9981	0.9130
Llama-3.1-8B-Instruct	gemma-2-9b-it	CKA	attn	0.9703	0.6536	0.9883	0.7712
Llama-3.1-8B-Instruct	gemma-2-9b-it	CKA	mlp	0.9697	0.6649	0.9866	0.7625
Llama-3.1-8B-Instruct	gemma-2-9b-it	CKA	hidden	0.9670	0.6730	0.9820	0.7730
gemma-2-9b-it	Llama-3.1-8B-Instruct	CCA	attn	0.9792	0.8250	0.9970	0.8807
gemma-2-9b-it	Llama-3.1-8B-Instruct	CCA	mlp	0.9827	0.8553	0.9981	0.9031
gemma-2-9b-it	Llama-3.1-8B-Instruct	CCA	hidden	0.9834	0.8481	0.9981	0.8815
Llama-3.1-8B-Instruct	gemma-2-9b-it	CCA	attn	0.9703	0.5823	0.9883	0.5693
Llama-3.1-8B-Instruct	gemma-2-9b-it	CCA	mlp	0.9697	0.6248	0.9866	0.6511
Llama-3.1-8B-Instruct	gemma-2-9b-it	CCA	hidden	0.9670	0.6016	0.9820	0.6176
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	attn	0.9792	0.8756	0.9970	0.9319
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	mlp	0.9827	0.9505	0.9981	0.9758
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	hidden	0.9834	0.9373	0.9981	0.9645
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	attn	0.9703	0.7963	0.9883	0.8816
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	mlp	0.9697	0.8606	0.9866	0.9274
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	hidden	0.9670	0.8322	0.9820	0.9252
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	attn	0.9941	0.9274	0.9994	0.9617
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	mlp	0.9951	0.9499	0.9997	0.9769
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	hidden	0.9932	0.9433	0.9996	0.9756
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	attn	0.9774	0.7578	0.9952	0.8691
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	mlp	0.9692	0.8050	0.9912	0.8976
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	hidden	0.9653	0.8252	0.9896	0.9476
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	attn	0.9783	0.7523	0.9979	0.9367

Continued on next page

Trainer	Tester	Approach	Type	AccTr	AccTe	AurocTr	AurocTe
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	mlp	0.9831	0.5894	0.9986	0.9401
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	hidden	0.9841	0.6643	0.9990	0.8852
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	attn	0.9565	0.7074	0.9873	0.7847
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	mlp	0.9532	0.7704	0.9698	0.8519
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	hidden	0.9538	0.8202	0.9716	0.9176
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	attn	0.9941	0.9796	0.9998	0.9950
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	mlp	0.9936	0.9653	0.9997	0.9886
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	hidden	0.9942	0.9659	0.9996	0.9926
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	attn	0.9747	0.9939	0.9948	0.9998
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	mlp	0.9615	0.9947	0.9917	0.9997
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	hidden	0.9549	0.9935	0.9826	0.9997

Table 4.3: Results for pair (LLama-3.1-8B-Instruct, Gemma-2-9B-IT) on HaluEval

Trainer	Tester	Approach	Type	AccTr	AccTe	AurocTr	AurocTe
gemma-2-9b-it	Llama-3.1-8B-Instruct	RidgeRegressor	attn	0.7184	0.6317	0.7702	0.6837
gemma-2-9b-it	Llama-3.1-8B-Instruct	RidgeRegressor	mlp	0.6922	0.6118	0.7487	0.6541
gemma-2-9b-it	Llama-3.1-8B-Instruct	RidgeRegressor	hidden	0.7103	0.6430	0.7714	0.6900
Llama-3.1-8B-Instruct	gemma-2-9b-it	RidgeRegressor	attn	0.5789	0.6418	0.5964	0.6863
Llama-3.1-8B-Instruct	gemma-2-9b-it	RidgeRegressor	mlp	0.5711	0.6458	0.5961	0.6987
Llama-3.1-8B-Instruct	gemma-2-9b-it	RidgeRegressor	hidden	0.5555	0.6673	0.5826	0.7138
gemma-2-9b-it	Llama-3.1-8B-Instruct	Procrustes	attn	0.7184	0.6179	0.7702	0.6596
gemma-2-9b-it	Llama-3.1-8B-Instruct	Procrustes	mlp	0.6922	0.6014	0.7487	0.6516
gemma-2-9b-it	Llama-3.1-8B-Instruct	Procrustes	hidden	0.7103	0.5997	0.7714	0.6525
Llama-3.1-8B-Instruct	gemma-2-9b-it	Procrustes	attn	0.5789	0.6808	0.5964	0.7400
Llama-3.1-8B-Instruct	gemma-2-9b-it	Procrustes	mlp	0.5711	0.6559	0.5961	0.7181
Llama-3.1-8B-Instruct	gemma-2-9b-it	Procrustes	hidden	0.5555	0.6754	0.5826	0.7342
gemma-2-9b-it	Llama-3.1-8B-Instruct	CKA	attn	0.7184	0.5416	0.7702	0.6596

Continued on next page

Trainer	Tester	Approach	Type	AccTr	AccTe	AurocTr	AurocTe
gemma-2-9b-it	Llama-3.1-8B-Instruct	CKA	mlp	0.6922	0.5546	0.7487	0.6516
gemma-2-9b-it	Llama-3.1-8B-Instruct	CKA	hidden	0.7103	0.5425	0.7714	0.6525
Llama-3.1-8B-Instruct	gemma-2-9b-it	CKA	attn	0.5789	0.6458	0.5964	0.7400
Llama-3.1-8B-Instruct	gemma-2-9b-it	CKA	mlp	0.5711	0.6512	0.5961	0.7181
Llama-3.1-8B-Instruct	gemma-2-9b-it	CKA	hidden	0.5555	0.6559	0.5826	0.7342
gemma-2-9b-it	Llama-3.1-8B-Instruct	CCA	attn	0.7184	0.5667	0.7702	0.5577
gemma-2-9b-it	Llama-3.1-8B-Instruct	CCA	mlp	0.6922	0.5425	0.7487	0.5193
gemma-2-9b-it	Llama-3.1-8B-Instruct	CCA	hidden	0.7103	0.5399	0.7714	0.5354
Llama-3.1-8B-Instruct	gemma-2-9b-it	CCA	attn	0.5789	0.5208	0.5964	0.5286
Llama-3.1-8B-Instruct	gemma-2-9b-it	CCA	mlp	0.5711	0.4987	0.5961	0.5053
Llama-3.1-8B-Instruct	gemma-2-9b-it	CCA	hidden	0.5555	0.5403	0.5826	0.5405
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	attn	0.7184	0.6092	0.7702	0.6556
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	mlp	0.6922	0.5971	0.7487	0.6303
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	hidden	0.7103	0.6161	0.7714	0.6475
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	attn	0.5789	0.6163	0.5964	0.6752
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	mlp	0.5711	0.5880	0.5961	0.6319
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	hidden	0.5555	0.6116	0.5826	0.6367
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	attn	0.7312	0.6205	0.7924	0.6670
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	mlp	0.7063	0.5962	0.7707	0.6313
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	hidden	0.7171	0.5971	0.7825	0.6354
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	attn	0.5858	0.6028	0.6257	0.6444
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	mlp	0.5745	0.6149	0.6083	0.6601
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	hidden	0.5858	0.5699	0.6117	0.6081
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	attn	0.6895	0.5243	0.7651	0.5092
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	mlp	0.6741	0.5217	0.7396	0.5081
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	hidden	0.6767	0.5373	0.7483	0.5540
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	attn	0.5719	0.5215	0.6074	0.5476

Continued on next page

Trainer	Tester	Approach	Type	AccTr	AccTe	AurocTr	AurocTe
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	mlp	0.5555	0.5215	0.5879	0.5399
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	hidden	0.5425	0.4906	0.5745	0.4993
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	attn	0.6942	0.5667	0.7759	0.5897
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	mlp	0.7016	0.5459	0.7594	0.5677
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	hidden	0.7030	0.5659	0.7703	0.5848
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	attn	0.6014	0.7285	0.6374	0.8066
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	mlp	0.5789	0.7184	0.6106	0.7824
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	hidden	0.5979	0.7413	0.6255	0.8172

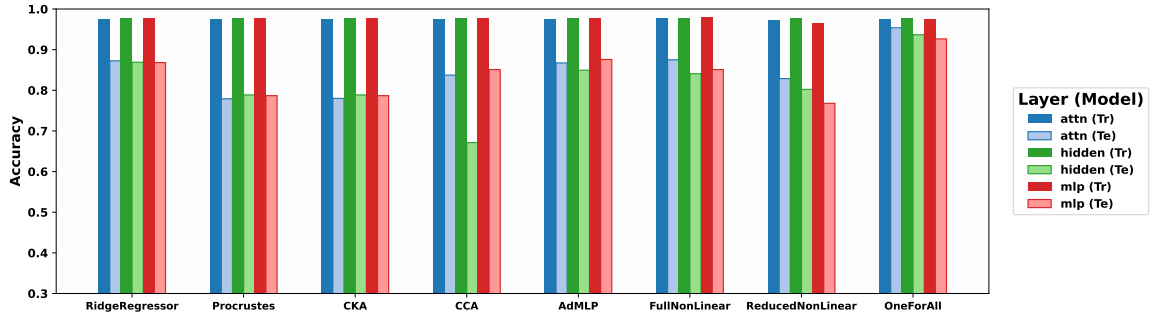


Figure 4.1: Gemma-2-9B-IT as Trainer, Llama-3.1-8B-Instruct as Tester on BeliefBankFacts

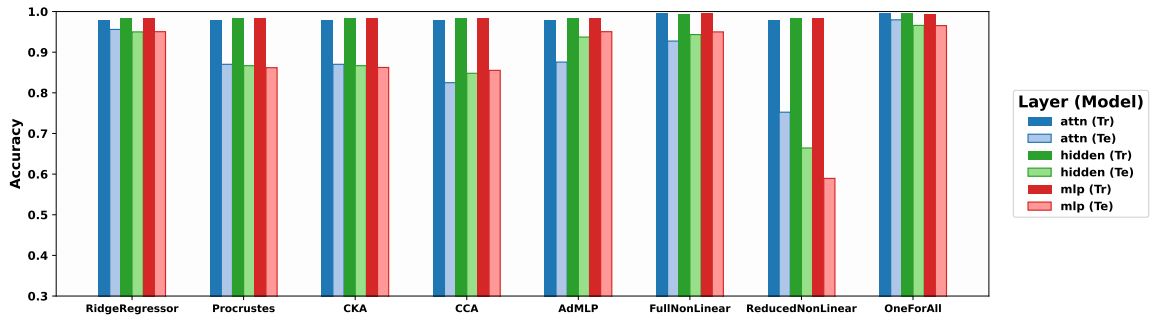


Figure 4.2: Llama-3.1-8B-Instruct as Trainer, Gemma-2-9B-IT as Tester on BeliefBankConstraints

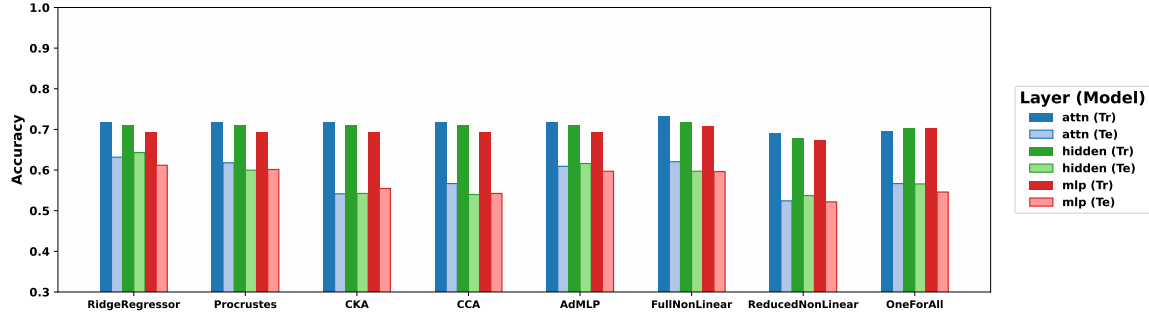


Figure 4.3: Gemma-2-9B-IT as Trainer, Llama-3.1-8B-Instruct as Tester on HaluEval

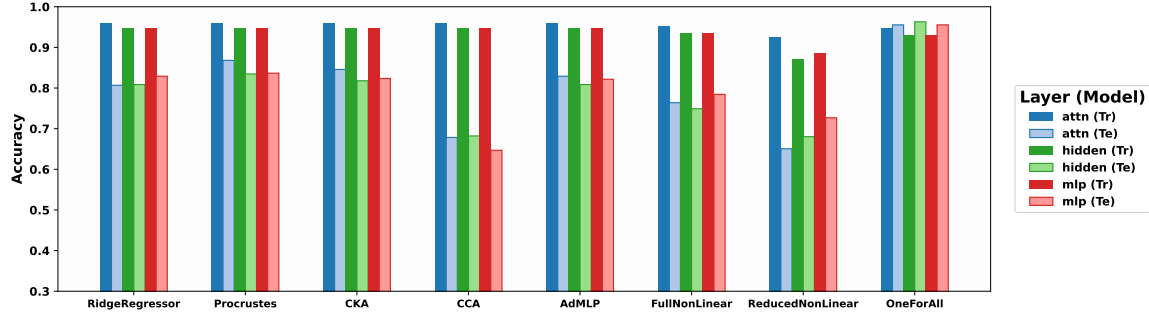


Figure 4.4: Llama-3.1-8B-Instruct as Trainer, Gemma-2-9B-IT as Tester on Belief-BankFacts

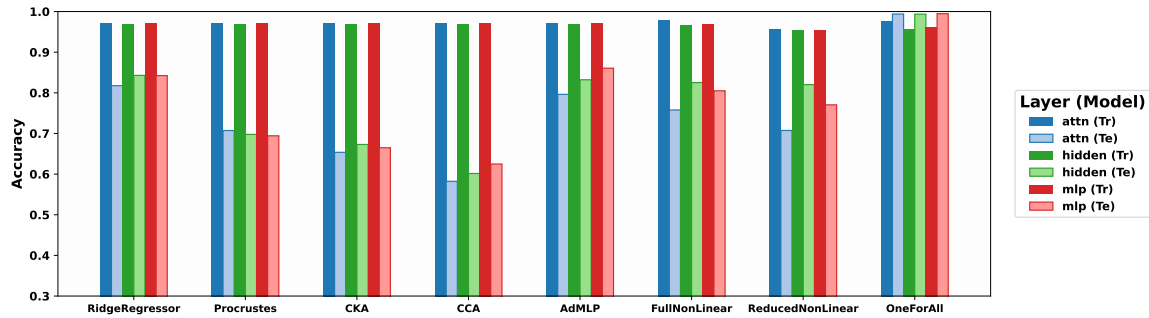


Figure 4.5: Llama-3.1-8B-Instruct as Trainer, Gemma-2-9B-IT as Tester on Belief-BankConstraints

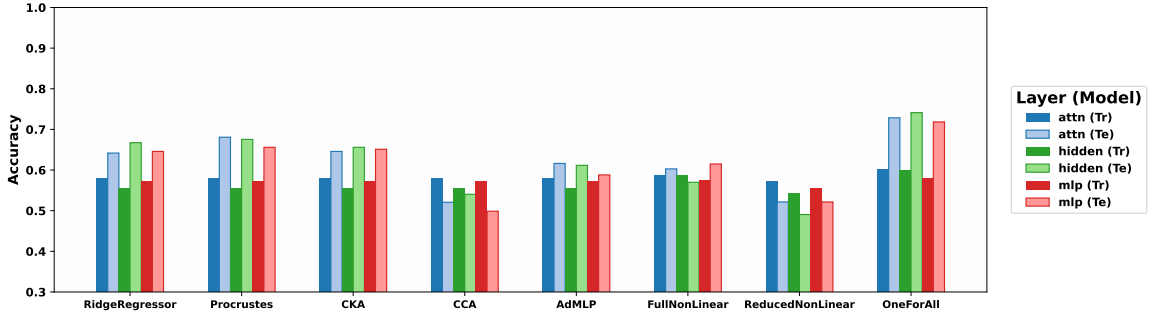


Figure 4.6: Llama-3.1-8B-Instruct as Trainer, Gemma-2-9B-IT as Tester on HaluEval

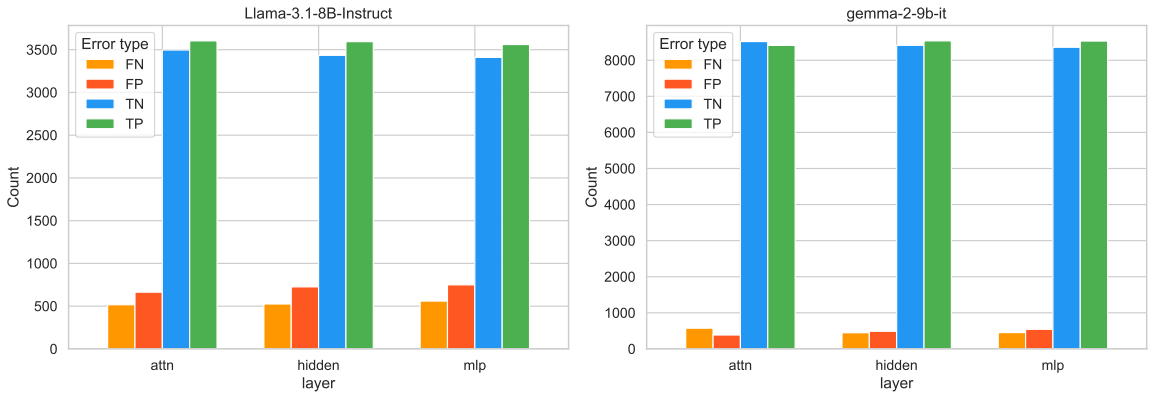


Figure 4.7: Error Type Breakdown for the One-For-All

The in-domain results reveal a clear hierarchy among the transfer approaches. On both BeliefBankFacts and BeliefBankConstraints, the *One-For-All* method is the strongest overall. It reaches very high tester performance in both transfer directions, up to 0.9628 AccTe and 0.9905 AUROC on BeliefBankFacts and up to 0.9947 AccTe and 0.9998 AUROC on BeliefBankConstraints, clearly outperforming the classical alignment methods. This dominance does not extend uniformly to HaluEval. In the Gemma→Llama direction the best detector is the simple *RidgeRegressor* (up to 0.6430 AccTe), with *One-For-All* notably weaker (between 0.546 and 0.567 AccTe); in the opposite direction, Llama→Gemma, *One-For-All* recovers its lead (between 0.718 and 0.741 AccTe, up to 0.8172 AUROC). At the bottom of the ranking, *CCA* and *ReducedNonLinear* are the least robust, frequently collapsing to near-random performance on HaluEval and under the harder transfer settings.

These numbers point to a marked separation in difficulty between the three datasets. BeliefBankConstraints is the easiest setting for transferred detection, near-perfect with *One-For-All* in both directions, although the classical approaches are strong only in selected directions (for example Gemma→Llama with *RidgeRegressor*) and degrade substantially when the direction is reversed. BeliefBankFacts is still favourable to transfer, but it shows larger method-dependent variability. HaluEval is the hardest

dataset by a wide margin: most methods stay between 0.54 and 0.68 AccTe, with only direction-specific exceptions. This is consistent with the PCA projections of the activations (Figures 3.6 and 3.7), where the HaluEval points form much noisier and less separable clusters than the BeliefBank ones. The likely cause is the length of the HaluEval prompts, which embed a full dialogue history and an external knowledge passage instead of a single short statement, so the truthfulness signal is diluted over a longer and more heterogeneous context.

A central finding is the **directional asymmetry** of transfer. The HaluEval behaviour of One-For-All, weak from Gemma to Llama but strong from Llama to Gemma, is the clearest example, and analogous asymmetries appear on BeliefBankConstraints for the linear methods. Transfer quality therefore depends not only on dataset complexity but also on which model supplies the representation and which one receives it. No single component (attention, MLP, or hidden) is consistently best either. In-domain One-For-All transfer is relatively stable across components on BeliefBank, but the component effects become pronounced under harder or shifted conditions, where the same train/test pair can give very different outcomes depending on the chosen component. The choice of component is thus a first-order design decision for universal probing rather than a secondary implementation detail.

4.1.2 Cross-Domain Transfer

To further evaluate the generalizability of transferred probers, cross-domain experiments were conducted. In this context, the *One-For-All* approach was evaluated by training the frozen_head and the encoder on a source dataset (“Train Set”), but classifying them using the trainer’s Head specific to a different target dataset (“Test Set”). This scenario evaluates the **universality of the latent space**: it tests whether the representation of truth learned in one domain is compatible with the decision boundary learned in another domain. The results are reported in Tab. 4.4. In this table and in the mitigation tables that follow, the datasets are abbreviated as BBF (BeliefBankFacts), BBC (BeliefBankConstraints), and HE (HaluEval).

Train	Test	trainer	tester	Layer	Acc (Tr)	AUROC (Tr)	Acc (Te)	AUROC (Te)
BBC	BBF	Gemma	Llama	attn	0.696	0.826	0.704	0.852
BBC	BBF	Gemma	Llama	mlp	0.524	0.556	0.696	0.828
BBC	BBF	Gemma	Llama	hidden	0.641	0.693	0.667	0.837
BBC	BBF	Llama	Gemma	attn	0.747	0.875	0.699	0.663
BBC	BBF	Llama	Gemma	mlp	0.686	0.805	0.534	0.534
BBC	BBF	Llama	Gemma	hidden	0.823	0.875	0.549	0.595
BBC	HE	Gemma	Llama	attn	0.510	0.555	0.529	0.534
BBC	HE	Gemma	Llama	mlp	0.562	0.587	0.501	0.500
BBC	HE	Gemma	Llama	hidden	0.562	0.577	0.519	0.532
BBC	HE	Llama	Gemma	attn	0.505	0.523	0.503	0.557
BBC	HE	Llama	Gemma	mlp	0.501	0.517	0.520	0.574
BBC	HE	Llama	Gemma	hidden	0.512	0.541	0.547	0.584
BBF	BBC	Gemma	Llama	attn	0.517	0.491	0.770	0.961
BBF	BBC	Gemma	Llama	mlp	0.527	0.607	0.761	0.935
BBF	BBC	Gemma	Llama	hidden	0.598	0.590	0.577	0.943
BBF	BBC	Llama	Gemma	attn	0.793	0.946	0.517	0.658
BBF	BBC	Llama	Gemma	mlp	0.605	0.928	0.521	0.706
BBF	BBC	Llama	Gemma	hidden	0.685	0.953	0.522	0.667
BBF	HE	Gemma	Llama	attn	0.500	0.553	0.517	0.554
BBF	HE	Gemma	Llama	mlp	0.511	0.589	0.502	0.506
BBF	HE	Gemma	Llama	hidden	0.521	0.619	0.479	0.469
BBF	HE	Llama	Gemma	attn	0.519	0.541	0.503	0.526
BBF	HE	Llama	Gemma	mlp	0.512	0.513	0.499	0.558
BBF	HE	Llama	Gemma	hidden	0.499	0.488	0.505	0.592
HE	BBC	Gemma	Llama	attn	0.491	0.465	0.507	0.778
HE	BBC	Gemma	Llama	mlp	0.498	0.565	0.402	0.211
HE	BBC	Gemma	Llama	hidden	0.493	0.415	0.812	0.849
HE	BBC	Llama	Gemma	attn	0.512	0.748	0.509	0.523
HE	BBC	Llama	Gemma	mlp	0.398	0.385	0.481	0.491
HE	BBC	Llama	Gemma	hidden	0.548	0.824	0.480	0.427
HE	BBF	Gemma	Llama	attn	0.799	0.888	0.528	0.563
HE	BBF	Gemma	Llama	mlp	0.714	0.820	0.477	0.442
HE	BBF	Gemma	Llama	hidden	0.690	0.863	0.627	0.661
HE	BBF	Llama	Gemma	attn	0.567	0.595	0.516	0.882
HE	BBF	Llama	Gemma	mlp	0.625	0.546	0.725	0.846
HE	BBF	Llama	Gemma	hidden	0.611	0.677	0.522	0.900

Table 4.4: Cross-Domain Results (One-For-All)

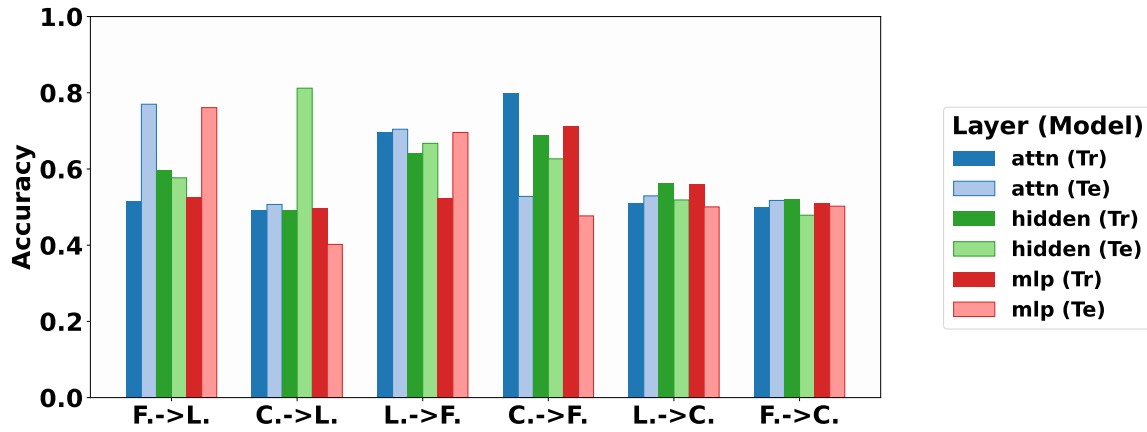


Figure 4.8: Cross-Domain Transfer: Gemma-2-9B-IT as Trainer, Llama-3.1-8B-Instruct as Tester

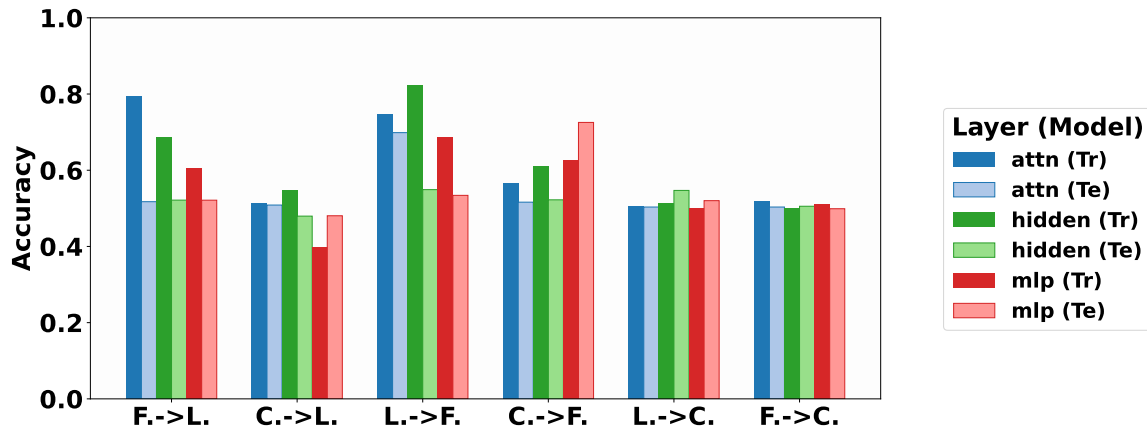


Figure 4.9: Cross-Domain Transfer: Llama-3.1-8B-Instruct as Trainer, Gemma-2-9B-IT as Tester

The cross-domain results temper the optimism of the in-domain analysis. Transfer toward HaluEval is essentially at chance level (mean AccTe around 0.51, mean AUROC around 0.54), irrespective of whether training begins from BeliefBankFacts or BeliefBankConstraints. The strongest cross-domain behaviour is instead concentrated in the BeliefBank-to-BeliefBank transfers, most notably from BeliefBankFacts to BeliefBankConstraints with Gemma as trainer (up to 0.770 AccTe and 0.961 AUROC). Two further patterns stand out. The first is a strong trainer/tester asymmetry, with Gemma→Llama outperforming Llama→Gemma on average, echoing the directional effect already seen in-domain. The second is a recurring calibration issue: several cases combine moderate-to-high AUROC with low AccTe, indicating that the transferred representation preserves the ranking quality of the truth signal but lacks a well-placed decision threshold once it has crossed into a new domain.

4.2 Hallucination Mitigation

This section presents and discusses the two mitigation approaches, TruthX and SLiMTruth, across the standard, cross-dataset, and cross-model settings. In the tables, *Gemma* and *Llama* denote Gemma-2-9B-IT and Llama-3.1-8B-Instruct.

4.2.1 TruthX

The TruthX mitigation approach was evaluated across three experimental settings: *Standard* (same model and dataset for training and inference), *Cross-Dataset* (same model, different dataset), and *Cross-Model* (different model, with an alignment strategy). The shared hyperparameters are reported in the methodology (Section 3.6.1).

Standard Setting

Tab. 4.5 compares the hallucination rate (HR) of each model at baseline (no intervention) against the standard TruthX setting, where the autoencoder and steering vectors are trained and applied on the same model and dataset.

Model	Dataset	Pairs	HR Base	HR TruthX	Δ
Gemma	BBC	6500	0.4466	0.4170	-6.6%
Gemma	BBF	6500	0.0286	0.0332	+16.3%
Gemma	HE	2500	0.2417	0.1581	-34.6%
Llama	BBC	6500	0.5283	0.4926	-6.8%
Llama	BBF	6500	0.0672	0.2812	+318.3%
Llama	HE	2500	0.2763	0.1375	-50.3%

Table 4.5: Baseline vs. standard TruthX hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).

In the standard setting, TruthX behaves unevenly. It is clearly effective on HaluEval (-34.6% on Gemma, -50.3% on Llama) and mildly helpful on BeliefBankConstraints (-6.6% and -6.8%). The critical failure is BeliefBankFacts, where Llama degrades from 0.0672 to 0.2812 HR (+318.3%) and Gemma also worsens slightly (+16.3%). This is a clear sign of over-correction when the baseline hallucination rate is already very low: with little genuine error to remove, the editing intervention introduces new mistakes instead.

Cross-Dataset Transfer

In the cross-dataset setting, the autoencoder and steering vectors are trained on one dataset and applied to a different evaluation dataset, using the **same model**. Tab. 4.6 reports the results.

Model	Dataset Train	Dataset Eval	Pairs	HR	HR Base	Δ
Gemma	BBF	BBC	6500	0.4180	0.4466	-6.4%
Gemma	BBC	BBF	6500	0.0337	0.0286	+17.8%
Gemma	BBF	HE	2500	0.1603	0.2417	-33.7%
Gemma	BBC	HE	2500	0.1580	0.2417	-34.6%
Gemma	HE	BBF	6500	0.0339	0.0286	+18.5%
Gemma	HE	BBC	6500	0.4265	0.4466	-4.5%
Llama	BBF	BBC	6500	0.5117	0.5283	-3.1%
Llama	BBC	BBF	6500	0.1629	0.0672	+142.4%
Llama	BBF	HE	2500	0.2508	0.2763	-9.2%
Llama	BBC	HE	2500	0.1591	0.2763	-42.4%
Llama	HE	BBF	6500	0.1459	0.0672	+117.1%
Llama	HE	BBC	6500	0.5536	0.5283	+4.8%

Table 4.6: Cross-dataset TruthX hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).

Cross-dataset TruthX confirms an asymmetric pattern. Transfers from BeliefBankFacts or BeliefBankConstraints toward HaluEval are often beneficial (up to -42.4% on Llama), whereas transfers toward BeliefBankFacts are consistently harmful (up to +142.4% on Llama). Transfer toward BeliefBankConstraints is moderate and less stable, yielding small gains in some directions and slight degradation in others. The picture that emerges is therefore the same one observed in the standard setting, amplified by domain shift: the direction of the transfer matters far more than the mere fact that the model is held fixed.

Cross-Model Transfer

In the cross-model setting, the autoencoder and steering vectors are trained on a **source model** and applied to a different **target model**. The alignment strategies that were compared are described in the methodology (Section 3.6.1). Tab.s 4.7 and 4.8 report the hallucination rates obtained with the Procrustes alignment.

Train Model	Eval Model	Dataset	HR Baseline	HR Procrustes	Δ
Gemma	Llama	BBF	0.0672	0.1332	+98.2%
Gemma	Llama	BBC	0.5283	0.5126	-3.0%
Gemma	Llama	HE	0.2763	0.1712	-38.0%

Table 4.7: Cross-model TruthX, Gemma $\rightarrow$ Llama, hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).

Train Model	Eval Model	Dataset	HR Baseline	HR Procrustes	Δ
Llama	Gemma	BBF	0.0286	0.0261	-8.7%
Llama	Gemma	BBC	0.4466	0.4125	-7.6%
Llama	Gemma	HE	0.2417	0.1531	-36.7%

Table 4.8: Cross-model TruthX, Llama $\rightarrow$ Gemma, hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).

With Procrustes alignment, cross-model TruthX is most reliable on HaluEval (Gemma $\rightarrow$ Llama: -38.0%; Llama $\rightarrow$ Gemma: -36.7%). Its effect on BeliefBankConstraints is smaller but generally positive, while on BeliefBankFacts the results remain mixed and sensitive to over-correction. Overall, TruthX is the more transferable of the two mitigation methods across architectures, even if it is not robust in low-error regimes.

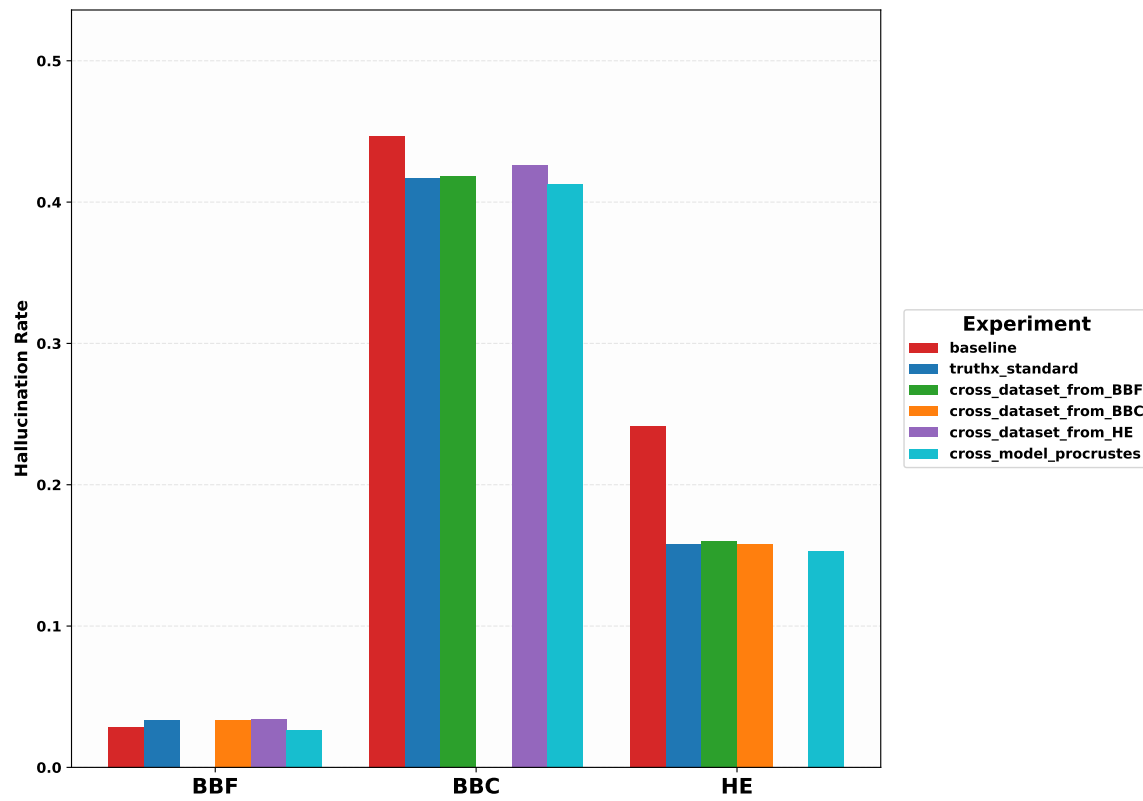


Figure 4.10: TruthX: Gemma

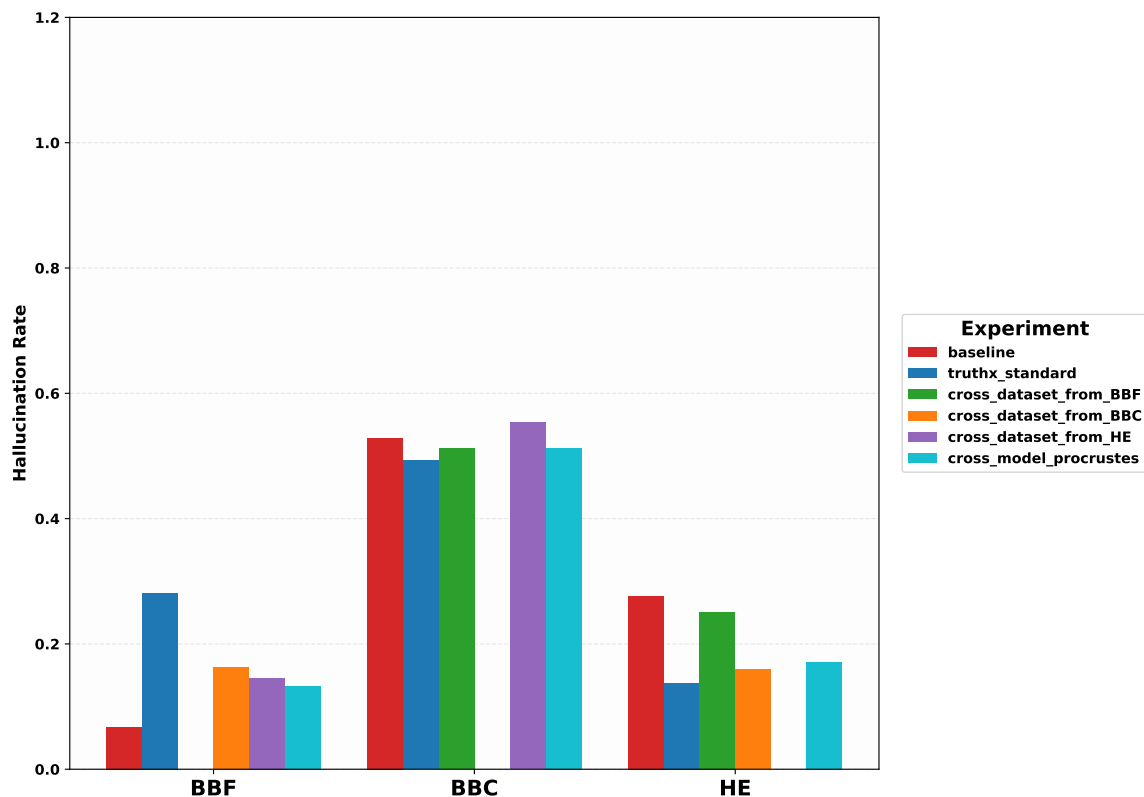


Figure 4.11: TruthX: Llama

4.2.2 SLiMTruth

The SLiMTruth mitigation approach was evaluated across the same experimental settings used for TruthX: *Standard* (same model and dataset for training and inference), *Cross-Dataset* (same model, different dataset), and *Cross-Model* (different model, with Procrustes alignment).

Standard Setting

Tab. 4.9 compares the hallucination rate (HR) of each model at baseline (no intervention) against the standard SLiMTruth setting, where the SLiM module is trained and applied on the same model and dataset.

Model	Dataset	Pairs	SLiM Layer	HR Base	HR SLiM	Δ
Gemma	BBC	6500	27	0.4466	0.1827	-59.1%
Gemma	BBF	6500	24	0.0286	0.0217	-24.1%
Gemma	HE	2000	21	0.2417	0.1751	-27.6%
Llama	BBC	6500	8	0.5283	0.2576	-51.2%
Llama	BBF	6500	13	0.0672	0.0068	-89.9%
Llama	HE	2000	15	0.2763	0.1164	-57.9%

Table 4.9: Baseline vs. standard SLiMTruth hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).

In-domain SLiMTruth is the strongest mitigation configuration of all. It reduces the hallucination rate in every model/dataset pair tested, often by a large margin: -89.9% for Llama on BeliefBankFacts, -59.1% for Gemma on BeliefBankConstraints, and -57.9% for Llama on HaluEval. It even improves on BeliefBankFacts, the dataset where TruthX fails most severely, because the layer-specific SLiM intervention is calibrated to the model’s own representations rather than imposed as a blanket edit.

Cross-Dataset Transfer

In the cross-dataset setting, the SLiM module is trained on one dataset and applied to a different evaluation dataset, using the **same model**. Tab. 4.10 reports the results.

Model	Dataset Train	Dataset Eval	Pairs	SLiM Layer	HR	HR Base	Δ
Gemma	BBF	BBC	6500	24	0.3830	0.4466	-14.2%
Gemma	BBC	BBF	6500	27	0.3231	0.0286	+1029.7%
Gemma	BBF	HE	6500	24	0.1612	0.2417	-33.3%
Gemma	BBC	HE	6500	27	0.1632	0.2417	-32.5%
Gemma	HE	BBF	2000	21	0.0376	0.0286	+31.5%
Gemma	HE	BBC	2000	21	0.4722	0.4466	+5.7%
Llama	BBF	BBC	6500	13	0.2851	0.5283	-46.0%
Llama	BBC	BBF	6500	8	0.6598	0.0672	+881.8%
Llama	BBF	HE	6500	13	0.1635	0.2763	-40.8%
Llama	BBC	HE	6500	8	0.1569	0.2763	-43.2%
Llama	HE	BBF	2000	15	0.2367	0.0672	+252.2%
Llama	HE	BBC	2000	15	0.4978	0.5283	-5.8%

Table 4.10: Cross-dataset SLiMTruth hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).

Under dataset shift, however, SLiMTruth becomes far more fragile. Some directions stay effective (notably BeliefBankFacts→HaluEval and BeliefBankConstraints→HaluEval), but transfer toward BeliefBankFacts can fail catastrophically, with increases of up to +1029.7% on Gemma and +881.8% on Llama for BeliefBankConstraints→BeliefBankFacts. SLiMTruth is therefore highly effective when matched to its training domain and highly sensitive when its correction patterns are applied out of domain.

Cross-Model Transfer

In the cross-model setting, the SLiM module is trained on a **source model** and applied to a different **target model** via *Procrustes alignment*. Tab.s 4.11 and 4.12 report the hallucination rates.

Train Model	Eval Model	Dataset	SLiM Layer	HR Baseline	HR Procrustes	Δ
Gemma	Llama	BBF	24	0.0672	0.1245	+85.3%
Gemma	Llama	BBC	27	0.5283	0.4979	-5.8%
Gemma	Llama	HE	21	0.2763	0.9631	+248.6%

Table 4.11: Cross-model SLiMTruth, Gemma → Llama, hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).

Train Model	Eval Model	Dataset	SLiM Layer	HR Baseline	HR Procrustes	Δ
Llama	Gemma	BBF	13	0.0286	0.0548	+91.6%
Llama	Gemma	BBC	8	0.4466	0.7061	+58.1%
Llama	Gemma	HE	15	0.2417	0.5631	+133.0%

Table 4.12: Cross-model SLiMTruth, Llama → Gemma, hallucination rate. The green row is the best result in the table (most favourable Δ , i.e. better), the red row the worst (least favourable Δ , i.e. worse).

Cross-model SLiMTruth is mostly ineffective or harmful. From Gemma to Llama it improves only on BeliefBankConstraints (-5.8%), and from Llama to Gemma it degrades on every dataset. Compared with TruthX, this suggests that the layer-specific SLiM interventions are too architecture-dependent to transfer reliably, even with Procrustes alignment: a modulation tuned to a particular layer of the source model has no guarantee of landing on a functionally equivalent layer of the target.

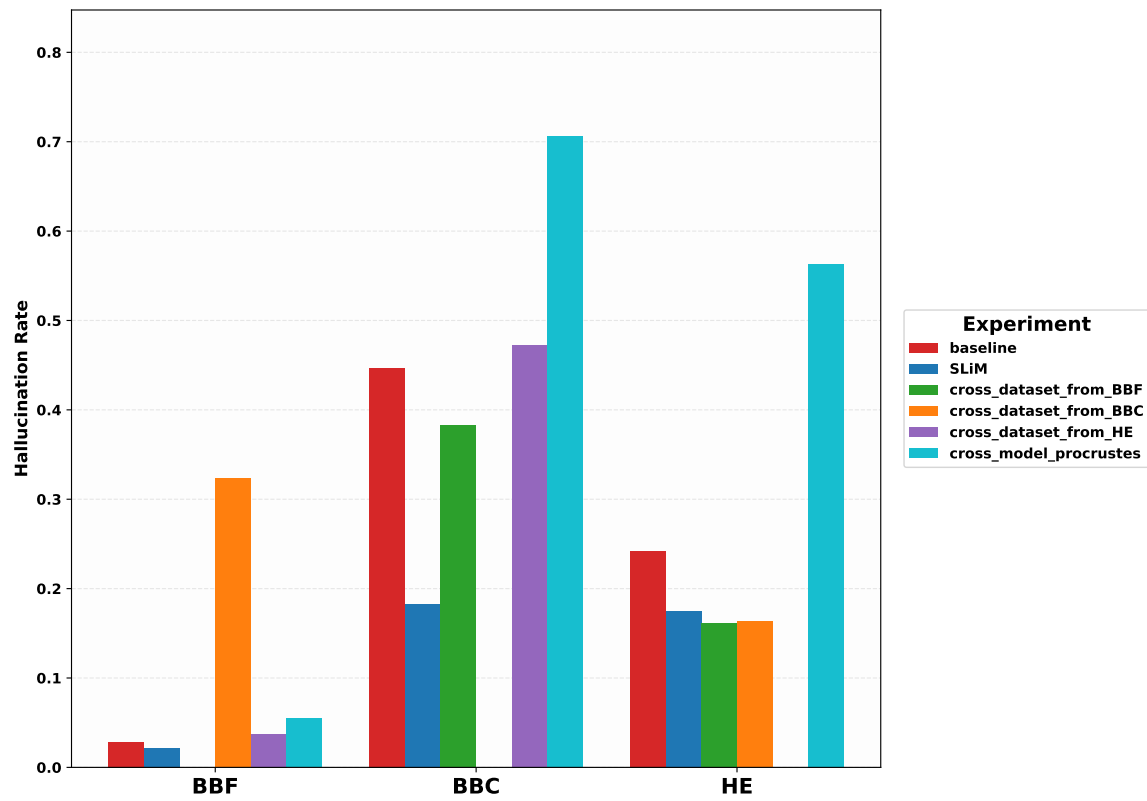


Figure 4.12: SLiMTruth: Gemma

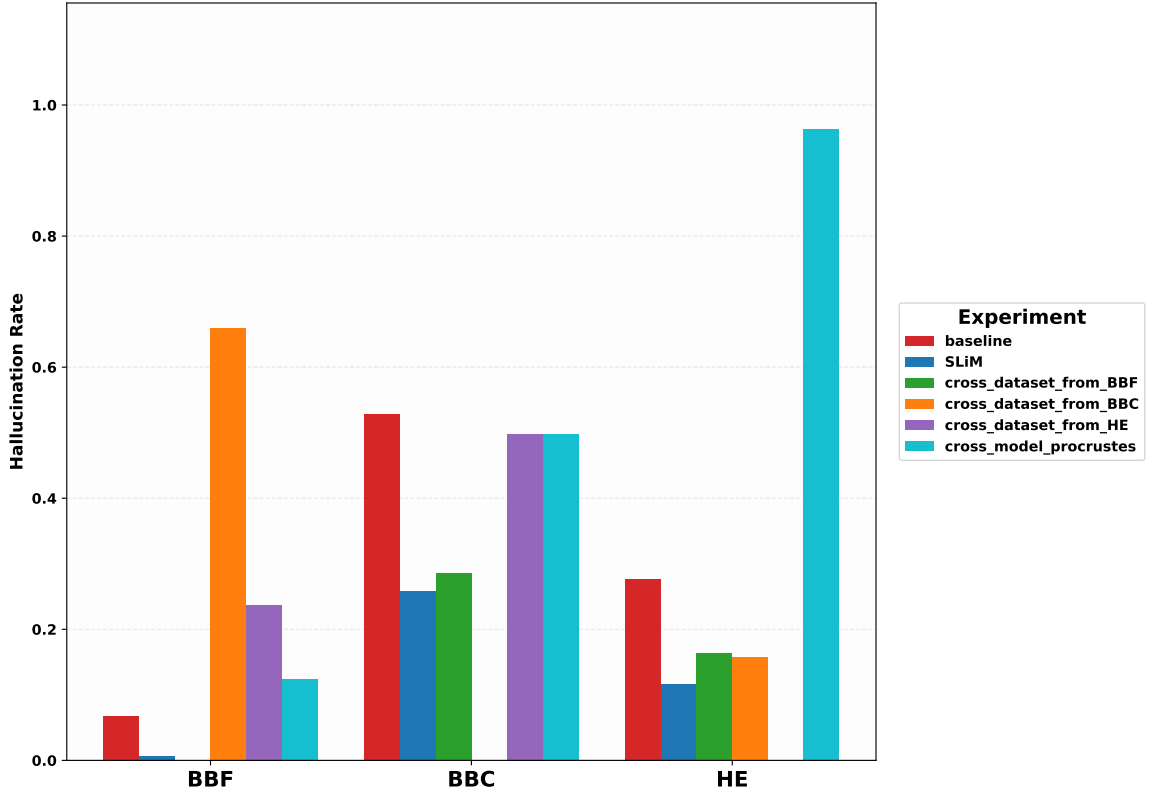


Figure 4.13: SLiMTruth: Llama

4.2.3 Comparative Analysis: TruthX vs. SLiMTruth

Taken together, the two methods occupy complementary niches. In the standard setting, SLiMTruth is superior and consistently effective, giving the largest and most stable reductions. Under dataset shift both methods are vulnerable toward BeliefBankFacts, but the SLiMTruth failures are far larger when the transfer is misaligned. Across models, TruthX is clearly the more viable option, especially on HaluEval, where SLiMTruth tends to collapse.

A recurring thread through all of these comparisons is the **BeliefBankFacts vulnerability**. Because BeliefBankFacts starts from a very low baseline hallucination rate, interventions trained on noisier domains tend to over-correct and introduce new errors. The resulting transferability hierarchy is intuitive: the standard regime (same model, same dataset) is the strongest, with SLiMTruth best overall; cross-dataset transfer (same model, different dataset) is viable only in some directions and carries a high risk toward BeliefBankFacts; and cross-model transfer (different model, same dataset) is currently reliable only for TruthX, and only in selected scenarios, mainly HaluEval.

Chapter 5

Conclusions

This chapter draws together the main outcomes of the experimental campaign and outlines the directions that they suggest for future work.

5.1 Summary of Results

This work investigated transferable mechanisms for both the **detection** and the **mitigation** of hallucinations across the Gemma and Llama models, using the BeliefBankFacts, BeliefBankConstraints, and HaluEval datasets.

On the detection side, the *One-For-All* approach proved to be the strongest in-domain transfer method on BeliefBank, achieving near-perfect tester performance on BeliefBankConstraints (up to 0.9947 AccTe and 0.9998 AUROC) and very high results on BeliefBankFacts (up to 0.9628 AccTe and 0.9905 AUROC). No single method, however, was universally best on HaluEval, where performance was substantially lower and strongly direction-dependent: simpler probes such as the RidgeRegressor remained competitive from Gemma to Llama, while One-For-All regained the advantage in the opposite direction. This directional asymmetry turned out to be a core factor throughout, with transfer quality depending heavily on the assignment of source and target models rather than on the choice of method alone. Cross-domain universality was limited: One-For-All transfer was generally weak toward HaluEval, close to chance on average, and strongest between BeliefBankFacts and BeliefBankConstraints. The choice of component also mattered consistently, with no global winner among the attn, mlp, and hidden representations and with component sensitivity growing under domain shift.

On the mitigation side, SLiMTruth gave the largest and most consistent reductions in the standard setting, where it was trained and evaluated on the same model and dataset, reaching reductions of up to -89.9% . TruthX was less dominant in-domain but markedly more transferable, retaining clear cross-model gains on HaluEval of around -37% in both directions with Procrustes alignment. Cross-dataset transfer was highly asymmetric for both methods: transfers toward BeliefBankFacts were

risky and occasionally catastrophic, especially for SLiMTruth. This BeliefBankFacts vulnerability recurred throughout the analysis and is a direct consequence of the dataset’s very low baseline hallucination rate, which leaves little genuine error to remove and makes aggressive steering prone to over-correction.

5.2 Future Work

These findings point to several priorities for future research. The pervasive directional asymmetry argues for **direction-aware transfer strategies** that model the source-to-target relationship explicitly, instead of assuming that transfer is equally effective in both directions. The frequent mismatch between an acceptable AUROC and a low AccTe suggests that **post-transfer calibration**, optimizing decision thresholds and score scaling after alignment, could recover much of the accuracy lost when a prober crosses into a new model or domain. The over-correction observed on BeliefBankFacts motivates **adaptive steering strength**, scaling the intensity of the intervention to the estimated baseline risk so as to avoid damaging settings that are already in a low-error regime. For SLiMTruth in particular, the architecture dependence of its layer-wise interventions calls for **architecture-aware transfer** that learns cross-model layer correspondences jointly with the modulation parameters, rather than relying on fixed layer mappings. Finally, **richer evaluation protocols** spanning additional domains and explicit robustness checks will be needed to establish whether the truthfulness features discovered here remain stable beyond the datasets considered in this work.

Bibliography

- [1] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.
- [2] Amos Azaria and Tom Mitchell. The internal state of an LLM knows when it’s lying. *arXiv preprint arXiv:2304.13734*, 2023. URL <https://arxiv.org/abs/2304.13734>.
- [3] Peter Balogh. The anxiety of influence: Bloom filters in transformer attention heads, 2026. URL <https://arxiv.org/abs/2602.17526>.
- [4] Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022.
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [6] Yuanpu Cao, Tianrong Zhang, Bochuan Cao, Ziyi Yin, Lu Lin, Fenglong Ma, and Jinghui Chen. Personalized steering of large language models: Versatile steering vectors through bi-directional preference optimization. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 49519–49551. Curran Associates, Inc., 2024. doi: 10.52202/079017-1567. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/58cbe393b4254da8966780a40d023c0b-Paper-Conference.pdf.
- [7] Zhongzhi Chen, Xingwu Sun, Xianfeng Jiao, Fengzong Lian, Zhanhui Kang, Di Wang, and Chengzhong Xu. Truth forest: Toward multi-scale truthfulness in large language models through intervention without tuning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(19):20967–20974, Mar. 2024. doi: 10.1609/aaai.v38i19.30087. URL <https://ojs.aaai.org/index.php/AAAI/article/view/30087>.

- [8] Bryan Cheng and Jasper Zhang. Single-position intervention fails: Distributed output templates drive in-context learning, 2026. URL <https://arxiv.org/abs/2605.04061>.
- [9] Ekaterina Fadeeva, Aleksandr Rubashevskii, Artem Shelmanov, Maxime Peyrard, Alexander Panchenko, and Iryna Gurevych. Fact-checking of AI-generated reports. *arXiv preprint arXiv:2307.14634*, 2023. URL <https://arxiv.org/abs/2307.14634>. Introduces Claim-Conditioned Probability (CCP).
- [10] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *CoRR*, abs/2101.03961, 2021. URL <https://arxiv.org/abs/2101.03961>.
- [11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [12] Hosein Hasani, Mohammadali Banayeeanzade, Ali Nafisi, Sadegh Mohammadian, Fatemeh Askari, Mobin Bagherian, Amirmohammad Izadi, and Mahdieh Soleymani Baghshah. Mechanistic interpretability of large-scale counting in llms through a system-2 strategy. *arXiv preprint arXiv:2601.02989*, 2026.
- [13] Harold Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of educational psychology*, 24(6):417, 1933.
- [14] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.
- [15] Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The platonic representation hypothesis. *arXiv preprint arXiv:2405.07987*, 2024.
- [16] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, 55(12), March 2023. ISSN 0360-0300. doi: 10.1145/3571730. URL <https://doi.org/10.1145/3571730>.
- [17] Adam Tauman Kalai, Ofir Nachum, Santosh S Vempala, and Edwin Zhang. Why language models hallucinate. *arXiv preprint arXiv:2509.04664*, 2025.
- [18] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. URL <https://arxiv.org/abs/2001.08361>.

- [19] Nora Kassner, Oyvind Tafjord, Hinrich Schütze, and Peter Clark. BeliefBank: Adding memory to a pre-trained language model for a systematic notion of belief. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 8849–8861, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.697. URL <https://aclanthology.org/2021.emnlp-main.697/>.
- [20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [21] Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. Halueval: A large-scale hallucination evaluation benchmark for large language models. *arXiv preprint arXiv:2305.11747*, 2023.
- [22] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 41451–41530. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/81b8390039b7302c909cb769f8b6cd93-Paper-Conference.pdf.
- [23] Yuxiao Li, Alina Fastowski, Efstratios Zaradoukas, Bardh Prenkaj, and Gjergji Kasneci. Analysing the safety pitfalls of steering vectors. *arXiv preprint arXiv:2603.24543*, 2026.
- [24] Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets. *arXiv preprint arXiv:2310.06824*, 2023. URL <https://arxiv.org/abs/2310.06824>.
- [25] Javier Marín. A geometric taxonomy of hallucinations in llms, 2026. URL <https://arxiv.org/abs/2602.13224>.
- [26] Callum McDougall, Arthur Conmy, Cody Rushing, Thomas McGrath, and Neel Nanda. Copy suppression: Comprehensively understanding an attention head. *arXiv preprint arXiv:2310.04625*, 2023. URL <https://arxiv.org/abs/2310.04625>.
- [27] Alex McKenzie, Keenan Pepper, Stijn Servaes, Martin Leitgab, Murat Cubuktepe, Mike Vaiana, Diogo de Lucena, Judd Rosenblatt, and Michael SA Graziano. Endogenous resistance to activation steering in language models. *arXiv preprint arXiv:2602.06941*, 2026.

- [28] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://arxiv.org/abs/2202.05262>.
- [29] Aayush Mishra, Daniel Khashabi, and Anqi Liu. Steered llm activations are non-surjective. *arXiv preprint arXiv:2604.09839*, 2026.
- [30] Andrew Nam, Henry Conklin, Yukang Yang, Thomas Griffiths, Jonathan Cohen, and Sarah-Jane Leslie. Causal head gating: A framework for interpreting roles of attention heads in transformers, 2025. URL <https://arxiv.org/abs/2505.13737>.
- [31] Jord Nguyen, Khiem Hoang, Carlo Leonardo Attubato, and Felix Hofstätter. Probing and steering evaluation awareness of language models, 2025. URL <https://arxiv.org/abs/2507.01786>.
- [32] Xinchu Qiu, Lei Yu, Yuchen Zhang, Aobo Yang, Narine Kokhlikyan, Nicola Cancedda, Diego Garcia-Olano, et al. Hallucination reduction with causal: Contrastive activation steering for amortized learning. *arXiv preprint arXiv:2510.02324*, 2025.
- [33] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023. URL <https://arxiv.org/abs/2305.18290>.
- [34] Delip Rao, Eric Wong, and Chris Callison-Burch. Detecting and correcting reference hallucinations in commercial llms and deep research agents. *arXiv preprint arXiv:2604.03173*, 2026.
- [35] Joshua Fonseca Rivera. Training introspective behavior: Fine-tuning induces reliable internal state detection in a 7b model. *arXiv preprint arXiv:2511.21399*, 2025.
- [36] Basavaraj Sangayya Hiremath, Marco Polignano, Marco Levantesi, Giovanni Semeraro, Ernesto William De Luca, and Amos Poznanski. State-wise linear modulation (slim): A novel approach for steering large language models. *Neural Networks*, 199:108708, 2026. ISSN 0893-6080. doi: <https://doi.org/10.1016/j.neunet.2026.108708>. URL <https://www.sciencedirect.com/science/article/pii/S089360802600170X>.
- [37] Aruna Sankaranarayanan, Amir Zur, Atticus Geiger, and Dylan Hadfield-Menell. Activation steering via generative causal mediation. *arXiv preprint arXiv:2602.16080*, 2026.

- [38] Mrinank Sharma, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Askill, Samuel R Bowman, Newton Cheng, Esin Durmus, Zac Hatfield-Dodds, Scott R Johnston, et al. Towards understanding sycophancy in language models. *arXiv preprint arXiv:2310.13548*, 2023.
- [39] Qingru Shi, Mingyang Chen, Jianfeng Liu, Tuo Zhao, and Haoming Wang. Tell your model where to attend: Post-hoc attention steering for LLMs. *arXiv preprint arXiv:2311.02262*, 2023. URL <https://arxiv.org/abs/2311.02262>.
- [40] Yitong Shou and Manhao Guan. Mechanistic decoding of cognitive constructs in llms. *arXiv preprint arXiv:2604.14593*, 2026.
- [41] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.
- [42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. URL <https://arxiv.org/abs/1706.03762>.
- [43] Xin Wang, Hong Chen, Si’ao Tang, Zihao Wu, and Wenwu Zhu. Disentangled representation learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):9677–9696, 2024.
- [44] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35: 24824–24837, 2022.
- [45] Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. A survey on knowledge distillation of large language models. *arXiv preprint arXiv:2402.13116*, 2024.
- [46] Aojie Yuan and Zhiyuan Su. Beyond language: Format-agnostic reasoning subspaces in large language models. *arXiv preprint arXiv:2605.09496*, 2026.
- [47] Fujie Zhang, Peiqi Yu, Biao Yi, Baolei Zhang, Tong Li, and Zheli Liu. Prompt-guided internal states for hallucination detection of large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21806–21818, 2025.
- [48] Shaolei Zhang, Tian Yu, and Yang Feng. Truthx: Alleviating hallucinations by editing large language models in truthful space. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8908–8949, 2024.

- [49] Zhenliang Zhang, Xinyu Hu, Huixuan Zhang, Junzhe Zhang, and Xiaojun Wan. ICR probe: Tracking hidden state dynamics for reliable hallucination detection in LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17986–18002, 2025.

Appendices

Appendix A

Layers' study

The following section presents additional preliminary studies conducted on the layers; specifically, it analyzes the behavior of the layers based on other parameters

A.1 F1Score

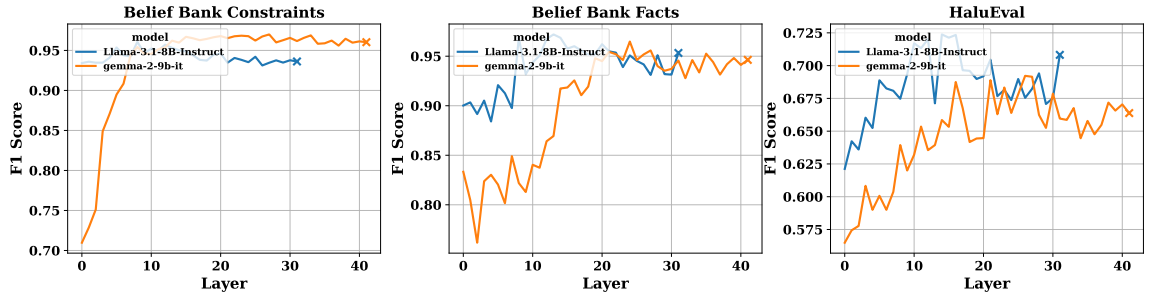


Figure A.1: F1Score comparison of attention modules across the datasets

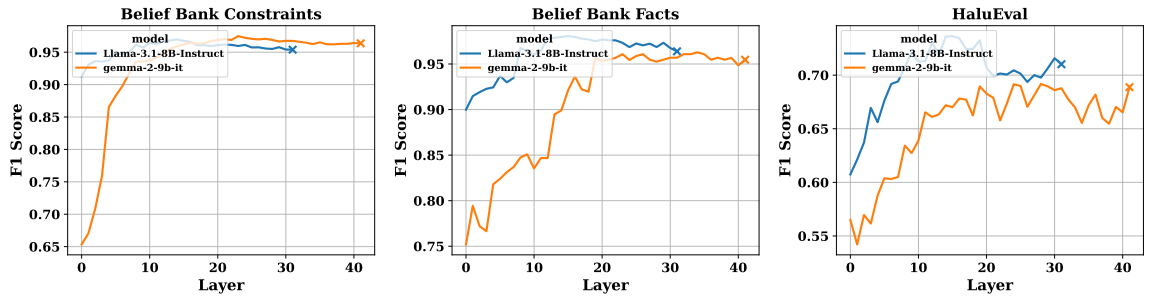


Figure A.2: F1Score comparison of hidden modules across the datasets

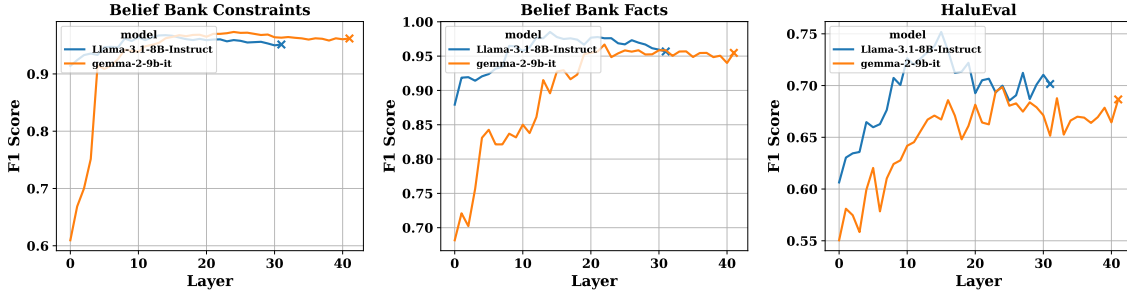


Figure A.3: F1Score comparison of mlp modules across the datasets

A.2 Auroc Score

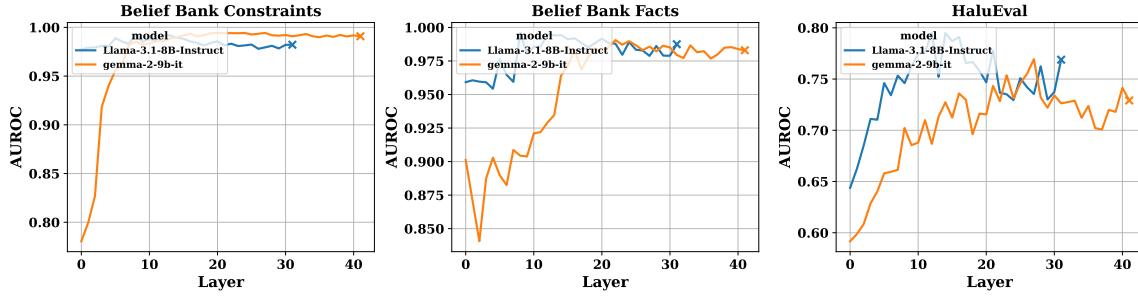


Figure A.4: Auroc comparison of attention modules across the datasets

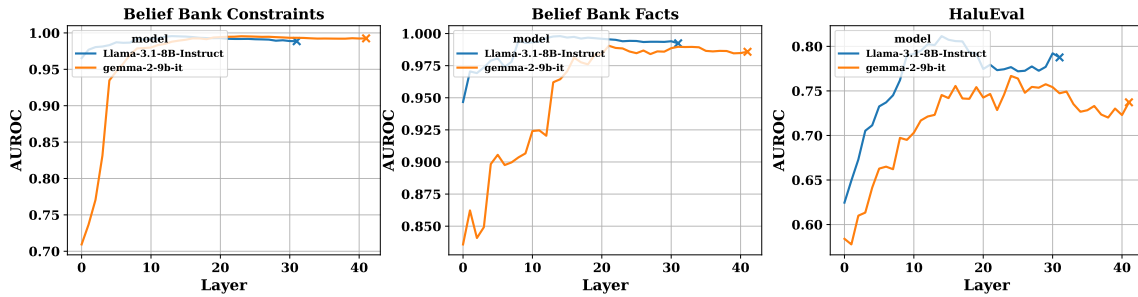


Figure A.5: Auroc comparison of hidden modules across the datasets

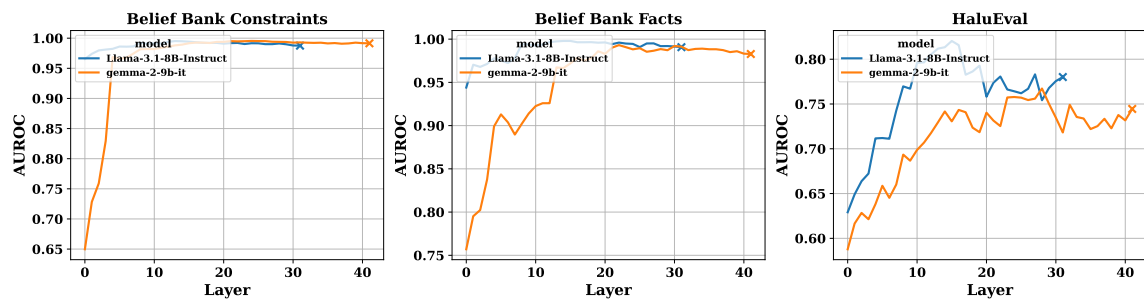


Figure A.6: Auroc comparison of mlp modules across the datasets

Appendix B

Error type analysis

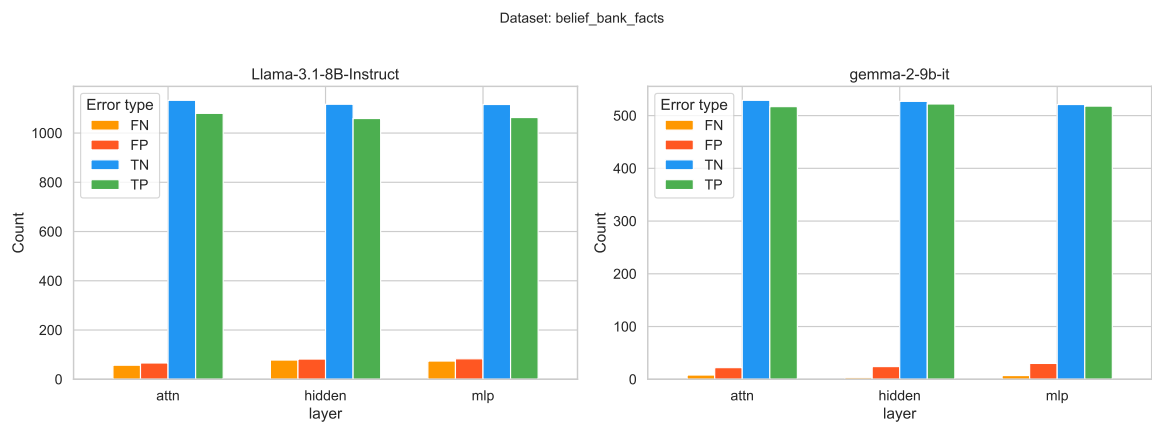


Figure B.1: Error Type Breakdown for the One-For-All on BeliefBankFacts

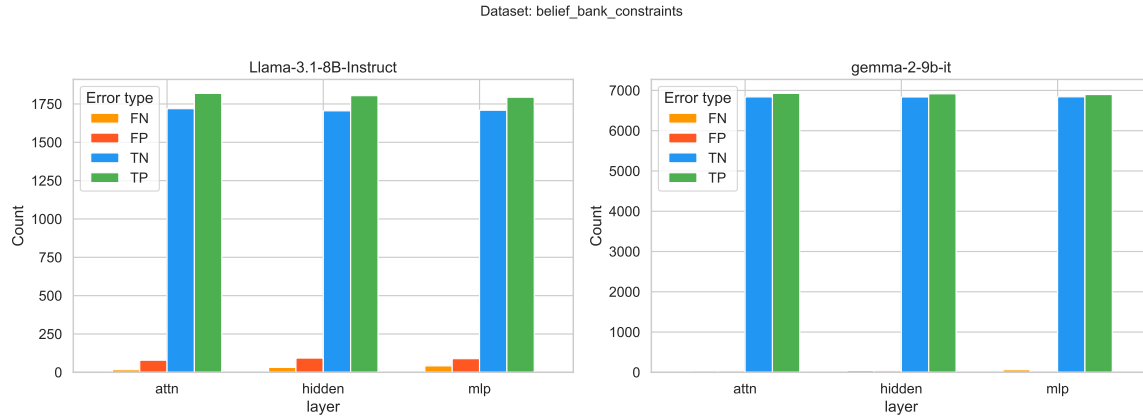


Figure B.2: Error Type Breakdown for the One-For-All on BeliefBankConstraints

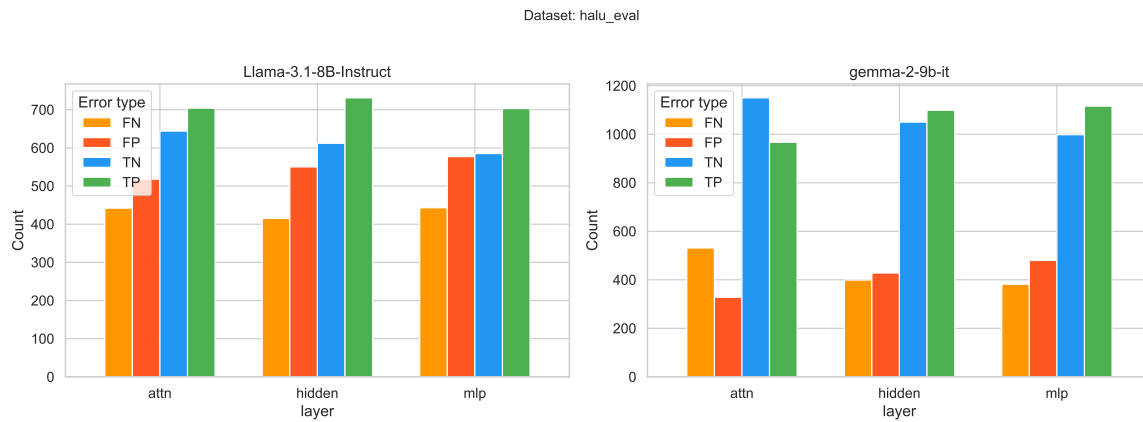


Figure B.3: Error Type Breakdown for the One-For-All on HaluEval

Chapter 6

Ringraziamenti

Lavora sodo, studia bene, e mangia e dormi a sufficienza.



Da un grande potere derivano grandi responsabilità.



Queste sono le frasi che mi hanno accompagnato fin dall'infanzia e a cui ho cominciato a dare un vero significato all'inizio di questo percorso universitario durato 5 anni. Chi mi conosce sa che da diversi anni sto affrontando un percorso di crescita personale affiancato dalla mia psicologa la dottoressa Caterina Armandi (che approfitto per ringraziare per tutto il supporto e l'aiuto che mi ha dato e sta continuando a darmi) dato che dall'ultimo anno delle superiori avevo iniziato ad annullarmi completamente per dedicarmi esclusivamente a uno studio matto e disperato (Giacomo un saluto) il che mi ha portato a diversi ricoveri in ospedale. E qui arriva la prima delle due citazioni: "è importante lavorare sodo ma allo stesso tempo è importante prendersi cura di se stessi, capire quando è il momento di fermarsi per poi poter ripartire con più forza". Solo di recente ho iniziato ad applicare questa filosofia di vita ed effettivamente mi ha aiutato a migliorare tantissimo.

Inizierei, o meglio continuerei, i ringraziamenti proseguendo con il mio supervisor, il professor Gennaro Vessio, che si è sempre dimostrato disponibile a una chiacchierata con gli studenti (che si trattasse di approfondire concetti visti a lezione o discutere di film, videogiochi e serie tv) e che finalmente è riuscito a spiegarmi in maniera comprensibile la **backpropagation** (la bestia nera del deep learning). Il secondo ringraziamento va ovviamente a quella povera ragazza che è stata costretta a vedersi arrivare anche 10 messaggi alle 02:00 di notte su Teams di me che alla fine facevo ragionamenti tra me e me / riportavo ogni minimo dubbio/idea che mi veniva in mente e a cui a volte pure rispondeva facendomi pensare "Bruh ma sono le 2 di notte, ma dormi! Perché stai ancora al PC" (cit.) ovvero la mia co-supervisor, la PhD

student Lucrezia Laraspata. Un saluto e un ringraziamento forse un pò atipico va a mio co-supervisor della tesi triennale, il Dr. Giuseppe Spillo con cui ancora oggi sono in contatto e con cui ho avuto modo di discutere di diversi argomenti, non solo accademici, e che mi ha sempre dato dei consigli preziosi (ricorda che è "colpa tua" se ho preso la magistrale e se ho iniziato ad appassionarmi al mondo della ricerca, mi hai sulla coscienza 😊).

Prima di ringraziare amici e famiglia vorrei discutere la seconda citazione. Da piccolo, credo come tutti, ho sempre pensato che il concetto di potere fosse legato agli assurdi superpoteri dei supereroi ma crescendo ho capito che quel potere si riferisce alla semplice possibilità e capacità di fare qualcosa. Molte volte mi è capitato di chiedere qualcosa nei gruppi universitari e ogni volta vedevo decine (se non un centinaio) di persone visualizzare il messaggio senza rispondere ed ero tipo "Zì se non lo sai scrivi non lo so, non ti costa nulla". Una situazione assurda che capita molto spesso e fa innervosire (non solo me). Ecco, in questo caso il potere è quello di aiutare qualcuno che magari è in difficoltà o che ha un dubbio e che magari non riesce a risolvere da solo, e la responsabilità è quella di rispondere. Da quel momento ho sempre cercato di rispondere a tutti i messaggi in quei gruppi il che mi ha portato anche a conoscere varie persone di cui una che spero sia qui presente, il mio "bro del secondo anno" Giuseppe Cannone che ho sempre avuto piacere di aiutare e con cui ho avuto diverse chiacchierate assurde e divertenti. Ovviamente questo dei messaggi universitari è un esempio scemo ma può far capire quanto a volte anche un piccolo gesto come rispondere a un messaggio possa essere importante per qualcuno.

Passiamo ora agli amici. Premessa: i seguenti ringraziamenti non sono in ordine di importanza, sono completamente a caso. Vorrei ringraziare tutte le persone conosciute durante questi anni con cui ho condiviso tanti momenti che mi hanno portato a crescere sia come persona sia come studente. Ringrazio quindi Giuditta con cui ho condiviso tante conversazioni interessanti, stimolanti che mi hanno portato a riflettere su diversi argomenti e che mi hanno aiutato a crescere come persona (ma quel meme Black Humour a cui seguirà la tua ramanzina sui diritti umani ci sarà sempre ❤️). Ringrazio Elisa e i suoi assurdi dolci (specialmente le torte alle mele) con cui ho sempre avuto piacere di chiacchierare di qualsiasi cosa: anime, videogiochi, film, serie tv ma anche robe più serie. Inoltre ringrazio ancora te e il tuo ragazzo Davide per avermi aiutato (ceh di fatto avete fatto tutto voi) nella realizzazione degli assurdi ciondoli che saranno (o sono già stati) distribuiti con la bomboniera. Un ringraziamento speciale va a Stefano, credo la prima persona con cui ho cominciato a legare in Università e con cui ho passato un'assurda quantità di ore sugli pseudo-codici di Abba in chiamata su Discord. Mi è dispiaciuto aver perso un pò i contatti tra il terzo anno della triennale e il primo della magistrale ma sono contento di aver riallacciato i rapporti con te (che sei diventato il mio spacciatore di sticker) [si amici, ogni volta che mi mando un assurdo sticker molto SUS sappiate che il fornitore è lui]. Vorrei ringraziare Claudio e Riccardo (e mia sorella, a cui tornerò più avanti) con cui ho trascorso vari weekend nei primi anni, quando non conoscevo ancora nessuno a Bari: grazie per avermi accolto e fatto sentire meno solo ❤️. Vorrei ringraziare Antonio che ho praticamente conosciuto

nell'ultimo anno e mezzo nonostante frequentassimo gli stessi corsi dalla triennale, la quale era troppo affollata e non sono riuscito a spottare quel bro in grado di riconoscere la citazione che richiedere un contratto a tempo indeterminato di doomscrolling sui social, ma fortunatamente l'ambiente ristretto della magistrale ha permesso la "Clown to Clown telepathy" 🤪. E soprattutto qualcuno che riconosca che il finale di The Boys faccia schifo. A proposito del finale di The Boys, Mario io ancora ora non riesco a capire come tu abbia potuto apprezzarlo dato che era la cosa più becera e assurda che potessero fare, ma da uno che continua a ripetermi che "Anime, Videogiochi e Supereroi sono cose da bambini" e poi guarda Amici non potevo aspettarmi altrimenti. Ovviamente sai che nonostante tutto ti considero uno dei miei più cari amici fin da quanto ti ho conosciuto. Ne approfitto per ringraziare Damiano e Giuseppe che, insieme a Mario, Gianni e altri, mi hanno accolto nel loro gruppo di amici ormai 10 anni fa. Anche se non ne capisco molto è sempre un piacere chiacchierare con voi "alla zona" e imparare qualcosa di nuovo sul mondo dei motori (per poi scordarmi tutto dopo 5 minuti 🤪). Un altro ringraziamento va a Gianni che è stata una delle prime persone che ho conosciuto alle superiori e con cui ho molti ricordi: le partite a Rainbow Six Siege appena tornati a casa dalle lezioni, tutti i colpi al casinò su GTA Online, ogni volta che ti aiutavo con lo studio e finivo per addormentarmi sulla scrivania (ops 🤪). E soprattutto ricorda: è grazie a noi che il Red Carpet campa. Un ringraziamento speciale va anche a Gabriel, anche lui uno dei primi amici con cui ho stretto amicizia alle superiori (mi ricordo ancora quando eri più basso di me). Ricorderò sempre con piacere le assurde partite a Rocket, Fortnite e Brawlhalla 🔥. Continuando con gli amici di Matera ringrazio Antonio con cui ho convissuto 6 mesi durante il primo anno di triennale e che è sempre ultra disponibile per qualsiasi cosa. Sappi che mi porterò i tuoi meme nella tomba (sottilezza e piadina verde, per intenderci). Ringrazio anche Nunzio con cui ho condiviso molti momenti: da chiacchierate molto personali al cantare a sguarciagola in macchina: White Girl Music sempre e comunque <3. Ringrazio Luigi (detto Cocco Luigi) che ho conosciuto durante l'ultimo anno di triennale ma che ormai si è unito al gruppo degli amici di Matera. Sappi (ma te l'ho già detto) che sei la prima persona che mi ha fatto tornare a Monte dopo i miei trascorsi con quel paese 🤪. Ti ringrazio per tutti i discorsi motivazionali e per avermi aiutato nei momenti più duri (anche se continuerò sempre a baitarti con i miei soliti discorsi). Ringrazio Donato con cui ho condiviso tutte le superiori e tutta l'università, di cui 4 anni da coinquilini. Grazie per le serate in tavernetta con gli altri e per tutti i piatti incredibili che ci preparavi a casa. Ringrazio Michele con cui, anche se un pò a tratti, sono in contatto praticamente dalle medie (penso che non mi scorderò mai quel ragazzino che trova 50€ fuori dalla scuola vicino al bidone della spazzatura). Un ringraziamento va anche a Leonardo, Vito e Mario che ho conosciuto durante questi anni di università e con cui ho trascorso un numero assurdo di ore a giocare 🔥 (grazie Lello per l'introduzione a Devil May Cry, grazie Vito per tutte le partite [e le sfuriate assurde] su CS:GO). Mario a te voglio dire una semplice frase: "DAJE ROMA DAJEEEE". Ringrazio inoltre Michele che non è qui con noi perchè sta in Austria in Erasmus, che è quel bro che vedi una volta ogni 6 mesi,

che ha i tempi di risposta su "WhatsApp" che si attestano sui 3/4 giorni lavorativi, ma che quando finalmente vedi ti diverti un sacco 😄. Fin da quando ci siamo conosciuti alle superiori ti ho sempre considerato uno dei miei amici più cari a cui voglio un mondo di bene (anche se all'inizio non mi potevi sopportare). Ti ringrazio per non avermi lasciato solo durante questi 5 anni universitari in cui fisicamente ci siamo allontanati un sacco (dato che hai scelto Torino per studiare) e per avermi presentato Pier e Dario con cui ho subito trovato intesa nel parlare. Vorrei ringraziare anche il gruppo di Dokkan: so che non siete qui presenti (dato che siamo sparsi in tutta Italia) ma siete persone che benomale sento ogni giorno e che spero di incontrare di persona prima o poi: grazie a voi che mi aiutate sempre (ormai l'avete capito che sono un nabbo) e con cui condivido la passione per DragonBall in generale. Riprendendo con gli Uni-Boys ringrazio Angelo che ho conosciuto solo di recente grazie al progetto di Ingegneria (team Nygaard sempre). Ringrazio Nicolò detto NicHOLA per tutte le chiacchierate (soprattutto le svariate lamentele sull'Università). Ringrazio Reb con cui ho condiviso diversi momenti di studio e di pranzi a mensa. Ringrazio Mattia e Flavio con cui ho condiviso anche tanti pranzi e cene a mensa, con cui ho avuto modo di parlare tantissime volte di svariati argomenti ma soprattutto li ringrazio per aver partecipato con me al Campionato Universitario AI di cui abbiamo vinto la tappa regionale portando avanti il nome dei "PizzaMensaBoys - A ThreeMoonsSpinOff". Parlando di ThreeMoons: inizio col ringraziare Mattia Curri detto Furry o Curry o Furby (potrei continuare per non so quanto) con cui condivido ormai casa a Bari da 4 anni e che ogni volta si subiva le mie assurde esposizioni sul programma di studio che avrei dovuto fare in giornata con annesso di tutte lamentele (riconosco inutili 😄) da parte mia. Grazie per avermi sopportato in questi anni e per essere stato sempre disponibile. Spero di continuare a condividere tanti momenti insieme anche dopo la laurea (soprattutto dobbiamo arrivare almeno Champions su Rocket). Ringrazio Michele che ho conosciuto durante il percorso di magistrale e con cui ho trovato subito intesa, con cui ho passato tante giornate tra studio e partite assurde (JoJo sound effects su Black Ops 🧟) e con cui ho scoperto di avere alcuni interessi in comune (banalmente qualsiasi roba divulgativa scientifica). Ringrazio Andrea che è il mio punto di riferimento dal punto di vista accademico e allo stesso tempo un grande amico con cui condivido ormai tutte le giornate in chiamata. Grazie per essere sempre disponibile per una partita, per una chiacchierata o per un supporto in un momento di difficoltà. Anche se fattelo dire: i tuoi cambi di umore e di pensiero sono veramente assurdi, andrebbero studiati con una SFT con window di 5 minuti per capire come funzionano (sappi che nell'esatto momento in cui sto scrivendo questo ringraziamento ti stai lamentando in chiamata dell'esame di CV nonostante ti stiamo ripetendo da 1 mese di non preoccuparti). Ringrazio Rossella. **GRAZIE** per avermi finalmente dato qualcuno con cui giocare a FighterZ (1v1 ti spacco con Goku Ultra Istinto 🍌). Grazie per avermi accolto nel vostro server e avermi fatto conoscere tantissime altre persone come Pier (penso una delle persone più simpatiche e assurde che abbia mai conosciuto), Franco (top1 persone brainrottate, Franco stai proprio cucinato ma ti vogliamo bene comunque), Vincenzo (che conoscevo già ma con cui ho avuto modo

di parlare di più ultimamente) e gli altri con cui parlo quotidianamente su Discord. Ultimo, ma non per importanza, ringrazio Mauro: anche lui sempre pronto per una partita o un'assurda sessione in chiamata, nonché compagno affidabile nelle giornate di studio. Ti ho voluto piazzare per ultimo non perchè sei meno importante ma perchè volevo scrivere quanti più **assurdo** possibili prima di nominarti (pls non trasformarti in TirannoSurdo).

Ora credo sia il momento di ringraziare la mia famiglia. Il primo pensiero va ai miei nonni paterni Carmela detta "Melina" e Emanuele che ormai non ci sono più ma che sono sicuro che da lassù mi stiano guardando e siano orgogliosi di me (anche perchè di sicuro la Juventus non sta giocando ora quindi non c'è nonno seduto sulla poltrona a sentire la partita alla radio con nonna pronto a sgridarlo). Ringrazio i miei nonni materni, Grazia e Emanuele per esserci sempre e per supportarmi negli studi (anche se credo non abbiano capito molto di quello che ho fatto in questi anni, ma va bene così) [a mala pena l'ho capito io]. Ringrazio zio Tonio (che al 90% non è qui ora) con cui sono cresciuto da piccolo: è grazie a te se mi sono appassionato a DragonBall, ai supereroi e ai videogiochi quindi è anche grazie a te se sono diventato quello che sono oggi. Ringrazio i miei zii Imma e Salvatore e la piccola Alma (la mia cuginetta a cui sono sempre pronto a dare una mano per sconfiggere il boss di qualche videogioco) che anche se non ci vediamo spesso so che ci sono sempre (zio sappi che se non dovessi trovare un bel lavoro sarai il primo che contatterò dato che mi proponi sempre robe incredibili). Ringrazio i miei zii Gianni, Maria, Tina, Eustachio (detto Uccio) e i miei cugini Carmen, Francesca e David con cui ho sempre piacere nel parlare e nel confrontare i diversi percorsi di studio che abbiamo intrapreso. Un ringraziamento va anche a Zia Maria, che tutti noi consideriamo come una terza nonna: da sempre mi hai sempre insegnato che lo studio è importante, che (come fai tu nonostante l'età) non è mai troppo tardi per imparare qualcosa di nuovo e che bisogna sempre essere curiosi. Grazie per avermi sempre supportato e per essere sempre lì per me. Infine, ma non per importanza, ringrazio i miei genitori, Angelo e Bruna, che sono sempre lì per me pronti ad aiutarmi e a supportarmi in qualsiasi cosa che vada da preoccupazioni, a problemi di salute o anche economici [dannata AdiSu che mi toglie la borsa da fuorisede]. Grazie per aver sempre creduto in me, per aver sempre approvato le mie scelte di studio e le mie richieste (incredibile che dopo 3 tatuaggi e 1 orecchino io non sia ancora stato diseredato 😂😂😂). Infine (se io sto leggendo e non state registrando mi dispiace per voi perchè non lo ripeterò mai più) ringrazio la persona più importante della mia vita, mia sorella Vanessa che è sempre stata al mio fianco, nel bene e nel male, nelle litigate e nei momenti di divertimento, nei momenti di difficoltà e in quelli di gioia. Grazie per essere sempre lì per me, per supportarmi e per sopportarmi. Anche se dico sempre che "tanto pure che sei tornata a casa la differenza è minima" sappi che non è vero. Ogni volta che sei a casa e vengo a disturbarti è perchè voglio passare un pochino di tempo con te ❤️.



Al contrario dell'ultima volta questo giro ho voluto scrivere dei ringraziamenti come si deve cercando di usare l'ironia che da sempre mi contraddistingue. Una piccola frase al me del passato: "So che ti senti diverso dagli altri, sbagliato o strano, ma sappi che andrà meglio, che incontrerai persone che ti capiranno e che condividono le tue stesse passioni. Però potevi pure studiare di meno e uscire di più, un pò te la sei cercata. E inoltre non dare retta alle persone (coff coff professore delle superiori di cui non farò il nome coff coff) quando ti dicono che non sei un bravo studente, che hai solo fortuna: ad oggi tu hai 2 paper pubblicati mentre loro zero (mic drop 🎤)

DISCLAIMER: Se non sei una persona che trascorre un tempo incredibilmente lungo sui social probabilmente mi darai per malato o deviato per alcuni di questi: sono MEME



Mi identifico

